

Operationalizing Secure-by-Design AI Through Deterministic Runtime Governance

John M. Willis
Sustainable Future Tech, Inc.
research@agcp.ai

June 18, 2026
Version 1.0

Abstract

This paper argues that Secure-by-Design AI requires more than lifecycle governance, model oversight, and post-hoc assurance. As AI systems become increasingly autonomous, delegated, and operationally consequential, existing frameworks often define governance objectives and control expectations while providing limited guidance regarding how governance should be evaluated at the point where proposed actions transition into execution.

The paper identifies this gap as the absence of an execution-layer governance architecture. It introduces runtime governance as a deterministic execution-time architecture that evaluates admissibility, validates authority and evidence, preserves canonical state, enforces structural refusal, and maintains governance continuity at the commit boundary. In this model, governance becomes an operational system responsible for determining whether a specific autonomous action remains legitimate under current conditions before execution occurs.

Rather than competing with existing frameworks, runtime governance is presented as a complementary implementation layer that operationalizes the objectives of the NIST AI Risk Management Framework (AI RMF), NIST Cybersecurity Framework (CSF) 2.0, NIST Special Publication 800-53, CISA Secure-by-Design guidance, MITRE ATLAS, OWASP Agentic AI guidance, and related assurance frameworks. The paper develops a conceptual runtime governance architecture, defines core execution-time governance concepts, and examines how deterministic governance, replayability, lifecycle integrity, and runtime assurance can support accountability, auditability, resilience, and operational control.

The central claim is that the emerging challenge for AI governance is not only governing models, but governing actions at runtime. As autonomous systems become increasingly capable of consequential action, execution-time governance may become a necessary architectural layer for Secure-by-Design AI.

Keywords: Runtime Governance, Secure-by-Design AI, Execution-Layer Governance, Agentic AI, Autonomous Systems, AI Governance, Deterministic Governance, Runtime Assurance, Operational Assurance, Critical Infrastructure Protection.

Suggested Citation

Willis JM. *Operationalizing Secure-by-Design AI Through Deterministic Runtime Governance*. Sustainable Future Tech, Inc.; 2026. Zenodo. DOI: <https://doi.org/10.5281/zenodo.20749457>

Contents

Executive Summary	13
1 Introduction	15
2 Terminology Used in This Paper	17
2.1 Runtime Governance	17
2.2 Admissibility Evaluation	17
2.3 Agentic System	17
2.4 Autonomous Action	17
2.5 Canonical State	17
2.6 Commit Boundary	17
2.7 Deterministic Governance	18
2.8 Evidence Continuity	18
2.9 Execution-Bound Authorization	18
2.10 Governance Continuity	18
2.11 Governance Lifecycle Integrity	18
2.12 Governance Replayability	18
2.13 Runtime Assurance	18
2.14 Runtime Governance Architecture	19
2.15 Structural Refusal	19
3 Principles of Runtime Governance	20
3.1 Governance at the Point of Action	20
3.2 Continuous Admissibility	20
3.3 Execution-Bound Authorization	20
3.4 Complete Governance Mediation	20
3.5 Canonical State Preservation	20
3.6 Evidence-Based Governance	20
3.7 Commit-Boundary Governance	21
3.8 Structural Refusal by Default	21
3.9 Deterministic Governance Behavior	21
3.10 Governance Continuity	21
3.11 Accountability Through Traceability	21
4 Core Assertions	22
5 Scope and Non-Claims	23
6 The Runtime Governance Gap	24
6.1 Current AI Governance Approaches	24
6.2 Current AI Security Approaches	25
6.3 The Persistence of Model-Centric Thinking	26
6.4 Limitations of Lifecycle-Centric Governance	27
6.4.1 Pre-Deployment Controls	27
6.4.2 Static Authorization Models	27
6.4.3 Post-Execution Auditing	27
6.5 Governance Failure at Operational Commitment	28

6.6	The Emerging Need for Runtime Governance	29
6.7	Applicability of Runtime Governance	29
7	Existing Governance Frameworks Remain Essential	31
7.1	Introduction	31
7.2	Contributions of Existing Governance Frameworks	31
7.3	The NIST AI Risk Management Framework	31
7.4	The NIST Cybersecurity Framework	32
7.5	NIST SP 800-53 Security and Privacy Controls	33
7.6	CISA Secure-by-Design Principles	34
7.7	MITRE ATLAS and Adversarial Threat Modeling	34
7.8	OWASP Agentic AI Guidance	35
7.9	Governance, Security, and Assurance Are Distinct Concerns	35
7.10	Runtime Governance as a Complementary Layer	36
8	Emerging Security Risks in Agentic AI	38
8.1	Introduction	38
8.2	Autonomous Tool Use	38
8.3	Multi-Agent Coordination	39
8.4	Delegated Authority	40
8.5	Prompt Injection and Instruction Manipulation	41
8.6	Context Manipulation	41
8.7	Authority Escalation	42
8.8	Cross-System Orchestration	43
8.9	Operational Drift	44
8.10	The Common Theme: Operational Legitimacy	44
8.11	Implications for Governance Architecture	45
9	Runtime Governance Principles	47
9.1	Introduction	47
9.2	Governance Objectives	47
9.2.1	Accountability	47
9.2.2	Traceability	48
9.2.3	Operational Assurance	48
9.2.4	Authorization Integrity	49
9.2.5	Governance Integrity	49
9.2.6	Safety Preservation	50
9.2.7	Replayability	50
9.3	Runtime Governance Mechanisms	50
9.3.1	Admissibility Evaluation	50
9.3.2	Execution-Bound Authorization	51
9.3.3	Authority Containment	52
9.3.4	Structural Refusal	52
9.3.5	Canonical-State Governance	53
9.3.6	Evidence Continuity	53
9.3.7	Governance Context Preservation	54
9.3.8	Governance Isolation	54
9.3.9	Governance Replayability	55

	9.3.10	Governance Lifecycle Integrity	55
	9.3.11	Enforcement Integrity	56
	9.3.12	Deterministic Governance	56
	9.3.13	Deterministic Governance and Evidence Uncertainty	57
9.4		Relationships Among Runtime Governance Mechanisms	57
9.5		From Principles to Runtime Assurance	58
10		Runtime Assurance and Operational Legitimacy	59
10.1		Introduction	59
10.2		The Evolution of Assurance	59
10.3		Operational Legitimacy Versus System Trustworthiness	60
10.4		The Need for Continuous Governance Evaluation	61
10.5		Commit-Time Governance	62
10.6		Canonical State and Operational Reality	63
10.7		Evidence Validity and Evidence Continuity	64
10.8		Governance Continuity Across Autonomous Workflows	65
10.9		Runtime Assurance and Structural Refusal	65
10.10		Deterministic Runtime Assurance	66
10.11		Runtime Assurance as an Architectural Discipline	67
10.12		From Runtime Assurance to Runtime Governance Controls	67
11		Runtime Governance Control Categories	69
11.1		Introduction	69
11.2		Identity Controls	69
11.3		Authorization Controls	70
11.4		State Controls	71
11.5		Evidence Controls	72
11.6		Execution Controls	73
11.7		Assurance Controls	74
11.8		Control Interdependencies	75
11.9		Relationship to Existing Security Controls	75
11.10		A Layered Runtime Governance Model	76
11.11		From Control Categories to Secure-by-Design Objectives	77
12		Secure-by-Design Mapping	78
12.1		Introduction	78
12.2		Secure Defaults	78
12.3		Least Privilege	79
12.4		Complete Mediation	79
12.5		Accountability	80
12.6		Auditability	81
12.7		Traceability	81
12.8		Resilience	81
12.9		Safe Failure	82
12.10		Human Accountability and Oversight	82
12.11		Recovery and Forensic Reconstruction	83
12.12		Summary Mapping	83
12.13		Runtime Governance as a Secure-by-Design Layer	84

13	Mapping to Existing Standards and Frameworks	85
13.1	Introduction	85
13.2	NIST AI Risk Management Framework (AI RMF)	85
13.2.1	Overview	85
13.2.2	Runtime Governance Contribution	85
13.2.3	Remaining Runtime Governance Gap	86
13.3	NIST Cybersecurity Framework 2.0	87
13.3.1	Overview	87
13.3.2	Runtime Governance Contribution	87
13.3.3	Remaining Runtime Governance Gap	87
13.4	NIST Special Publication 800-53	88
13.4.1	Overview	88
13.4.2	Runtime Governance Contribution	89
13.4.3	Remaining Runtime Governance Gap	89
13.5	ISO/IEC 42001 Artificial Intelligence Management Systems	89
13.5.1	Overview	89
13.5.2	Runtime Governance Contribution	90
13.5.3	Remaining Runtime Governance Gap	90
13.6	MITRE ATLAS	91
13.6.1	Overview	91
13.6.2	Runtime Governance Contribution	91
13.6.3	Remaining Runtime Governance Gap	92
13.7	OWASP Agentic AI Guidance	92
13.7.1	Overview	92
13.7.2	Runtime Governance Contribution	92
13.7.3	Remaining Runtime Governance Gap	93
13.8	CISA AI Security Guidance	94
13.8.1	Overview	94
13.8.2	Runtime Governance Contribution	94
13.8.3	Remaining Runtime Governance Gap	95
13.9	European Union Artificial Intelligence Act	95
13.9.1	Overview	95
13.9.2	Runtime Governance Contribution	95
13.9.3	Remaining Runtime Governance Gap	96
13.10	Cross-Framework Observations	96
13.11	Standards Alignment Rather Than Standards Replacement	97
14	Reference Runtime Governance Architecture	99
14.1	Introduction	99
14.2	Architectural Design Objectives	99
14.3	Conceptual Architecture Overview	99
14.4	Core Architectural Components	101
14.4.1	Identity and Context Services	101
14.4.2	Policy and Constraint Services	102
14.4.3	Evidence Services	102
14.4.4	State Services	103
14.4.5	Governance Evaluation Services	103
14.4.6	Commit Adjudication Services	103

	14.4.7	Governance Record Services	104
14.5		A Five-Stage Governance Pipeline	104
	14.5.1	Stage 1: Identity and Context	105
	14.5.2	Stage 2: Action Representation	105
	14.5.3	Stage 3: Rule Evaluation	105
	14.5.4	Stage 4: Commit Adjudication	106
	14.5.5	Stage 5: Governance Record Generation	106
14.6		Commit Boundary Placement	107
14.7		Centralized and Distributed Implementations	108
	14.7.1	Centralized Governance	108
	14.7.2	Distributed Governance	108
14.8		Multi-Agent Governance Considerations	109
14.9		Example Implementation Approaches	109
14.10		From Architecture to Operational Use Cases	110
15		Critical Infrastructure Use Cases	111
15.1		Introduction	111
15.2		Energy Sector	111
	15.2.1	Scenario	111
	15.2.2	Runtime Evaluation	111
	15.2.3	Authorization Decision	112
	15.2.4	Execution Outcome	112
	15.2.5	Auditability Outcome	112
15.3		Water Sector	112
	15.3.1	Scenario	112
	15.3.2	Runtime Evaluation	113
	15.3.3	Authorization Decision	113
	15.3.4	Execution Outcome	113
	15.3.5	Auditability Outcome	113
15.4		Transportation Sector	113
	15.4.1	Scenario	113
	15.4.2	Runtime Evaluation	114
	15.4.3	Authorization Decision	114
	15.4.4	Execution Outcome	114
	15.4.5	Auditability Outcome	114
15.5		Financial Services Sector	115
	15.5.1	Scenario	115
	15.5.2	Runtime Evaluation	115
	15.5.3	Authorization Decision	115
	15.5.4	Execution Outcome	115
	15.5.5	Auditability Outcome	115
15.6		Healthcare Sector	116
	15.6.1	Scenario	116
	15.6.2	Runtime Evaluation	116
	15.6.3	Authorization Decision	116
	15.6.4	Execution Outcome	116
	15.6.5	Auditability Outcome	116
15.7		Government Operations	117

15.7.1	Scenario	117
15.7.2	Runtime Evaluation	117
15.7.3	Authorization Decision	117
15.7.4	Execution Outcome	117
15.7.5	Auditability Outcome	117
15.8	Cross-Sector Observations	118
15.9	A Generalized Runtime Governance Pattern	118
15.10	Implications for Critical Infrastructure	120
16	Runtime Governance Requirements	121
16.1	Introduction	121
16.2	Requirement Organization	121
16.3	RG-1 Lifecycle Governance	122
16.4	RG-2 Execution Governance	123
16.5	RG-3 Governance Object Integrity	124
16.6	RG-4 Canonical-State Governance	125
16.7	RG-5 Refusal and Escalation Governance	125
16.8	RG-6 Governance Evidence and Receipts	126
16.9	RG-7 Multi-Agent Runtime Governance	127
16.10	RG-8 Governance Interoperability	127
16.11	RG-9 Runtime Governance Metrology	128
16.12	RG-10 Continuation Integrity Governance	129
16.13	RG-11 Composite Governance	130
16.14	RG-12 Governance Compilation and Constitutional Governance	131
16.15	Conceptual Mapping to Runtime Governance Principles	131
16.16	Requirements as an Example Runtime Governance Specification	132
17	Implementation Considerations	134
17.1	Introduction	134
17.2	Centralized Governance Architectures	134
17.3	Distributed Governance Architectures	136
17.4	Federated Governance Architectures	137
17.5	Multi-Tenant Environments	137
17.6	Agent-to-Agent Governance	138
17.7	Human-in-the-Loop and Human-on-the-Loop Models	139
17.7.1	Human-in-the-Loop	139
17.7.2	Human-on-the-Loop	140
17.8	Governance Performance Considerations	141
17.9	Governance Availability and Resilience	142
17.10	Data Protection and Privacy Considerations	142
17.11	Governance Assurance and Independent Validation	143
17.12	Adoption Pathways	144
17.13	Implementation as an Engineering Discipline	145
17.14	Summary	146
18	Future Research Directions	148
18.1	Introduction	148
18.2	Runtime Governance for Autonomous Agents	148

18.3	Multi-Agent Governance Systems	149
18.4	Cross-Domain Delegation and Authority Management	149
18.5	Governance Interoperability Standards	150
18.6	Machine-Verifiable Governance	150
18.7	Formal Verification of Governance Behavior	151
18.8	Deterministic Governance and Reproducibility	152
18.9	Runtime Assurance Methodologies	152
18.10	Governance Metrology	152
18.11	Runtime Governance for Physical Systems	153
18.12	Constitutional and Policy-Constrained AI Systems	154
18.13	Governance as a First-Class Architectural Discipline	154
18.14	Toward a Science of Runtime Governance	155
18.15	Transition to Conclusion	156
19	Conclusion	157
	Author Biography	161
	Copyright Notice	162
	Trademark Notice	162
A	Runtime Governance Terminology Crosswalk	163
A.1	Purpose	163
A.2	Core Runtime Governance Terminology	163
A.3	Relationship Among Key Concepts	165
A.4	Terminology and Framework Neutrality	166
B	Runtime Governance Control Categories	167
B.1	Purpose	167
B.2	Runtime Governance Control Categories	167
B.3	Control Categories and Secure-by-Design Objectives	168
B.4	Control Categories and Runtime Governance Functions	168
B.5	Relationship to Runtime Governance Architectures	169
B.6	Summary	170
C	Runtime Governance Concept Mapping to AGCP Requirement Groups	171
C.1	Purpose	171
C.2	Concept-to-Requirement Group Mapping	171
C.3	AGCP Requirement Group Summary	172
C.4	Concept Coverage Across Requirement Groups	173
C.5	Conceptual Layering	174
C.6	Relationship to Secure-by-Design Objectives	174
C.7	Illustrative Characteristics of AGCP	175
C.8	Summary	176
D	Secure-by-Design Runtime Governance Mapping	177
D.1	Purpose	177
D.2	Secure-by-Design Objective Mapping	177

D.3	Runtime Governance Contribution by Secure-by-Design Principle	178
D.3.1	Secure Defaults	178
D.3.2	Least Privilege	178
D.3.3	Complete Mediation	179
D.3.4	Accountability	179
D.3.5	Auditability	179
D.3.6	Resilience	180
D.3.7	Human Oversight	180
D.4	Secure-by-Design and the Runtime Governance Gap	180
D.5	Illustrative Mapping to Runtime Governance Control Categories	181
D.6	Summary	181
E	Standards and Framework Crosswalk	183
E.1	Purpose	183
E.2	Crosswalk Overview	184
E.3	NIST AI Risk Management Framework	184
E.4	NIST Cybersecurity Framework 2.0	185
E.5	NIST SP 800-53	185
E.6	CISA Secure-by-Design	186
E.7	MITRE ATLAS	186
E.8	OWASP Agentic AI	186
E.9	ENISA AI Security Guidance	187
E.10	ISO/IEC 42001	187
E.11	Framework Coverage Perspective	188
E.12	Illustrative Coverage Model	189
E.13	Summary	189
F	Reference Runtime Governance Architecture	190
F.1	Purpose	190
F.2	Reference Architecture Overview	190
F.3	Architectural Components	191
F.3.1	Requesting Actor	191
F.3.2	Autonomous Agent	192
F.3.3	Proposed Action	192
F.3.4	Runtime Governance Layer	193
F.4	Illustrative Governance Evaluation Pipeline	193
F.5	Commit Boundary	194
F.6	Operational System	194
F.7	Governance Records and Receipts	194
F.8	Runtime Assurance Functions	195
F.9	Relationship to Secure-by-Design	196
F.10	Relationship to Runtime Governance Concepts	196
F.11	Reference Architecture Variations	197
F.12	Architectural Implications for Autonomous Systems	197
F.13	Summary	198
G	Example Runtime Governance Sequence	199
G.1	Purpose	199

G.2	Scenario Overview	199
G.3	Illustrative Runtime Governance Sequence	199
G.4	Step 1: Proposed Action Generation	201
G.5	Step 2: Governance Context Collection	201
G.6	Step 3: Evidence Validation	202
G.7	Step 4: Canonical-State Validation	202
G.8	Step 5: Authorization Validation	202
G.9	Step 6: Admissibility Evaluation	203
G.10	Step 7: Refusal or Escalation Decision	203
G.11	Step 8: Commit Evaluation	204
G.12	Step 9: Commit Boundary Crossing	204
G.13	Step 10: Operational Execution	204
G.14	Step 11: Governance Receipt Generation	205
G.15	Step 12: Runtime Assurance Activities	205
G.16	Governance Properties Demonstrated	205
G.17	Summary	206

List of Figures

1	Illustrative relationship between lifecycle governance activities and execution-time runtime governance activities.	37
2	Illustrative runtime governance control hierarchy.	76
3	Comparison of traditional execution workflows, agentic execution workflows, and runtime-governed execution workflows.	100
4	Illustrative placement of runtime governance controls relative to the commit boundary.	107
5	Generalized runtime governance sequence across critical infrastructure use cases. . .	119
6	Illustrative centralized governance model.	135
7	Reference Runtime Governance Architecture	191
8	Illustrative Runtime Governance Sequence	200

List of Tables

1	Conceptual Runtime Governance Applicability Spectrum	30
2	Illustrative Mapping Between Secure-by-Design Objectives and Runtime Governance Capabilities	84
3	NIST AI RMF and Runtime Governance	86
4	NIST CSF and Runtime Governance	88
5	AGCP Runtime Governance Requirement Groups	122
6	Mapping of Runtime Governance Concepts to AGCP Requirement Groups	132
7	Core Runtime Governance Terminology	163
8	Runtime Governance Control Categories	167
9	Runtime Governance Control Categories and Secure-by-Design Objectives	169
10	Concept-to-Requirement Group Mapping	171
11	AGCP Requirement Groups	173
12	Illustrative Governance Layering	174
13	Secure-by-Design Objective Mapping	177
14	Secure-by-Design Objectives and Runtime Governance Control Categories	181
15	Framework Alignment Summary	184
16	NIST AI RMF and Runtime Governance	184
17	NIST CSF 2.0 and Runtime Governance	185
18	Representative NIST SP 800-53 Alignment	185
19	CISA Secure-by-Design and Runtime Governance	186
20	MITRE ATLAS and Runtime Governance	186
21	OWASP Agentic AI and Runtime Governance	186
22	ENISA AI Security Guidance and Runtime Governance	187
23	ISO/IEC 42001 and Runtime Governance	188
24	Illustrative Governance Layering Across Frameworks	189
25	Reference Architecture Components and Runtime Governance Concepts	196
26	Runtime Governance Concepts Demonstrated by the Example	206

Executive Summary

Artificial intelligence governance has matured rapidly over the past decade. Governments, standards bodies, industry consortia, and research organizations have developed increasingly sophisticated approaches for managing the risks associated with AI systems. Frameworks such as the NIST AI Risk Management Framework, ISO/IEC 42001, the Executive Order on Safe, Secure, and Trustworthy AI, MITRE ATLAS, and numerous sector-specific governance initiatives have established important foundations for responsible AI deployment (NIST, 2023a,b, 2024b; ISO, 2023; Executive Office of the President, 2023; MITRE, 2024; DHS, 2024; CISA, 2023).

These frameworks generally emphasize governance activities occurring throughout the system lifecycle. Common focus areas include model development, dataset quality, security testing, validation, transparency, documentation, monitoring, auditing, incident response, and organizational accountability (NIST, 2023a,b, 2024b; ISO, 2023; NIST, 2024a, 2020a; DHS, 2024; CISA, 2023). Such controls remain critically important and should continue to serve as foundational elements of trustworthy AI programs.

However, the emergence of agentic and autonomous AI systems introduces a class of governance challenges that are not fully addressed through lifecycle-oriented controls alone. Increasingly, AI systems are capable of invoking tools, coordinating with other agents, initiating transactions, accessing operational systems, modifying state, and triggering consequential actions without direct human involvement in every decision cycle (Russell, 2019; Amodei et al., 2016; OpenAI, 2023; Park et al., 2023; Wang et al., 2023; Xi et al., 2023).

As autonomy increases, governance concerns shift from merely understanding how a model was trained to determining whether a specific action should be permitted at the moment of execution. A model may be technically valid, appropriately tested, and deployed within policy, yet still propose actions that are inappropriate, unsafe, unauthorized, or inconsistent with current operational conditions. In such circumstances, governance effectiveness depends not only on development-time controls but also on execution-time decision mechanisms (Russell, 2019; Amodei et al., 2016; Wang et al., 2023; Xi et al., 2023; NIST, 2020b).

This paper uses the term *runtime governance* to describe a category of architectural capabilities that evaluate proposed actions against policy, operational state, evidence, authority, constraints, and contextual conditions immediately prior to execution. Rather than focusing exclusively on what a system is permitted to do in theory, runtime governance focuses on whether a specific action remains legitimate under current operational conditions. The need for such execution-oriented governance emerges naturally from advances in autonomous agents, AI safety research, organizational governance requirements, and operational decision systems (Wang et al., 2023; Xi et al., 2023; Anthropic, 2022; Defense Innovation Board, 2019; NSCAI, 2021).

Several emerging security challenges illustrate the need for runtime governance. Agentic systems may coordinate activities across multiple platforms, operate under delegated authority, interact with external services, and adapt behavior in response to changing context. These capabilities create opportunities for prompt injection, authority escalation, context manipulation, unintended delegation, operational drift, and cross-system propagation of errors (Russell, 2019; Amodei et al., 2016; Leike et al., 2018; OpenAI, 2023; Park et al., 2023; Wang et al., 2023; Xi et al., 2023). In many cases, traditional governance artifacts such as policies, approvals, or model documentation provide limited protection once an autonomous action is already in progress.

The runtime governance model proposed in this paper includes mechanisms such as admissibility evaluation, execution-bound authorization, structural refusal, evidence validation, canonical-state verification, governance continuity, and replayable governance records. These concepts are not presented as proprietary capabilities nor as replacements for existing governance frameworks. Rather, they represent architectural patterns derived from established principles of authorization, systems security engineering, resiliency engineering, and continuous operational assessment (NIST, 2018b, 2020a,b, 2021, 2018a, 2022; OASIS, 2013).

The paper further argues that runtime governance aligns naturally with Secure-by-Design objectives. Concepts such as least privilege, complete mediation, accountability, traceability, resilience, and auditability have long been recognized as foundational principles of cybersecurity engineering and systems assurance (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2018b,a, 2022; CNSS, 2022). Runtime governance extends these principles into autonomous operational environments by providing mechanisms that continuously evaluate the legitimacy of actions as they transition from proposal to execution.

Importantly, this paper does not argue that existing governance frameworks are inadequate or obsolete. Existing frameworks remain essential and provide the foundation upon which runtime governance capabilities can be constructed. Instead, the paper proposes that runtime governance represents an additional architectural layer that may become increasingly necessary as AI systems acquire greater autonomy and operational influence.

To support this position, the paper introduces a conceptual runtime governance model, maps runtime governance concepts to established standards and guidance, examines critical-infrastructure use cases, and proposes implementation-oriented requirements for execution-time governance controls. The paper further explores architectural patterns and governance mechanisms that may enable organizations to preserve policy intent, accountability, and operational control as AI systems assume increasing levels of autonomy.

The architectural focal point of runtime governance is the commit boundary: the transition point at which proposed actions become operational reality. The concepts developed throughout this paper—including admissibility evaluation, execution-bound authorization, structural refusal, evidence continuity, and governance continuity—are ultimately concerned with determining whether an action should be permitted to cross this boundary.

The commit boundary therefore represents the primary locus of governance authority within autonomous systems, since it is the point at which proposed intent becomes operational reality.

The future challenge for AI governance is no longer solely whether organizations can govern AI models. Increasingly, the challenge is whether organizations can govern autonomous actions at runtime. As AI systems become more deeply integrated into critical infrastructure, operational decision-making, and cross-domain workflows, execution-time governance controls may become an essential component of Secure-by-Design AI architectures (CISA, 2023; DHS, 2024; NIST, 2021; NSCAI, 2021).

1 Introduction

Artificial intelligence governance has advanced substantially through risk-management frameworks, cybersecurity guidance, assurance practices, and emerging regulatory approaches. Frameworks such as the NIST Artificial Intelligence Risk Management Framework, NIST Cybersecurity Framework 2.0, ISO/IEC 42001, CISA Secure-by-Design guidance, MITRE ATLAS, OWASP Agentic AI guidance, and sector-specific critical-infrastructure frameworks provide important foundations for trustworthy AI development, deployment, monitoring, and organizational accountability (NIST, 2023b, 2024a; ISO, 2023; CISA, 2023; MITRE, 2024; OWASP, 2025; DHS, 2024).

These frameworks remain essential. They help organizations identify risk, establish governance processes, define security objectives, assess controls, manage incidents, and improve assurance across the system lifecycle (NIST, 2023a, 2020a, 2022, 2018a,b, 2021). However, the increasing autonomy of AI systems introduces a governance problem that is not fully addressed by lifecycle governance, model evaluation, policy documentation, or post-execution audit alone. Agentic and autonomous systems may invoke tools, coordinate with other agents, access operational systems, initiate transactions, modify state, or trigger physical-world effects with limited direct human involvement in each decision cycle (Russell, 2019; Amodei et al., 2016; Park et al., 2023; Wang et al., 2023; Xi et al., 2023).

As a result, the governance question increasingly shifts from whether an AI system was responsibly developed or appropriately deployed to whether a specific proposed action should be permitted to execute under current operational conditions. A system may be trained, tested, approved, and deployed within policy while still proposing an action that is unauthorized, unsafe, contextually inappropriate, or inconsistent with current operational state. This creates a gap between governance intent and operational execution.

This paper identifies that gap as the absence of an explicit execution-layer governance architecture. Existing frameworks define important governance objectives and control expectations, but they generally leave implementation-specific the question of how governance should be evaluated at the point where a proposed autonomous action becomes operationally consequential.

This paper argues that the central governance challenge of autonomous systems is determining what controls should govern the commit boundary, the transition point at which proposed actions become operational reality. As autonomous systems become increasingly capable of consequential action, governance effectiveness depends upon the ability to evaluate legitimacy, authority, evidence, constraints, and operational context immediately before execution occurs.

At or immediately before the commit boundary, governance decisions must determine whether execution remains legitimate, authorized, admissible, and supported by valid evidence.

The paper uses the term *runtime governance* to describe architectural mechanisms that evaluate proposed actions during operational execution. Runtime governance includes capabilities such as admissibility evaluation, execution-bound authorization, canonical-state validation, evidence continuity, structural refusal, governance continuity, lifecycle integrity, deterministic decision behavior, and governance replayability. These mechanisms are not presented as replacements for existing governance frameworks. Rather, they are presented as execution-time mechanisms that can help operationalize existing Secure-by-Design, risk-management, cybersecurity, and assurance objectives (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2018b, 2021; CISA, 2023).

The central thesis of this paper is that Secure-by-Design AI increasingly requires governance mech-

anisms that operate before autonomous actions produce operational consequences. In this view, governance is not only a policy function, compliance function, or lifecycle-management activity. It also becomes a runtime architectural capability that mediates whether specific actions may proceed.

The paper makes four primary contributions. First, it defines a vocabulary for runtime governance, including admissibility, canonical state, commit boundary, execution-bound authorization, structural refusal, governance continuity, and governance replayability. Second, it explains how runtime governance concepts relate to established Secure-by-Design principles such as least privilege, complete mediation, accountability, auditability, resilience, and safe failure. Third, it maps runtime governance concepts to existing standards and frameworks, including NIST AI RMF, NIST CSF 2.0, NIST SP 800-53, CISA Secure-by-Design guidance, MITRE ATLAS, OWASP Agentic AI guidance, and related assurance frameworks. Fourth, it presents a reference runtime governance architecture, critical-infrastructure use cases, and implementation-oriented requirements for execution-time governance controls.

The concepts presented throughout this paper should not be interpreted as features of a particular product, implementation, or governance platform. Rather, they are presented as elements of a broader architectural category concerned with governing autonomous actions during execution. In the same way that organizations recognize disciplines such as identity governance, access governance, data governance, and security governance, increasingly autonomous systems may require a distinct discipline focused on execution-time governance. This paper refers to that category as runtime governance.

The remainder of the paper is organized as follows. Section 2 defines the terminology used throughout the paper. Subsequent sections examine the runtime governance gap, explain why existing governance frameworks remain essential, analyze emerging security risks in agentic AI, and develop runtime governance principles. Later sections examine runtime assurance, control categories, Secure-by-Design mappings, standards alignment, reference architecture patterns, critical-infrastructure use cases, implementation requirements, and future research directions. The appendices provide terminology crosswalks, control-category mappings, AGCP requirement-group mappings, Secure-by-Design mappings, standards crosswalks, a reference architecture, and an illustrative runtime governance sequence.

2 Terminology Used in This Paper

This section establishes the terminology used throughout the paper. The definitions provided are intended to create a consistent vocabulary for discussing runtime governance concepts and do not imply endorsement of any specific implementation approach.

2.1 Runtime Governance

Runtime governance refers to the evaluation, mediation, authorization, constraint enforcement, and accountability mechanisms applied to proposed actions during operational execution. Runtime governance focuses on actions as they transition from intent to execution rather than solely on lifecycle governance activities performed during development, deployment, or post-hoc auditing.

2.2 Admissibility Evaluation

Admissibility evaluation refers to the process of determining whether a proposed action satisfies the conditions necessary for execution under current operational circumstances. Admissibility may consider policy constraints, authorization status, evidence availability, operational context, safety requirements, and system state.

2.3 Agentic System

For purposes of this paper, an agentic system refers to an AI-enabled system capable of independently selecting actions, invoking tools, coordinating tasks, interacting with external systems, or pursuing goals with limited direct human intervention.

2.4 Autonomous Action

An autonomous action refers to an action initiated or selected by a system without direct human approval of that specific action at the time of execution.

2.5 Canonical State

Canonical state refers to the authoritative representation of operational conditions used for governance decision-making. Canonical state may include system status, environmental conditions, governance context, delegated authority relationships, operational constraints, and other information necessary to evaluate proposed actions consistently.

2.6 Commit Boundary

A commit boundary represents the point at which a proposed action transitions into an operationally consequential action. Governance controls operating at or immediately before the commit boundary evaluate whether the action should be permitted to affect systems, resources, or state.

2.7 Deterministic Governance

Deterministic governance refers to governance processes that produce consistent outcomes when presented with equivalent inputs, evidence, policies, and operational conditions. Deterministic governance supports reproducibility, auditability, assurance, and verification of governance behavior.

2.8 Evidence Continuity

Evidence continuity refers to the preservation and availability of the evidence supporting governance decisions across the lifecycle of an action. Evidence continuity enables later validation of why a governance decision was reached and whether supporting conditions remained valid.

2.9 Execution-Bound Authorization

Execution-bound authorization refers to authorization decisions that remain explicitly linked to a specific proposed action and its execution context. Authorization is evaluated in relation to the action being performed rather than relying solely on static permissions established at an earlier point in time.

2.10 Governance Continuity

Governance continuity refers to the preservation of governance intent, accountability, constraints, and authorization semantics throughout multi-step workflows, delegated operations, and multi-agent interactions.

2.11 Governance Lifecycle Integrity

Governance lifecycle integrity refers to the preservation of traceable relationships among proposals, evaluations, authorization decisions, execution events, governance records, and resulting state transitions throughout the lifecycle of an action.

2.12 Governance Replayability

Governance replayability refers to the ability to reconstruct and reproduce governance decisions using retained governance records, evidence, state information, and decision logic. Replayability supports auditing, investigation, assurance, and forensic analysis.

2.13 Runtime Assurance

Runtime assurance refers to the capability to determine whether a system remains within authorized operational conditions at the time an action is proposed or executed. Runtime assurance extends traditional assurance concepts into dynamic operational environments where context, authority, and system state may continuously change.

2.14 Runtime Governance Architecture

A runtime governance architecture is an architectural arrangement of governance services, controls, records, and decision mechanisms designed to mediate operational actions during execution. Runtime governance architectures may be centralized, distributed, federated, or embedded within autonomous systems.

2.15 Structural Refusal

Structural refusal refers to governance mechanisms that prevent execution when admissibility conditions are not satisfied. Structural refusal is distinguished from discretionary or advisory responses because it prevents operational commitment rather than merely recommending alternative behavior.

3 Principles of Runtime Governance

The runtime governance concepts introduced throughout this paper are guided by several foundational architectural principles. These principles are intended to describe the characteristics of a governance architecture capable of evaluating and controlling autonomous actions at execution time.

3.1 Governance at the Point of Action

Governance should be evaluated at the point where proposed actions transition into execution. Governance effectiveness depends not only on development-time controls, policies, and approvals, but also on the ability to determine whether a specific action remains legitimate under current operational conditions.

3.2 Continuous Admissibility

Authorization alone is insufficient for autonomous operation. Actions should remain subject to admissibility evaluation throughout execution, with governance decisions incorporating current state, context, evidence, constraints, and operational conditions.

3.3 Execution-Bound Authorization

Authorization should be bound to specific actions, contexts, authorities, and execution conditions. Governance decisions should not rely solely on static permissions established before deployment.

3.4 Complete Governance Mediation

Every autonomous action crossing a governance boundary should be subject to governance evaluation. No action should bypass required governance controls through workflow design, delegation chains, tool invocation paths, or architectural shortcuts.

3.5 Canonical State Preservation

Governance decisions should be based upon an authoritative representation of operational state. Governance architectures should preserve state integrity and prevent decisions from being made using stale, incomplete, or inconsistent information.

3.6 Evidence-Based Governance

Governance decisions should be supported by verifiable evidence. Authorization, admissibility, assurance, and accountability depend upon the ability to demonstrate the basis upon which governance decisions were made.

3.7 Commit-Boundary Governance

Governance authority becomes operationally consequential at the commit boundary, where proposed actions transition into execution. Runtime governance architectures should therefore ensure that admissibility, authorization, evidence validation, and policy evaluation occur before actions are permitted to cross this boundary.

3.8 Structural Refusal by Default

When governance requirements cannot be satisfied, execution should not proceed. Failure to establish admissibility, authority, evidence continuity, policy compliance, or governance integrity should result in refusal rather than speculative execution.

3.9 Deterministic Governance Behavior

Governance decisions should be reproducible given the same inputs, policies, state, and evidence. Deterministic behavior supports auditability, replayability, assurance, and independent validation.

3.10 Governance Continuity

Governance should persist throughout the operational lifecycle of an autonomous action. Authorization, admissibility, evidence, and accountability relationships should remain traceable across delegation, orchestration, workflow execution, and state transitions.

3.11 Accountability Through Traceability

Governance decisions should be attributable, auditable, and reconstructable. Organizations should be able to determine what decision was made, why it was made, under whose authority it was made, and what evidence supported the decision.

Together, these principles establish a conceptual foundation for runtime governance architectures. Subsequent sections examine how these principles address emerging governance challenges associated with increasingly autonomous and agentic AI systems.

4 Core Assertions

The central arguments developed throughout this paper may be summarized through the following assertions. Together, these assertions define the conceptual foundation of deterministic runtime governance and motivate the architectural mechanisms discussed in subsequent sections.

1. **Autonomous systems introduce governance challenges that cannot be addressed solely through development-time, deployment-time, or post-execution controls.**

Lifecycle governance remains essential; however, increasing autonomy, delegation, orchestration, and operational decision-making create governance requirements that emerge during execution.

2. **Governance effectiveness increasingly depends upon the ability to evaluate proposed actions immediately prior to execution using current operational conditions.**

Decisions that were appropriate at deployment may no longer remain appropriate when operational state, environmental conditions, available evidence, delegated authority, or policy constraints have changed.

3. **Authorization and admissibility are distinct governance concerns.**

Authorization determines whether an actor possesses the authority to perform an action. Admissibility determines whether execution remains appropriate under current circumstances. An action may remain authorized while no longer being admissible.

4. **The commit boundary represents the primary locus of governance authority within autonomous systems because it is the point at which proposed intent becomes operational reality.**

The transition between proposed intent and operational execution represents the point at which governance decisions become operationally consequential. Runtime governance therefore focuses on evaluating legitimacy at the commit boundary before execution occurs.

5. **Runtime governance complements rather than replaces existing governance, security, assurance, and Secure-by-Design frameworks.**

Runtime governance is not intended to supplant lifecycle governance, risk management, security engineering, safety engineering, or organizational oversight. Instead, it provides an execution-time implementation layer that operationalizes governance objectives during operation.

6. **Deterministic runtime governance can improve accountability, auditability, replayability, and operational assurance.**

By preserving governance evidence, maintaining canonical state, enforcing execution-bound authorization, and supporting reproducible decision processes, deterministic governance architectures may provide stronger foundations for operational trust and assurance.

The remainder of this paper develops the architectural concepts, governance mechanisms, and implementation considerations associated with these assertions and examines how runtime governance may contribute to the realization of Secure-by-Design AI systems.

5 Scope and Non-Claims

This paper introduces deterministic runtime governance as an execution-time governance architecture for autonomous and agentic AI systems. The scope of the paper is limited to governance mechanisms that evaluate proposed actions immediately prior to execution and determine whether those actions remain legitimate under current operational conditions.

The objective is not to replace existing governance, security, assurance, safety, or risk-management frameworks. Rather, the paper examines how execution-time governance mechanisms may complement those frameworks by operationalizing governance decisions at the point where actions transition into execution.

Accordingly, this paper does not make the following claims:

- Runtime governance does not replace lifecycle governance activities such as system design, risk assessment, model validation, testing, deployment review, monitoring, auditing, or organizational oversight.
- Runtime governance does not replace existing governance frameworks including the NIST AI Risk Management Framework (AI RMF), NIST Cybersecurity Framework (CSF) 2.0, NIST SP 800-53, ISO/IEC 42001, CISA Secure-by-Design guidance, MITRE ATLAS, OWASP guidance, or related standards and best practices.
- Runtime governance does not eliminate security, safety, reliability, operational, or organizational risks associated with autonomous systems.
- Runtime governance does not guarantee correctness of AI-generated outputs, plans, recommendations, or decisions.
- Runtime governance does not eliminate the need for security engineering, safety engineering, assurance engineering, human oversight, incident response, or operational monitoring.
- Deterministic governance does not guarantee that governance decisions are correct. Determinism ensures that governance behavior is reproducible and auditable given the same inputs, policies, state, and evidence.

Instead, the central claim of this paper is that increasingly autonomous systems introduce governance challenges that emerge during operation and therefore require governance mechanisms capable of evaluating admissibility, authority, evidence, and policy compliance immediately prior to execution.

Within this context, runtime governance is presented as a complementary architectural layer that helps operationalize Secure-by-Design objectives by providing execution-time evaluation, execution-bound authorization, structural refusal, governance continuity, and accountability mechanisms at the commit boundary.

The concepts presented throughout this paper should therefore be interpreted as complementary to existing governance, security, assurance, and risk-management practices rather than as substitutes for them.

6 The Runtime Governance Gap

Runtime governance is best understood as an architectural category rather than a specific implementation approach. Multiple governance architectures, control models, policy frameworks, and technical implementations may exist within this category. In the same way that organizations recognize disciplines such as identity governance, access governance, data governance, and security governance, increasingly autonomous systems may require a distinct governance discipline focused on evaluating and controlling actions during execution.

6.1 Current AI Governance Approaches

Over the past decade, artificial intelligence governance has evolved from a largely theoretical discipline into a structured collection of frameworks, standards, regulatory approaches, and operational practices. Governments, standards bodies, industry organizations, and private enterprises have invested substantial effort in developing governance mechanisms intended to promote trustworthy, secure, explainable, and accountable AI systems (NIST, 2023b; ISO, 2023; Executive Office of the President, 2023; DHS, 2024; CISA, 2023).

Contemporary governance frameworks generally emphasize lifecycle management. Governance activities are typically organized around phases such as system design, model development, data acquisition, validation, deployment, monitoring, incident response, and retirement (NIST, 2023b; ISO, 2023; NIST, 2020a). These activities support important objectives including transparency, accountability, fairness, robustness, security, compliance, and risk management.

Examples include the NIST Artificial Intelligence Risk Management Framework (AI RMF), which structures AI risk activities around the Govern, Map, Measure, and Manage functions (NIST, 2023b). Similarly, ISO/IEC 42001 establishes organizational management-system requirements for AI governance and oversight (ISO, 2023). The European Union Artificial Intelligence Act introduces risk-based obligations for providers and deployers of AI systems (European Union, 2024). Additional guidance from CISA, MITRE, OWASP, and sector-specific organizations contributes further controls intended to improve AI security and operational trustworthiness (CISA, 2023; MITRE, 2024; OWASP, 2025).

These governance initiatives have produced significant advances. Organizations today possess substantially more guidance regarding model validation, security testing, documentation, monitoring, accountability, and risk management than was available only a few years ago. Consequently, many governance failures that previously occurred during development or deployment can now be identified and addressed through structured governance processes.

Nevertheless, most existing governance approaches share a common characteristic: they focus primarily on the lifecycle of an AI system rather than the governance of individual operational actions during execution.

This observation should not be interpreted as a criticism of current frameworks. Rather, it reflects the historical context in which many governance approaches were developed. Traditional machine learning systems typically produced recommendations, classifications, predictions, or analytical outputs that remained subject to human review before operational consequences occurred (Russell, 2019; Amodei et al., 2016; OpenAI, 2023). In such environments, governance effectiveness depended largely on the quality of models, datasets, testing procedures, and organizational oversight.

As AI systems acquire increasing autonomy, however, governance challenges begin to shift toward operational execution itself.

6.2 Current AI Security Approaches

A similar pattern is evident within AI security practices. Existing AI security programs typically emphasize activities such as:

- Secure development practices
- Model security testing
- Adversarial robustness evaluation
- Data protection controls
- Supply-chain security
- Access management
- Monitoring and detection
- Incident response
- Compliance verification

These controls address many important threats. Organizations routinely perform penetration testing, red teaming, adversarial testing, vulnerability management, model validation, and security reviews before systems are placed into production (NIST, 2018b, 2020b, 2012; MITRE, 2024; CISA, 2023).

Recent guidance from organizations including CISA, NIST, MITRE, OWASP, Microsoft, Google, Anthropic, and OpenAI has expanded awareness of AI-specific attack vectors such as prompt injection, model poisoning, indirect prompt attacks, tool abuse, and unauthorized data access (CISA, 2023; MITRE, 2024; OWASP, 2025; Anthropic, 2022; OpenAI, 2023).

However, most security controls remain focused on either:

1. preventing vulnerabilities from entering a system, or
2. detecting undesirable behavior after execution has occurred.

A comparatively smaller body of guidance addresses how security and governance controls should mediate operational decisions immediately before consequential actions are executed.

This distinction becomes increasingly important in environments where AI systems are capable of initiating transactions, invoking tools, modifying state, coordinating workflows, or interacting with critical infrastructure.

In such environments, the relevant security question is often no longer:

Was the system developed securely?

Instead, the more consequential question may be:

Should this specific action be permitted to execute under current operational conditions?

6.3 The Persistence of Model-Centric Thinking

Many existing governance and security approaches remain rooted in what may be described as *model-centric thinking*. Under this perspective, governance effectiveness is evaluated primarily through characteristics of the model itself, including:

- accuracy
- robustness
- fairness
- explainability
- transparency
- security
- compliance

These characteristics remain important and should continue to receive substantial attention. However, increasing autonomy introduces additional governance questions that cannot be answered solely through model evaluation.

A model may exhibit excellent performance characteristics while still proposing actions that are inappropriate within a particular operational context. Likewise, a system may satisfy organizational policy requirements while simultaneously attempting an action that is inconsistent with current environmental conditions, delegated authority constraints, or mission objectives.

For example, a model may correctly identify a maintenance condition, recommend an operational adjustment, or determine that a financial transaction appears justified. Yet whether the resulting action should actually be executed may depend upon contextual factors that extend beyond the model itself, including:

- current system state
- operational constraints
- authority relationships
- concurrent activities
- safety conditions
- regulatory restrictions

- human oversight requirements
- mission priorities

Consequently, governance effectiveness increasingly depends not only upon model quality but also upon mechanisms that evaluate proposed actions within their operational context.

6.4 Limitations of Lifecycle-Centric Governance

Lifecycle-oriented governance remains necessary but may be insufficient for increasingly autonomous environments.

Several limitations become apparent when governance controls are concentrated primarily before deployment or after execution.

6.4.1 Pre-Deployment Controls

Pre-deployment controls evaluate anticipated system behavior prior to operational use. Examples include testing, validation, certification, security reviews, and governance approvals.

These activities establish confidence that a system is generally suitable for deployment. However, they cannot anticipate every operational condition that may arise during execution (NIST, 2023b; ISO, 2023; NIST, 2020a).

As environments become dynamic, authorization decisions frequently depend upon circumstances that emerge after deployment.

6.4.2 Static Authorization Models

Many existing systems rely on authorization decisions established before execution occurs. Such decisions may grant access rights, permissions, capabilities, or delegated authority based upon anticipated operating conditions.

Static authorization approaches function effectively when environments remain relatively predictable. Autonomous systems, however, may encounter conditions that differ substantially from those assumed when permissions were originally granted (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2018b).

The legitimacy of an action may therefore depend upon factors that can only be evaluated at execution time.

6.4.3 Post-Execution Auditing

Auditing remains one of the most important governance mechanisms available to organizations. Audit records support accountability, compliance, investigations, and organizational learning.

However, auditing is fundamentally retrospective.

Audit mechanisms typically explain why an event occurred after operational consequences have already been realized (NIST, 2022; CNSS, 2022; NIST, 2018a; OASIS, 2013).

While such information is valuable, retrospective visibility alone may not prevent undesirable outcomes in systems capable of acting autonomously at machine speed.

6.5 Governance Failure at Operational Commitment

A recurring theme across many AI incidents is that governance mechanisms often weaken as actions approach operational commitment.

Policies may exist.

Approvals may exist.

Audit mechanisms may exist.

Risk assessments may exist.

Yet governance frequently becomes less visible precisely at the moment when an action transitions from proposed intent to operational reality.

This paper refers to that transition point as the *commit boundary*.

The commit boundary represents the moment at which a proposed action acquires the ability to affect operational systems, organizational resources, external stakeholders, physical processes, or consequential state changes.

Examples include:

- initiating a financial transaction
- modifying industrial control parameters
- changing access privileges
- issuing operational commands
- approving workflow transitions
- executing external tool actions
- committing database updates
- dispatching autonomous agents

At the commit boundary, governance questions become highly specific:

- Is this action still authorized?
- Is the supporting evidence still valid?
- Has operational context changed?
- Does the action remain admissible?

- Are required constraints satisfied?
- Is escalation required?
- Should execution proceed?

Many current governance frameworks do not explicitly define mechanisms for answering these questions at execution time.

As a result, organizations may possess extensive governance processes while simultaneously lacking systematic methods for mediating autonomous actions at the point of operational commitment.

6.6 The Emerging Need for Runtime Governance

The increasing autonomy of AI systems suggests the emergence of a governance problem that is distinct from traditional model governance.

Model governance seeks to determine whether a system should be trusted to operate.

Runtime governance seeks to determine whether a specific action should be permitted to execute.

These questions are related but not identical.

A trustworthy model may propose an inadmissible action.

A properly authorized system may encounter conditions that invalidate a previously acceptable decision.

A compliant deployment may nevertheless require execution-time intervention.

Consequently, future governance architectures may require mechanisms capable of evaluating operational legitimacy continuously rather than relying exclusively on governance decisions established before execution occurs.

The remainder of this paper explores runtime governance as a potential architectural response to this emerging challenge. The objective is not to replace existing governance frameworks but rather to examine how execution-time governance controls may complement existing Secure-by-Design, cybersecurity, safety, and assurance practices as AI systems become increasingly autonomous.

6.7 Applicability of Runtime Governance

Runtime governance is not proposed as a universal requirement for all artificial intelligence systems.

The need for execution-time governance generally increases as systems become more autonomous, gain access to external tools and resources, exercise delegated authority, coordinate across multiple systems, or acquire the ability to produce consequential operational effects.

Many AI applications operate primarily as advisory or decision-support systems. In such environments, lifecycle governance, security controls, assurance activities, and human oversight may provide sufficient governance mechanisms because the final operational decision remains under direct human control.

As autonomy increases, however, governance requirements may shift. Systems capable of initiating actions, invoking tools, coordinating workflows, interacting with external systems, or producing

operational effects without immediate human review introduce governance challenges that may not be fully addressed through pre-deployment controls or post-execution auditing alone.

Table 1 illustrates a conceptual spectrum of runtime governance applicability.

Table 1: Conceptual Runtime Governance Applicability Spectrum

System Type	Typical Runtime Governance Need
Informational and Conversational Systems	Low
Decision-Support and Recommendation Systems	Moderate
Tool-Using Agents	High
Multi-Agent Coordination Systems	Very High
Autonomous Operational Systems with External Effects	Essential

The classifications shown above are illustrative rather than normative. Actual governance requirements depend upon operational context, authority boundaries, risk tolerance, regulatory obligations, safety considerations, and the potential consequences of system actions.

The central argument of this paper is not that every AI system requires a runtime governance architecture. Rather, the argument is that as systems become increasingly capable of autonomous action, execution-time governance may become an important architectural mechanism for maintaining operational legitimacy, accountability, and control at the point where actions transition into execution.

7 Existing Governance Frameworks Remain Essential

7.1 Introduction

The argument presented in this paper should not be interpreted as a critique of existing artificial intelligence governance frameworks. Current governance, risk management, security, safety, and compliance approaches remain indispensable foundations for trustworthy AI deployment. Organizations that fail to implement established governance practices are unlikely to benefit from additional runtime governance mechanisms regardless of their sophistication.

Consequently, the objective of this paper is not to advocate replacing existing frameworks. Rather, the objective is to examine whether increasingly autonomous and agentic systems introduce execution-time governance requirements that complement existing governance activities.

The distinction is important. Runtime governance should be viewed as an extension of Secure-by-Design and operational assurance principles rather than as an alternative governance philosophy.

7.2 Contributions of Existing Governance Frameworks

Modern AI governance frameworks address many of the most significant risks associated with artificial intelligence systems. Collectively, these frameworks provide guidance for organizational accountability, risk management, security engineering, transparency, monitoring, testing, validation, and operational oversight (NIST, 2023b, 2024b; ISO, 2023; European Union, 2024; Executive Office of the President, 2023; DHS, 2024; NIST, 2024a, 2020a).

Although the frameworks differ in scope and emphasis, they share several common objectives:

- reducing organizational risk
- improving transparency
- strengthening accountability
- promoting safety and security
- increasing trustworthiness
- improving governance maturity
- enabling regulatory compliance
- supporting operational resilience

These contributions remain essential regardless of future developments in runtime governance.

7.3 The NIST AI Risk Management Framework

The NIST Artificial Intelligence Risk Management Framework (AI RMF) provides one of the most influential contemporary approaches to AI governance (NIST, 2023b). The framework organizes AI risk management activities around four functions:

1. Govern
2. Map
3. Measure
4. Manage

The AI RMF emphasizes organizational accountability, risk identification, measurement processes, governance structures, monitoring, and continuous improvement. Importantly, the framework acknowledges that AI risks may evolve throughout the lifecycle of a system and that governance must remain adaptive (NIST, 2023b).

The AI RMF establishes many foundational capabilities necessary for trustworthy AI operation, including:

- governance structures
- risk assessment
- documentation
- monitoring
- transparency
- accountability
- measurement processes

These capabilities remain prerequisites for any future runtime governance architecture.

However, the AI RMF intentionally avoids prescribing specific implementation architectures for execution-time governance. The framework identifies governance objectives while allowing organizations flexibility regarding implementation mechanisms.

This distinction creates an opportunity for runtime governance architectures to serve as implementation approaches supporting AI RMF objectives rather than competing with them.

7.4 The NIST Cybersecurity Framework

The NIST Cybersecurity Framework (CSF) has become a foundational reference for cybersecurity programs across both public and private sectors (NIST, 2024a).

The CSF functions:

1. Govern
2. Identify
3. Protect

4. Detect
5. Respond
6. Recover

provide a structured approach for managing cybersecurity risks across organizational environments.

Many runtime governance concepts align naturally with established cybersecurity principles found throughout the CSF, including:

- least privilege
- accountability
- traceability
- resilience
- continuous monitoring
- response coordination

Rather than introducing entirely new governance objectives, runtime governance may be viewed as extending established cybersecurity principles into increasingly autonomous operational environments.

7.5 NIST SP 800-53 Security and Privacy Controls

NIST Special Publication 800-53 provides one of the most comprehensive collections of security and privacy controls available for information systems and organizations (NIST, 2020a).

Control families such as:

- Access Control
- Audit and Accountability
- Identification and Authentication
- System and Communications Protection
- Risk Assessment
- System and Information Integrity

establish many of the security principles that underpin contemporary governance systems.

Runtime governance concepts frequently map to objectives already represented within SP 800-53. For example:

- execution-time authorization extends access-control principles
- governance attribution extends accountability principles
- governance replayability extends auditability objectives
- canonical-state validation extends integrity objectives

The existence of these mappings reinforces the view that runtime governance is evolutionary rather than revolutionary.

7.6 CISA Secure-by-Design Principles

CISA’s Secure-by-Design initiative emphasizes the responsibility of technology providers to incorporate security directly into products rather than relying primarily on downstream users to mitigate risk (CISA, 2023).

Core Secure-by-Design themes include:

- secure defaults
- reduced attack surface
- least privilege
- accountability
- resilience
- operational transparency

These principles are particularly relevant to autonomous and agentic systems because many operational decisions may occur without immediate human review.

As autonomy increases, security characteristics can no longer be evaluated solely through static configuration or development practices. Systems increasingly require mechanisms that continuously enforce security and governance intent throughout operational execution.

Runtime governance may therefore be interpreted as a natural extension of Secure-by-Design principles into execution-time decision processes.

7.7 MITRE ATLAS and Adversarial Threat Modeling

MITRE ATLAS provides a structured knowledge base of adversarial tactics, techniques, and procedures relevant to AI-enabled systems (MITRE, 2024).

ATLAS contributes significantly to understanding:

- attack surfaces
- adversarial behaviors

- exploitation pathways
- defensive strategies

Many attack scenarios documented within ATLAS involve the manipulation of system behavior after deployment. Examples include poisoning, evasion, prompt manipulation, model misuse, and unauthorized influence over operational decisions. Similar threat patterns have also been identified in broader analyses of AI system security and operational risk (ENISA, 2024).

Such threats highlight an important observation:

Governance decisions established before deployment may not remain sufficient throughout operational execution.

Runtime governance does not eliminate adversarial threats. However, execution-time evaluation mechanisms may provide additional opportunities to identify, constrain, or refuse actions influenced by compromised context or manipulated operational conditions. Execution-time controls may be particularly valuable when adversaries seek to manipulate operational context, exploit delegated authority, influence agent decision-making, or induce actions that remain technically executable but are no longer operationally legitimate under current conditions (MITRE, 2024; ENISA, 2024).

7.8 OWASP Agentic AI Guidance

Emerging OWASP guidance for agentic AI systems highlights security concerns associated with autonomous workflows, tool invocation, memory manipulation, prompt injection, authority escalation, and agent coordination (OWASP, 2025).

These concerns differ from many traditional software security issues because they frequently arise during operational decision-making rather than solely from implementation defects.

For example, an agent may possess legitimate access to a tool yet employ that tool in an inappropriate context due to manipulated instructions, compromised state information, or flawed delegation logic.

Such scenarios illustrate why governance questions increasingly arise at execution time rather than exclusively during development or deployment.

Runtime governance architectures may therefore provide complementary mechanisms that support the objectives identified by OWASP guidance while preserving operational flexibility.

7.9 Governance, Security, and Assurance Are Distinct Concerns

One reason runtime governance remains comparatively underrepresented within existing frameworks is that governance, security, and assurance are often treated as overlapping concepts.

Although related, these concerns are not identical.

Security focuses primarily on protecting systems against unauthorized access, misuse, disruption, modification, or destruction (NIST, 2024a, 2020a).

Governance focuses on establishing authority, accountability, policy, oversight, and acceptable behavior (NIST, 2023b; ISO, 2023; European Union, 2024; Executive Office of the President, 2023).

Assurance focuses on establishing confidence that systems operate according to intended objectives and constraints (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Goodloe and Pike, 2010).

Runtime governance occupies a position at the intersection of these domains.

A runtime governance mechanism may simultaneously:

- enforce policy
- validate authorization
- preserve accountability
- support assurance objectives
- improve operational resilience

Understanding this intersection is essential because the governance challenges introduced by autonomous systems frequently involve all three domains simultaneously.

7.10 Runtime Governance as a Complementary Layer

The central argument of this paper is not that existing governance frameworks are inadequate.

Rather, the argument is that increasing autonomy creates a category of execution-time governance challenges that existing frameworks intentionally leave open for implementation-specific solutions.

Most frameworks define:

- governance objectives
- risk management processes
- security principles
- assurance goals
- accountability expectations

They generally do not prescribe how individual operational actions should be evaluated, authorized, constrained, or refused at the moment of execution (NIST, 2023b; ISO, 2023; NIST, 2024a, 2020a).

Runtime governance may therefore be viewed as a complementary architectural layer that helps operationalize governance objectives already established by existing frameworks.

Under this perspective:

- NIST AI RMF defines governance objectives.
- NIST CSF defines cybersecurity objectives.

- SP 800-53 defines security controls.
- CISA Secure-by-Design defines engineering principles.
- MITRE ATLAS defines adversarial knowledge.
- OWASP Agentic AI identifies emerging risks.
- Runtime governance architectures provide execution-time control mechanisms that support these objectives.

This relationship is illustrated conceptually in Figure 1.

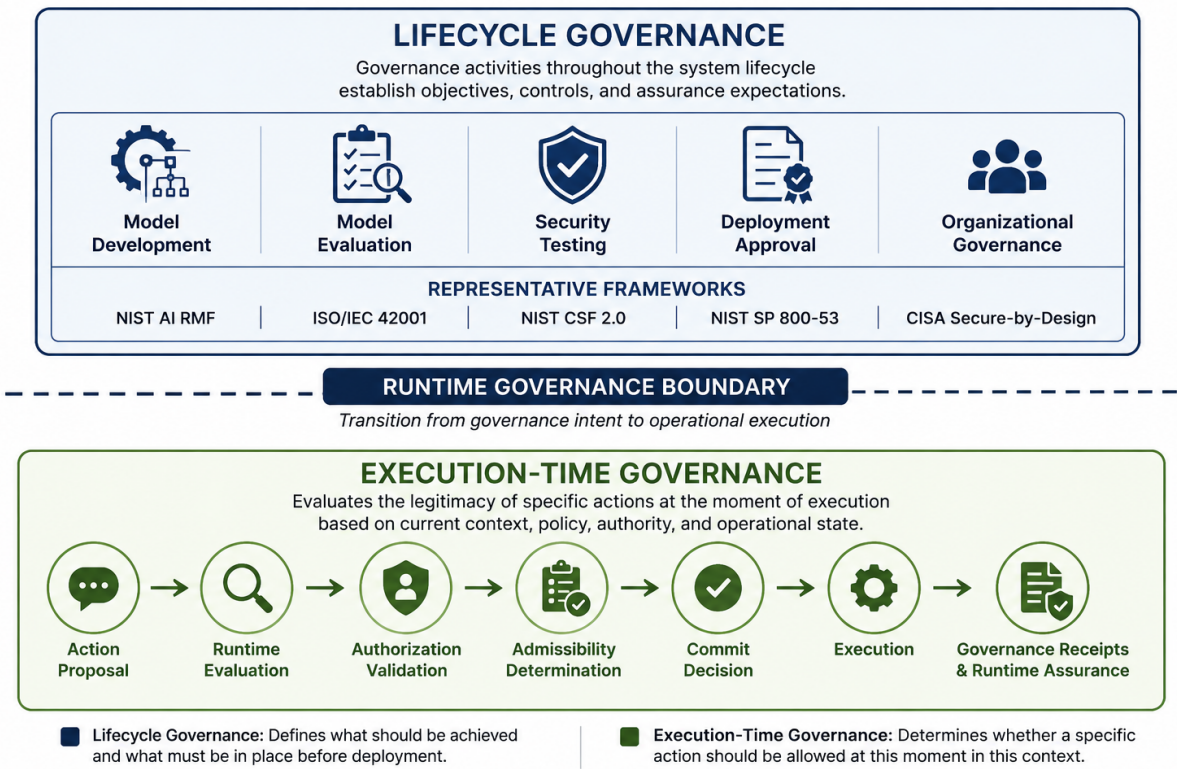


Figure 1: Illustrative relationship between lifecycle governance activities and execution-time runtime governance activities.

The remainder of this paper examines the security challenges that emerge beyond this boundary and explores runtime governance as a potential architectural response to those challenges.

8 Emerging Security Risks in Agentic AI

8.1 Introduction

The emergence of agentic AI systems represents a significant shift in the operational role of artificial intelligence. Traditional AI systems primarily functioned as analytical tools that generated recommendations, predictions, classifications, or insights for human decision-makers. In contrast, agentic systems increasingly possess the capability to initiate actions, invoke tools, coordinate workflows, interact with external systems, and pursue objectives with varying degrees of autonomy (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OpenAI, 2023; Anthropic, 2022).

These capabilities create substantial opportunities for productivity, automation, and operational efficiency. At the same time, they introduce governance and security challenges that differ materially from those associated with conventional machine learning systems.

Many established security controls assume that humans remain directly involved in consequential decisions. Agentic systems increasingly challenge this assumption by creating execution pathways in which multiple operational decisions may occur before human intervention becomes practical or even possible (Russell, 2019; Amodei et al., 2016; Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OpenAI, 2023).

As a result, governance concerns increasingly arise not merely from model behavior but from the operational consequences of autonomous action.

This section examines several emerging security risks associated with agentic AI systems and explores the governance challenges they create.

8.2 Autonomous Tool Use

One of the defining characteristics of agentic systems is the ability to invoke external tools and services in pursuit of objectives (Park et al., 2023; Wang et al., 2023; Xi et al., 2023).

Examples include:

- database queries
- file system operations
- software deployment tools
- cloud administration interfaces
- financial transaction systems
- industrial control systems
- enterprise workflow platforms
- communication systems

Tool invocation dramatically expands the operational reach of an AI system. A model that previously generated text may now possess the ability to modify infrastructure, transfer funds, alter records, trigger workflows, or interact with physical systems.

The security implications are significant because the consequences of an error no longer remain confined to generated output. Instead, incorrect decisions may directly affect operational systems.

The governance challenge is therefore not merely determining whether a tool may be used. The governance challenge is determining whether a specific tool invocation remains appropriate under current operational conditions.

For example:

An infrastructure automation agent may possess legitimate authority to restart services. Whether restarting a particular service is admissible at a specific moment may depend upon operational state, maintenance schedules, dependency relationships, safety constraints, or human approval requirements.

Such determinations frequently require execution-time evaluation rather than static authorization alone.

8.3 Multi-Agent Coordination

Agentic environments increasingly involve multiple agents operating simultaneously across shared workflows (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OpenAI, 2023).

Examples include:

- planning agents
- research agents
- orchestration agents
- execution agents
- validation agents
- monitoring agents

In many architectures, no individual agent possesses complete visibility into the entire operational context.

While specialization improves scalability and modularity, it also introduces governance challenges.

Questions arise such as:

- Which agent is accountable for a decision?
- Which agent possesses authority?

- Which evidence supports the action?
- Which constraints apply?
- How should conflicting decisions be resolved?

Traditional access-control systems often assume direct relationships between users and actions. Multi-agent systems introduce chains of delegated decisions that may span numerous components. Without mechanisms for preserving governance context across agent interactions, accountability and traceability can degrade rapidly.

8.4 Delegated Authority

Delegation is a fundamental operational requirement in complex systems. Humans routinely delegate authority to processes, applications, services, and automation platforms (NSCAI, 2021; Defense Innovation Board, 2019; Saltzer and Schroeder, 1975).

Agentic systems extend this pattern significantly.

Examples include:

- human-to-agent delegation
- agent-to-agent delegation
- workflow-based delegation
- policy-driven delegation
- conditional delegation

Delegation introduces a critical governance question:

How can authority be propagated without losing accountability?

An agent may receive authority to perform a task without receiving unrestricted authority to perform every possible action associated with that task.

For example:

A procurement agent may be authorized to gather pricing information but not to commit organizational funds.

A maintenance agent may be authorized to recommend corrective actions but not to modify safety-critical operational parameters.

Maintaining appropriate delegation boundaries requires governance mechanisms capable of evaluating authority relationships continuously throughout execution.

8.5 Prompt Injection and Instruction Manipulation

Prompt injection has emerged as one of the most widely discussed security challenges associated with large language model systems (MITRE, 2024; OWASP, 2025; OpenAI, 2023; Anthropic, 2022).

Prompt injection attacks attempt to alter system behavior through manipulated instructions embedded within user inputs, retrieved content, external documents, tool outputs, or other contextual sources.

Unlike many traditional software vulnerabilities, prompt injection frequently targets decision-making processes rather than implementation flaws.

Examples include:

- malicious instructions embedded in web content
- manipulated retrieved documents
- adversarial tool outputs
- indirect prompt attacks
- instruction override attempts

As agentic systems acquire greater operational authority, successful prompt injection attacks may influence consequential actions rather than merely generated text.

The governance challenge becomes determining whether a proposed action remains legitimate when the underlying decision process may have been influenced by compromised context.

This challenge cannot always be resolved through model evaluation alone because the relevant concern is not model quality but operational legitimacy.

8.6 Context Manipulation

Agentic systems increasingly rely upon contextual information to guide decision-making.

Examples include:

- conversation history
- memory systems
- retrieved documents
- external data feeds
- environmental state
- operational telemetry
- workflow records

Context frequently influences not only what an agent knows but also what it believes it is authorized to do (MITRE, 2024; OWASP, 2025; Wang et al., 2023).

Consequently, manipulation of context may alter decision outcomes even when the underlying model remains unchanged.

Examples include:

- corrupted memory entries
- altered governance records
- manipulated operational state
- forged evidence
- stale authorization information
- inconsistent system views

The governance challenge is ensuring that actions are evaluated against authoritative operational conditions rather than potentially compromised contextual information.

This challenge motivates concepts such as canonical-state validation and evidence verification.

8.7 Authority Escalation

Authority escalation has long been recognized as a cybersecurity concern (NIST, 2020a; Bishop, 2018; NIST, 2018b).

Agentic systems introduce new pathways through which authority escalation may occur.

Examples include:

- chained tool invocations
- delegated execution paths
- indirect privilege acquisition
- workflow escalation
- recursive delegation
- cross-agent authority inheritance

An individual action may appear benign when evaluated in isolation yet contribute to a sequence that ultimately produces unauthorized outcomes.

For example:

An agent may possess authority to create resources but not to grant permissions. Through interaction with other systems, however, resource creation may indirectly enable privilege expansion.

Such scenarios demonstrate why evaluating actions solely according to static permissions may be insufficient.

Execution-time governance mechanisms may be required to evaluate broader operational implications.

8.8 Cross-System Orchestration

Modern enterprise environments frequently involve interactions among numerous systems, services, APIs, databases, cloud platforms, and operational technologies.

Agentic systems increasingly serve as orchestration layers connecting these environments (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OpenAI, 2023).

Examples include:

- cloud management
- DevSecOps workflows
- financial operations
- supply-chain systems
- healthcare platforms
- industrial control systems

Cross-system orchestration introduces governance complexity because individual systems may possess different:

- authorization models
- trust assumptions
- risk tolerances
- accountability requirements
- operational constraints

An action that is acceptable within one system may be inappropriate when evaluated within the broader context of an orchestrated workflow.

Governance therefore increasingly requires visibility beyond individual systems and toward the overall operational intent of an action.

8.9 Operational Drift

Operational environments evolve continuously (NIST, 2023b, 2012, 2021).

Examples include:

- changing policies
- personnel changes
- evolving mission objectives
- infrastructure modifications
- regulatory updates
- environmental changes

As conditions change, decisions that were previously appropriate may become inappropriate.

This phenomenon may be described as operational drift.

Operational drift is particularly challenging for autonomous systems because previously granted authority may persist long after underlying assumptions have changed.

Examples include:

- expired approvals
- obsolete evidence
- stale context
- invalid delegation relationships
- changed operational priorities
- altered safety conditions

The governance challenge becomes ensuring that actions remain valid under current conditions rather than conditions that existed when authority was originally granted.

8.10 The Common Theme: Operational Legitimacy

Although the threats discussed above differ substantially, they share a common characteristic.

Each ultimately raises a question regarding operational legitimacy.

Specifically:

Should this action be permitted to execute under current conditions?

This question differs from many traditional governance and security questions.

For example:

- Is the model trustworthy?
- Is the software secure?
- Is the deployment compliant?
- Was the system tested?
- Is the organization following policy?

All of these questions remain important.

However, autonomous systems introduce an additional question that must be answered repeatedly throughout operation:

Is this specific action currently legitimate?

Answering this question often requires evaluating:

- authority
- evidence
- context
- operational state
- constraints
- safety conditions
- accountability requirements

at the moment an action is proposed.

8.11 Implications for Governance Architecture

The security challenges associated with agentic systems suggest that future governance architectures may require capabilities beyond traditional lifecycle governance controls.

Specifically, governance mechanisms may need to:

- evaluate actions before execution
- preserve authority relationships
- validate supporting evidence

- maintain governance continuity
- verify operational state
- support accountability
- prevent inadmissible actions
- preserve replayable governance records

These observations motivate the runtime governance principles introduced in the next section.

Rather than beginning with implementation approaches, the next section examines the foundational architectural concepts that may support execution-time governance in increasingly autonomous environments.

9 Runtime Governance Principles

9.1 Introduction

The preceding sections argued that increasingly autonomous and agentic systems introduce governance challenges that arise at the point of operational execution rather than exclusively during development, deployment, or post-execution review. Addressing these challenges requires more than additional policies, audits, or lifecycle documentation. It requires governance mechanisms capable of evaluating operational legitimacy continuously as actions are proposed and executed.

Before discussing implementation architectures, it is useful to establish a set of foundational principles that characterize runtime governance. These principles are not tied to any particular framework, product, vendor, or implementation approach. Instead, they represent architectural concepts that may help organizations preserve governance intent in environments where autonomous systems increasingly participate in consequential operational decisions.

The principles described in this section are organized into two categories:

1. Governance Objectives
2. Runtime Governance Mechanisms

Governance objectives describe desired outcomes.

Runtime governance mechanisms describe architectural approaches that may help achieve those outcomes.

The terminology introduced in this section should be interpreted as conceptual runtime-governance constructs proposed for architectural analysis rather than as formally standardized terms. Several concepts, including authorization integrity, evidence continuity, governance context preservation, governance lifecycle integrity, governance replayability, and deterministic governance, are synthesized from established principles found in governance, cybersecurity, systems security engineering, assurance, auditability, and access-control literature. The intent is not to redefine existing standards terminology but rather to provide a vocabulary for discussing execution-time governance requirements in increasingly autonomous operational environments.

9.2 Governance Objectives

9.2.1 Accountability

Accountability remains one of the most fundamental objectives of governance systems (NIST, 2023b; ISO, 2023; NIST, 2020a).

In traditional environments, accountability is often established through identifiable decision-makers, documented approvals, audit trails, and organizational oversight structures. Agentic systems complicate accountability because actions may be initiated through autonomous decision processes, delegated workflows, or multi-agent coordination chains.

Runtime governance should preserve the ability to answer questions such as:

- Who initiated the action?
- What authority enabled the action?
- What conditions justified the action?
- What evidence supported the decision?
- What system ultimately executed the action?

Without accountability, organizations may be unable to determine responsibility for consequential operational outcomes.

9.2.2 Traceability

Traceability refers to the ability to reconstruct relationships among decisions, evidence, authorizations, state transitions, and resulting actions (NIST, 2023b; ISO, 2023; NIST, 2020a, 2022).

Traceability extends beyond traditional logging.

A traceable governance system should enable observers to understand:

- why an action occurred
- how the action was evaluated
- what evidence was considered
- what constraints were applied
- how authorization was determined

Traceability is particularly important in autonomous environments because operational decisions may occur at machine speed and across multiple systems.

9.2.3 Operational Assurance

Assurance refers to confidence that a system behaves according to intended objectives, constraints, and governance requirements (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Goodloe and Pike, 2010).

Operational assurance extends this concept into runtime environments.

Rather than asking only whether a system was designed correctly, operational assurance asks:

Is the system currently operating within authorized conditions?

This distinction becomes increasingly important as operational context changes after deployment.

Operational assurance also applies to governance mechanisms themselves. Assurance requires confidence not only that actions are evaluated correctly, but also that governance enforcement remains

non-bypassable, that sufficient governance context is available at the point of enforcement, and that governance decisions are enforced consistently throughout execution (Rushby, 2008; Seshia et al., 2016; NIST, 2020a).

Runtime governance contributes to operational assurance by continuously evaluating whether proposed actions remain consistent with governance expectations.

9.2.4 Authorization Integrity

Authorization integrity refers to preserving the correctness, legitimacy, and applicability of authorization decisions throughout operational execution (NIST, 2020a,b; OASIS, 2013).

In many environments, permissions are granted long before actions occur. However, operational conditions may evolve substantially between authorization issuance and execution.

Runtime governance seeks to preserve authorization integrity by ensuring that actions remain appropriately authorized at the moment they become operationally consequential.

9.2.5 Governance Integrity

Governance systems must preserve not only the integrity of operational decisions but also the integrity of the governance mechanisms responsible for making those decisions (NIST, 2020a; ISO, 2023; NIST, 2023b; CISA, 2023).

Governance integrity refers to maintaining the correctness, protection, and trustworthiness of governance artifacts, policies, constraints, decision processes, and enforcement mechanisms throughout operation.

This objective extends beyond preventing unauthorized operational actions. It also seeks to prevent unauthorized modification, weakening, circumvention, or degradation of governance controls themselves.

Examples include:

- protection of governance policies
- preservation of constitutional constraints
- prevention of unauthorized governance modification
- preservation of enforcement integrity
- protection against governance bypass

As autonomous systems become increasingly capable of adapting operational behavior, preserving governance integrity becomes necessary to ensure that governance remains authoritative, bounded, and enforceable throughout system operation.

9.2.6 Safety Preservation

Many autonomous systems operate within environments where inappropriate actions may create safety, reliability, financial, legal, or mission-related consequences (NIST, 2024b; CISA, 2023; NSCAI, 2021; Defense Innovation Board, 2019; Leveson, 2011).

Governance therefore serves not merely administrative objectives but also operational safety objectives.

Runtime governance mechanisms should help prevent actions that violate safety constraints, exceed operational tolerances, or create unacceptable risk under current conditions.

9.2.7 Replayability

Replayability refers to the ability to reconstruct governance decisions and reproduce governance outcomes using retained records, evidence, and decision logic (NIST, 2020a, 2022; Rushby, 2008).

Replayability supports:

- audits
- investigations
- assurance activities
- incident analysis
- regulatory review
- system validation

Without replayability, governance outcomes may become difficult to explain, verify, or defend after execution occurs.

9.3 Runtime Governance Mechanisms

The objectives described above require implementation mechanisms capable of preserving governance intent during execution.

The following concepts represent architectural mechanisms that may contribute to those objectives.

9.3.1 Admissibility Evaluation

Admissibility evaluation refers to determining whether a proposed action satisfies the conditions necessary for execution under current operational circumstances.

Importantly, admissibility differs from authorization.

Authorization addresses questions such as (NIST, 2020a,b; OASIS, 2013):

Does this subject possess authority to perform this action?

Admissibility addresses broader questions:

Should this action be permitted to execute under current conditions?

An action may be authorized yet inadmissible.

Examples include:

- valid authority but invalid operational state
- valid permissions but expired evidence
- authorized action but violated safety conditions
- delegated authority but missing approvals

Admissibility evaluation therefore acts as a governance decision process that considers operational legitimacy rather than authority alone.

9.3.2 Execution-Bound Authorization

Traditional authorization approaches frequently evaluate permissions separately from operational execution (NIST, 2020b,a; OASIS, 2013).

Execution-bound authorization links authorization directly to a specific proposed action and its associated execution context.

Under this approach, authorization decisions remain tied to:

- the action being proposed
- the operational context
- supporting evidence
- applicable constraints
- delegated authority relationships

Execution-bound authorization helps ensure that governance decisions remain relevant at the moment operational consequences occur.

9.3.3 Authority Containment

Authority containment refers to maintaining explicit boundaries around delegated execution authority as actions propagate across systems, agents, workflows, and organizational domains (Saltzer and Schroeder, 1975; NIST, 2020a,b).

Autonomous environments frequently rely upon delegation mechanisms that allow authority to be exercised indirectly through subordinate agents or operational processes. Without containment mechanisms, delegated authority may expand beyond its intended scope, creating opportunities for privilege amplification, unintended authority propagation, or unauthorized operational influence.

Authority containment seeks to ensure that delegated authority remains bounded by its originating governance constraints and does not implicitly create additional authority beyond that which was explicitly granted.

Examples include:

- bounded delegation
- constrained authority propagation
- recursive delegation controls
- scope-limited execution authority
- domain-specific authorization boundaries

Authority containment contributes to preserving least-privilege principles in increasingly autonomous operational environments (Saltzer and Schroeder, 1975; NIST, 2020a).

9.3.4 Structural Refusal

Governance systems require mechanisms for preventing actions that fail governance evaluation (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018).

This paper refers to such mechanisms as structural refusal.

Structural refusal differs from advisory recommendations.

A recommendation may suggest that an action should not occur.

Structural refusal prevents the action from proceeding.

Examples include:

- denial of execution
- rejection of transactions
- prevention of state changes
- interruption of workflow progression

- termination of inadmissible actions

Structural refusal provides a mechanism for enforcing governance decisions rather than merely recording them.

9.3.5 Canonical-State Governance

Many governance failures arise from decisions based upon incomplete, stale, inconsistent, or manipulated information (NIST, 2021; MITRE, 2024; OWASP, 2025).

Canonical-state governance seeks to reduce this risk by evaluating actions against authoritative operational conditions.

The canonical state may include:

- system status
- operational telemetry
- authority relationships
- policy state
- governance context
- environmental conditions
- workflow status

By grounding governance decisions in authoritative operational information, organizations may improve consistency and reduce susceptibility to context manipulation.

9.3.6 Evidence Continuity

Governance decisions frequently depend upon supporting evidence (NIST, 2022, 2018a; ISO, 2023).

Examples include:

- approvals
- certifications
- policy evaluations
- risk assessments
- operational measurements
- compliance determinations

Evidence continuity refers to preserving both the availability and integrity of such evidence throughout the lifecycle of an action.

Without evidence continuity, organizations may later be unable to determine whether governance decisions were justified when they were made.

9.3.7 Governance Context Preservation

Autonomous systems often operate across complex workflows involving multiple actors, services, agents, and operational systems (NIST, 2021, 2020b; Wang et al., 2023).

Governance context preservation seeks to maintain:

- authority relationships
- accountability information
- policy constraints
- governance decisions
- supporting evidence

throughout these interactions.

Governance context preservation is particularly important in multi-agent environments where coordinated behavior may emerge through interactions among autonomous components. Preserving governance context helps ensure that coordination remains bounded by applicable governance constraints rather than evolving into unmanaged operational behavior (NIST, 2021; Wang et al., 2023).

Without context preservation, governance intent may degrade as actions move across operational boundaries.

9.3.8 Governance Isolation

Governance isolation refers to preserving separation among governance domains, authorities, policies, and operational contexts (NIST, 2020a, 2021, 2020b).

Many autonomous systems operate across organizational, tenant, mission, or administrative boundaries. Governance decisions that are valid within one domain may not be applicable within another.

Governance isolation seeks to ensure that governance authority, evidence, policies, and decisions remain appropriately scoped to the domains within which they are authorized.

Examples include:

- tenant isolation
- administrative boundary enforcement
- cross-domain governance separation

- authority scoping
- policy-domain isolation

Governance isolation helps prevent unintended authority transfer and reduces the risk of governance decisions affecting systems beyond their authorized scope (NIST, 2021, 2020b).

9.3.9 Governance Replayability

Governance replayability extends the objective of replayability into a concrete governance capability (Rushby, 2008; NIST, 2022; Seshia et al., 2016).

A replayable governance process enables reconstruction of:

- proposed actions
- evaluated evidence
- authorization decisions
- applicable constraints
- resulting outcomes

Such capabilities support assurance, auditing, validation, and forensic analysis.

Replayability also contributes to organizational trust because governance outcomes become independently verifiable rather than dependent upon undocumented decision processes.

9.3.10 Governance Lifecycle Integrity

Governance decisions should not become disconnected from the actions they authorize.

Governance lifecycle integrity refers to preserving traceable relationships among (NIST, 2018a, 2020a; ISO, 2023):

- proposals
- evaluations
- authorizations
- executions
- resulting state transitions
- governance records

throughout the operational lifecycle of an action.

Lifecycle integrity enables organizations to determine not only what occurred but also whether governance controls remained intact throughout execution.

9.3.11 Enforcement Integrity

Governance decisions provide value only when they are enforced reliably at the point where actions become operationally consequential (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2020a).

Enforcement integrity refers to preserving trustworthy relationships among governance evaluation, authorization decisions, and operational execution.

Effective enforcement requires that governance mechanisms possess sufficient information to evaluate admissibility correctly and that operational systems remain unable to bypass governance decisions (Saltzer and Schroeder, 1975; NIST, 2020a).

Examples include:

- governance-aware enforcement points
- non-bypassable enforcement paths
- complete governance context availability
- enforcement decision consistency
- execution-time policy enforcement

Enforcement integrity helps ensure that governance outcomes remain operationally meaningful rather than merely advisory (Saltzer and Schroeder, 1975; Anderson, 2020).

9.3.12 Deterministic Governance

Deterministic governance refers to governance processes that produce consistent outcomes when presented with equivalent inputs, policies, evidence, and operational conditions (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

Determinism contributes to:

- reproducibility
- auditability
- assurance
- testing
- formal verification

Deterministic governance does not require that all systems behave identically under all circumstances. Rather, it requires that governance decisions remain explainable and reproducible when relevant conditions are equivalent.

This characteristic becomes increasingly important in high-consequence environments where organizations must demonstrate not only that governance decisions occurred but also that those decisions were reached through reliable and repeatable processes.

9.3.13 Deterministic Governance and Evidence Uncertainty

Deterministic governance does not imply that all operational inputs, evidence sources, or environmental conditions are themselves deterministic.

Runtime governance may operate in environments characterized by uncertainty, incomplete information, delayed telemetry, probabilistic model outputs, conflicting evidence sources, changing operational conditions, or imperfect sensor observations. Such uncertainty is a normal characteristic of many real-world systems.

Within this paper, deterministic governance refers to the repeatable evaluation of governance decisions given the evidence, policies, authorities, and canonical state available at the time of evaluation.

Under this model, identical governance inputs produce identical admissibility and authorization outcomes. The determinism applies to the governance process itself rather than to the underlying environment from which governance evidence is derived.

Consequently, governance decisions may evolve as operational conditions change and new evidence becomes available. Such changes do not represent nondeterministic governance behavior. Rather, they reflect deterministic evaluation performed against a different evidentiary or operational context.

This distinction is important because runtime governance seeks to provide consistency, replayability, accountability, and traceability in governance decision-making even when operating within environments that contain uncertainty or probabilistic system components.

9.4 Relationships Among Runtime Governance Mechanisms

The mechanisms described above are complementary rather than independent.

For example:

- admissibility evaluation may depend upon canonical-state validation
- execution-bound authorization may depend upon evidence continuity
- structural refusal may result from failed admissibility evaluation
- replayability may depend upon lifecycle integrity
- accountability may depend upon governance context preservation
- authority containment may depend upon governance isolation
- enforcement integrity may depend upon governance context preservation

Consequently, runtime governance should be viewed as an interconnected system of controls rather than a collection of isolated features.

The effectiveness of any individual mechanism often depends upon the integrity of the broader governance architecture within which it operates.

9.5 From Principles to Runtime Assurance

The principles introduced in this section establish a conceptual foundation for runtime governance. However, principles alone do not explain how organizations determine whether autonomous systems remain within acceptable operational conditions.

This challenge leads naturally to the concept of runtime assurance.

Runtime assurance focuses on the continuous evaluation of operational legitimacy throughout execution and provides a bridge between governance principles and practical governance architectures.

The next section examines runtime assurance as an emerging discipline for autonomous and agentic systems.

10 Runtime Assurance and Operational Legitimacy

10.1 Introduction

The runtime governance principles described in the previous section establish a conceptual foundation for evaluating and constraining autonomous actions. However, governance principles alone do not answer a critical operational question:

How can organizations determine whether an autonomous system remains within authorized operational conditions at the moment an action is proposed?

This question lies at the heart of runtime assurance.

Traditional assurance approaches focus primarily on demonstrating that a system satisfies specified requirements, security objectives, safety goals, or compliance obligations prior to deployment (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Goodloe and Pike, 2010; NIST, 2022; ISO, 2023). Such approaches remain essential. However, increasingly autonomous systems introduce circumstances in which operational legitimacy may change after deployment due to evolving context, authority relationships, environmental conditions, mission priorities, or system state.

Consequently, assurance must increasingly become a continuous operational activity rather than a one-time validation event.

This section examines runtime assurance as an architectural approach for maintaining confidence in autonomous actions throughout operational execution.

10.2 The Evolution of Assurance

Historically, assurance activities have focused on demonstrating that systems satisfy predetermined requirements before they enter production.

Examples include:

- security accreditation
- certification processes
- compliance validation
- safety reviews
- formal verification
- penetration testing
- independent assessment

These activities establish confidence that systems are generally suitable for operation (NIST, 2020a, 2022; ISO, 2023; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

The underlying assumption is often that if a system is sufficiently trustworthy at deployment, acceptable behavior will continue throughout operation.

This assumption becomes increasingly difficult to sustain in highly dynamic autonomous environments.

Agentic systems may encounter:

- changing operational conditions
- evolving authority relationships
- new information
- modified objectives
- altered constraints
- adversarial influences
- degraded infrastructure

As a result, the legitimacy of individual actions may vary substantially over time even when the underlying system remains unchanged.

Runtime assurance addresses this challenge by evaluating operational legitimacy continuously rather than relying exclusively on prior certification or validation.

10.3 Operational Legitimacy Versus System Trustworthiness

An important distinction exists between system trustworthiness and operational legitimacy.

System trustworthiness refers to confidence that a system was designed, implemented, tested, and deployed according to appropriate governance, security, and engineering practices (NIST, 2023b; ISO, 2023; CISA, 2023; NIST, 2020a, 2018b, 2021).

Operational legitimacy refers to whether a specific action should be permitted under current conditions.

These concepts are related but not identical.

A trustworthy system may propose an illegitimate action.

Conversely, a less sophisticated system may occasionally propose actions that remain entirely legitimate.

Consider the following examples:

- A highly reliable maintenance agent proposes a shutdown during an emergency response event.
- A compliant financial agent initiates a transfer based upon stale account information.

- A validated orchestration agent attempts to execute a workflow after required approvals have expired.

In each case, the system itself may remain trustworthy.

The issue is whether the specific action remains legitimate under current operational circumstances.

Runtime assurance focuses on this distinction.

10.4 The Need for Continuous Governance Evaluation

Many governance mechanisms operate periodically.

Examples include:

- quarterly reviews
- annual audits
- certification cycles
- periodic risk assessments
- scheduled compliance evaluations

These activities provide important oversight but may be poorly aligned with environments where consequential decisions occur continuously (Wang et al., 2023; Xi et al., 2023; Park et al., 2023; NIST, 2021).

Autonomous systems may initiate thousands of actions per hour.

Multi-agent environments may generate complex chains of delegated decisions within seconds.

Consequently, governance effectiveness increasingly depends upon the ability to evaluate legitimacy at operational timescales rather than administrative timescales.

Runtime assurance addresses this challenge through continuous governance evaluation.

Instead of asking:

Was this system approved?

Runtime assurance asks:

Should this action be approved right now?

This distinction represents a significant shift in governance philosophy.

10.5 Commit-Time Governance

One useful way to understand runtime assurance is through the concept of commit-time governance.

A proposed action generally passes through multiple stages before producing operational consequences.

A simplified sequence might include:

1. Action proposal
2. Evidence collection
3. Constraint evaluation
4. Authorization evaluation
5. Operational commitment
6. Execution
7. State transition

The point at which an action becomes operationally consequential was previously defined as the commit boundary.

Commit-time governance focuses governance evaluation at or immediately before this boundary.

This approach recognizes that the most relevant information regarding operational legitimacy may be available only at the moment execution becomes possible (Rushby, 2008; Goodloe and Pike, 2010; NIST, 2021).

Examples include:

- current system health
- active incident conditions
- authority status
- workflow state
- environmental constraints
- safety conditions
- competing operational priorities

Evaluating legitimacy at commit time helps ensure that governance decisions reflect current operational reality rather than historical assumptions.

10.6 Canonical State and Operational Reality

Runtime assurance depends upon accurate understanding of operational conditions.

Unfortunately, distributed systems often contain multiple representations of state.

Examples include:

- local caches
- replicated databases
- workflow state machines
- agent memory stores
- telemetry systems
- authorization repositories
- governance records

These representations may diverge (NIST, 2021; MITRE, 2024; OWASP, 2025).

Consequently, governance decisions based upon incomplete or inconsistent state information may produce incorrect outcomes.

Canonical-state governance addresses this challenge by establishing an authoritative representation of operational reality against which governance decisions are evaluated.

The canonical state may include:

- operational status
- governance context
- authorization relationships
- environmental conditions
- workflow progress
- safety constraints
- active incidents

Runtime assurance depends heavily upon the ability to evaluate actions against such authoritative information.

Without reliable state information, governance decisions may become increasingly difficult to justify or reproduce.

10.7 Evidence Validity and Evidence Continuity

Many governance decisions rely upon supporting evidence (NIST, 2022, 2018a; ISO, 2023).

Examples include:

- approvals
- certifications
- policy evaluations
- risk assessments
- operational measurements
- compliance determinations

A common assumption is that once evidence has been collected, its validity remains stable.

In practice, evidence often has a limited useful lifetime (NIST, 2012, 2018a).

Examples include:

- expired approvals
- superseded policies
- outdated assessments
- stale telemetry
- invalidated certifications
- changed operational conditions

Runtime assurance therefore requires not only evidence availability but also evidence continuity.

Evidence continuity ensures that supporting evidence remains:

- available
- attributable
- verifiable
- relevant
- temporally appropriate

at the moment governance decisions occur.

10.8 Governance Continuity Across Autonomous Workflows

Agentic systems frequently operate across complex workflows involving multiple actors and systems (Park et al., 2023; Wang et al., 2023; Xi et al., 2023).

Examples include:

- human operators
- orchestration agents
- execution agents
- approval systems
- enterprise applications
- infrastructure platforms

Governance intent may degrade as actions move through such environments.

Authority relationships may become unclear.

Evidence may become disconnected from resulting actions.

Accountability may become fragmented.

Governance continuity seeks to preserve governance semantics across these transitions (NIST, 2021, 2020b).

Examples include maintaining:

- attribution
- authority lineage
- evidence relationships
- authorization context
- operational constraints

throughout the lifecycle of an action.

Governance continuity is therefore a critical component of runtime assurance.

Without it, organizations may lose visibility into how governance decisions propagate through autonomous workflows.

10.9 Runtime Assurance and Structural Refusal

Assurance requires more than observation.

Assurance also requires the ability to act upon governance conclusions.

If a governance evaluation determines that an action is inadmissible, the system must possess mechanisms capable of preventing execution (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018).

This requirement motivates structural refusal.

Structural refusal transforms runtime assurance from a passive monitoring activity into an operational control mechanism.

Examples include:

- blocking execution
- terminating workflows
- denying transactions
- preventing state transitions
- requiring escalation
- invoking human review

Without refusal mechanisms, runtime assurance may identify governance concerns without possessing the ability to prevent undesirable outcomes.

10.10 Deterministic Runtime Assurance

High-consequence environments frequently require more than governance visibility.

They require governance predictability.

Organizations often need to demonstrate:

- why a decision occurred
- how a decision occurred
- whether the decision can be reproduced
- whether equivalent conditions would produce equivalent outcomes

These requirements motivate deterministic runtime assurance (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

Deterministic assurance does not imply that systems always reach identical decisions.

Rather, it implies that governance outcomes remain reproducible when presented with equivalent:

- evidence
- state

- policies
- constraints
- operational conditions

Determinism supports auditing, assurance, testing, investigation, and formal analysis.

It also strengthens organizational confidence that governance outcomes are the result of defined processes rather than opaque or unpredictable behavior.

10.11 Runtime Assurance as an Architectural Discipline

The concepts described throughout this section suggest that runtime assurance should be viewed as a distinct architectural discipline rather than merely an extension of monitoring or auditing.

Runtime assurance integrates:

- governance
- security
- authorization
- accountability
- evidence management
- operational state evaluation
- execution control

into a unified operational capability.

Its purpose is not to replace existing governance frameworks.

Its purpose is to preserve governance intent at the moment autonomous actions become operationally consequential.

As AI systems become increasingly capable of acting independently, runtime assurance may become an essential mechanism for maintaining trust, accountability, and operational legitimacy.

10.12 From Runtime Assurance to Runtime Governance Controls

The principles and assurance concepts introduced thus far establish a conceptual foundation for runtime governance.

The next question is practical:

What categories of controls are required to implement runtime governance and runtime assurance capabilities?

The next section introduces a taxonomy of runtime governance control categories that organizes these capabilities into identity, authorization, state, evidence, execution, and assurance control domains.

11 Runtime Governance Control Categories

11.1 Introduction

The preceding sections introduced the concepts of runtime governance and runtime assurance as complementary mechanisms for preserving governance intent in increasingly autonomous environments. While these concepts provide useful architectural principles, organizations ultimately require practical methods for implementing governance controls within operational systems.

Historically, cybersecurity, safety, and governance disciplines have benefited from organizing controls into categories that address distinct aspects of system operation. Examples include identity management, access control, auditability, integrity protection, and incident response (NIST, 2024a, 2020a, 2018b). Similar categorization can assist organizations in understanding and implementing runtime governance capabilities.

This section introduces a proposed taxonomy of runtime governance control categories. The taxonomy is not intended to replace existing security or governance control frameworks. Instead, it provides a conceptual structure for organizing execution-time governance capabilities that may support Secure-by-Design objectives in autonomous and agentic systems.

The categories described below represent functional governance domains rather than implementation requirements. Individual organizations may choose different architectural approaches while still addressing the same governance objectives.

11.2 Identity Controls

Identity controls establish the foundation for governance accountability by ensuring that actions can be associated with identifiable subjects (NIST, 2020a,b).

In traditional systems, identity controls focus primarily on human users and service accounts. Autonomous systems introduce additional identity considerations because actions may originate from agents, orchestrators, workflows, delegated authorities, or combinations thereof.

Runtime governance identity controls seek to answer questions such as:

- Who initiated this action?
- Which agent proposed the action?
- Under whose authority is the action operating?
- Which system is responsible for execution?
- Which delegation chain applies?

Identity controls are also important for preserving authority provenance across delegation chains. In multi-agent environments, governance systems should be able to reconstruct not only who initiated an action, but also how authority was propagated and whether that propagation remained consistent with governing constraints (NIST, 2020a, 2021; Wang et al., 2023).

Representative identity-control capabilities include:

- subject identification
- agent identification
- delegation lineage
- authority attribution
- identity verification
- execution attribution

Without reliable identity controls, governance accountability becomes increasingly difficult to establish.

11.3 Authorization Controls

Authorization controls determine whether a subject possesses authority to perform a proposed action.

Traditional authorization mechanisms often focus on permissions associated with users, groups, roles, or services (NIST, 2020a,b; OASIS, 2013). Autonomous environments introduce additional complexity because authorization may depend upon context, delegation relationships, evidence, operational conditions, and workflow state.

In autonomous and multi-agent environments, authorization controls must also ensure that delegated authority remains bounded by its originating governance constraints. Delegation should not implicitly create additional authority, expand privilege beyond approved scope, or permit recursive authority amplification across agent or workflow boundaries (Saltzer and Schroeder, 1975; NIST, 2020a,b).

Runtime governance authorization controls seek to answer questions such as:

- Does the subject possess authority?
- Is delegated authority valid?
- Does the authorization remain applicable?
- Are escalation requirements satisfied?
- Does the proposed action exceed granted authority?
- Does delegated authority remain within its approved scope?
- Could authority propagation create unintended privilege expansion?

Representative authorization-control capabilities include:

- execution-time authorization
- delegated authority validation

- escalation management
- authority scope validation
- authorization continuity
- authorization attribution
- authority containment
- recursive delegation controls

Authorization controls establish the legitimacy of authority relationships but do not by themselves determine whether execution should occur.

11.4 State Controls

State controls establish mechanisms for evaluating actions against authoritative operational conditions.

Many governance failures occur because decisions are made using incomplete, stale, inconsistent, or manipulated information (NIST, 2021; MITRE, 2024; OWASP, 2025). Autonomous systems often rely on multiple representations of operational state, creating opportunities for divergence and inconsistency.

State controls may also be responsible for maintaining separation among governance domains, tenants, organizations, or administrative authorities. Governance decisions that are valid within one domain may not necessarily be valid within another, making authoritative determination of governance scope and state ownership increasingly important in distributed and multi-agent environments (NIST, 2021, 2020b,a).

Runtime governance state controls seek to answer questions such as:

- What is the authoritative operational state?
- Has state information changed?
- Are all participants observing consistent conditions?
- Is the proposed action compatible with current state?
- Have relevant operational conditions been verified?
- Does the governance state belong to the correct authority domain?

Representative state-control capabilities include:

- canonical-state validation
- state synchronization
- operational condition verification

- context validation
- state consistency assurance
- authoritative-state determination
- governance-domain isolation

State controls provide the foundation for ensuring that governance decisions reflect operational reality rather than historical assumptions.

11.5 Evidence Controls

Governance decisions frequently depend upon supporting evidence (NIST, 2022, 2018a; ISO, 2023).

Evidence may include:

- approvals
- policy evaluations
- certifications
- risk assessments
- operational measurements
- compliance determinations
- workflow records

Evidence controls seek to preserve the integrity, availability, attribution, and relevance of such information throughout the lifecycle of an action (NIST, 2022, 2020a).

Runtime governance evidence controls seek to answer questions such as:

- What evidence supports this action?
- Is the evidence authentic?
- Is the evidence current?
- Has the evidence been altered?
- Does the evidence remain applicable?

Representative evidence-control capabilities include:

- evidence validation
- evidence continuity

- provenance verification
- temporal validity checking
- evidence attribution
- evidence retention

Evidence controls are particularly important in autonomous environments where governance decisions may occur without direct human review.

11.6 Execution Controls

Execution controls operate closest to the point of operational commitment (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018).

These controls focus on evaluating whether actions should be permitted to execute under current conditions.

Execution controls differ from identity, authorization, state, and evidence controls because they ultimately determine whether a proposed action proceeds, is modified, is delayed, or is refused.

Runtime governance execution controls seek to answer questions such as:

- Is the action admissible?
- Are all required conditions satisfied?
- Should execution proceed?
- Should escalation occur?
- Should execution be prevented?

Execution controls may also constrain coordinated behavior among multiple autonomous components. As agentic systems increasingly cooperate, governance mechanisms may need to evaluate not only individual actions but also the admissibility of collective behavior emerging from coordinated interactions (NIST, 2021; Wang et al., 2023; Seshia et al., 2018).

Representative execution-control capabilities include:

- admissibility evaluation
- execution-time decision making
- constraint validation
- operational commitment control
- structural refusal
- escalation initiation
- autonomous coordination governance

Execution controls serve as the primary governance mechanism operating at the commit boundary.

11.7 Assurance Controls

Assurance controls provide confidence that governance processes are operating correctly and can be independently validated (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; NIST, 2022).

These controls support both operational oversight and post-execution verification.

Runtime governance assurance controls seek to answer questions such as:

- Can governance decisions be reconstructed?
- Can governance outcomes be reproduced?
- Can governance correctness be evaluated?
- Can governance failures be investigated?
- Can governance effectiveness be measured?
- Can governance enforcement be bypassed?
- Is sufficient governance context available at enforcement time?
- Can enforcement decisions be independently verified?

Representative assurance-control capabilities include:

- governance replayability
- governance lifecycle integrity
- deterministic evaluation
- governance auditability
- governance verification
- governance measurement
- enforcement integrity validation
- governance enforcement verification
- non-bypassability assessment

Assurance controls are particularly important in high-consequence environments where organizations must demonstrate not only that governance decisions occurred but also that those decisions were reached through reliable and verifiable processes.

11.8 Control Interdependencies

Although the categories described above are presented separately, they are highly interdependent. For example:

- Authorization controls depend upon identity controls.
- Execution controls depend upon authorization controls.
- Admissibility evaluation may depend upon state controls.
- Authorization decisions may depend upon evidence controls.
- Assurance controls depend upon records generated throughout all governance activities.
- Authority containment depends upon both identity and authorization controls.
- Governance isolation depends upon state controls and authoritative context management.
- Enforcement integrity depends upon execution controls and assurance controls.

These relationships illustrate an important architectural principle:

Runtime governance effectiveness depends not on individual controls in isolation but on the integrity of the governance system as a whole.

Failures in any control category may undermine the effectiveness of the broader governance architecture.

11.9 Relationship to Existing Security Controls

The control categories introduced here are not intended to replace existing cybersecurity controls. Rather, they complement established security practices (Saltzer and Schroeder, 1975; Anderson, 2020; NIST, 2020a, 2018b).

For example:

- Identity controls extend authentication and attribution principles.
- Authorization controls extend access-control principles.
- State controls extend integrity and configuration-management principles.
- Evidence controls extend auditability and provenance principles.
- Execution controls extend complete mediation principles.
- Assurance controls extend verification and accountability principles.

- Governance isolation, authority containment, and enforcement controls extend least-privilege, trust-boundary, and complete-mediation principles.

This relationship reinforces the view that runtime governance represents an evolution of established security concepts rather than an entirely new discipline.

11.10 A Layered Runtime Governance Model

The runtime governance control categories introduced throughout this section can be organized into a layered architectural model. Figure 2 illustrates how authorization, execution governance, and runtime assurance capabilities interact to provide governance mediation at the commit boundary.

Rather than representing independent control functions, these layers establish a dependency hierarchy in which higher-order governance capabilities rely upon services provided by lower governance layers. This structure helps explain how runtime governance extends established security principles such as complete mediation, least privilege, accountability, and operational assurance into autonomous execution environments.

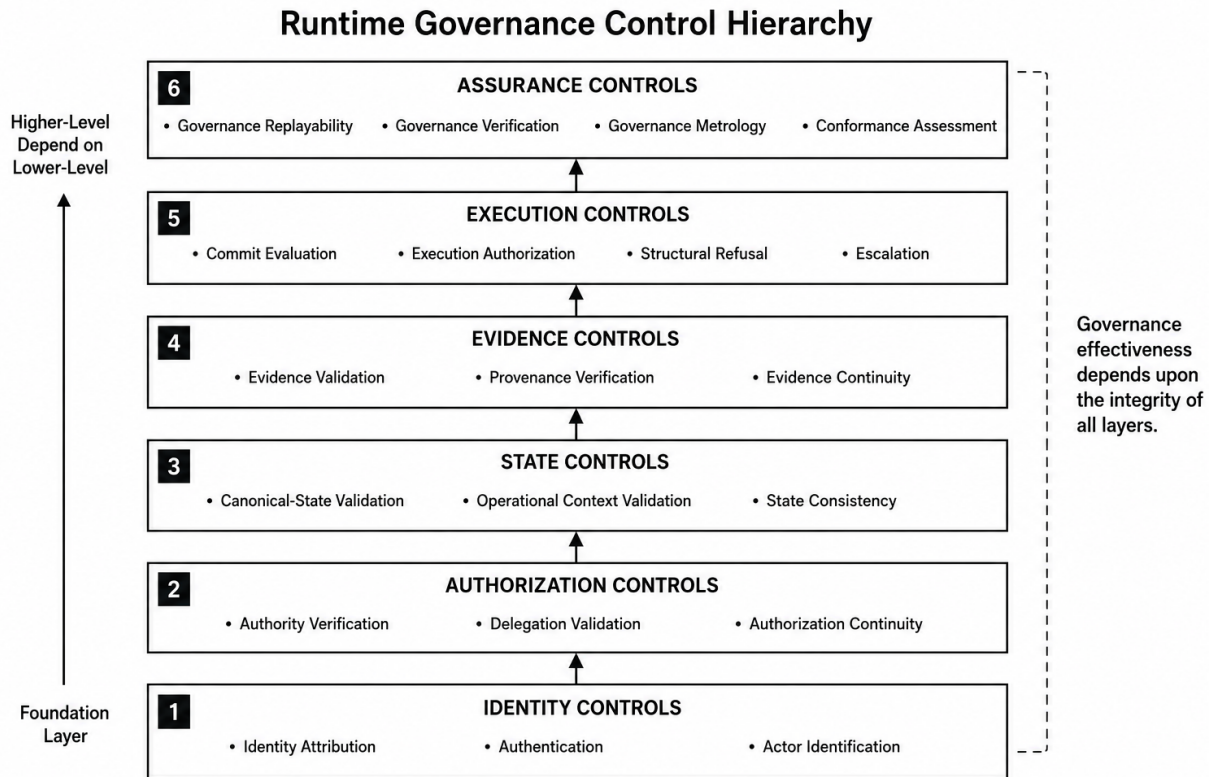


Figure 2: Illustrative runtime governance control hierarchy.

The hierarchy illustrated in Figure 2 is intentionally structured so that higher-order governance capabilities depend upon the correct operation of lower governance layers. Authorization controls establish whether an actor, system, or process possesses the authority necessary to request an action. Execution governance operates above authorization because possession of authority

alone does not guarantee that execution remains appropriate under current operational conditions. Execution-time evaluation therefore provides an additional layer of governance capable of assessing admissibility, operational legitimacy, policy compliance, environmental constraints, and governance context immediately prior to commitment. Runtime assurance functions occupy the highest layer because assurance depends upon evidence generated by the underlying governance processes. Assurance mechanisms cannot validate governance decisions unless authorization, evaluation, enforcement, and evidence-generation activities have already occurred. Viewed collectively, the hierarchy supports the principle of complete mediation by ensuring that operationally consequential actions are evaluated through each governance layer before execution is permitted. Rather than relying solely upon prior authorization decisions, the architecture continuously evaluates whether execution remains legitimate at the point where actions become operationally binding.

This model highlights the dependency relationships among governance capabilities while emphasizing the role of execution controls at the point of operational commitment.

11.11 From Control Categories to Secure-by-Design Objectives

The taxonomy presented in this section provides a structured way to organize runtime governance capabilities.

The next question is how these capabilities contribute to broader Secure-by-Design objectives already recognized by industry, government, and standards organizations (CISA, 2023; NIST, 2023b; ISO, 2023).

The following section examines explicit relationships between runtime governance controls and Secure-by-Design principles, demonstrating how execution-time governance mechanisms may operationalize established security objectives in increasingly autonomous environments.

12 Secure-by-Design Mapping

12.1 Introduction

Secure-by-Design principles have emerged as a central theme in contemporary cybersecurity and technology governance discussions. Organizations such as the Cybersecurity and Infrastructure Security Agency (CISA), NIST, and numerous industry groups have emphasized the importance of designing security into systems from the outset rather than relying exclusively on compensating controls or downstream operational mitigation (CISA, 2023; NIST, 2024a, 2018b; ISO, 2023).

The increasing autonomy of AI-enabled systems creates a natural extension of this philosophy.

Historically, Secure-by-Design efforts focused primarily on software architecture, secure development practices, vulnerability reduction, identity management, and operational resilience. As autonomous systems acquire greater authority to initiate actions, coordinate workflows, and interact with operational systems, Secure-by-Design objectives increasingly require mechanisms that preserve security and governance intent during execution.

Runtime governance can be viewed as one approach for operationalizing Secure-by-Design principles at the point where autonomous actions become operationally consequential.

This section examines how runtime governance control categories may contribute to established Secure-by-Design objectives.

12.2 Secure Defaults

Secure-by-Design emphasizes the importance of secure default behavior (CISA, 2023; NIST, 2020a).

Systems should behave safely when uncertainty exists. The burden of establishing legitimacy should generally rest upon demonstrating that an action is acceptable rather than assuming that execution should proceed automatically.

Runtime governance contributes to secure defaults through mechanisms that prevent execution unless required governance conditions are satisfied.

Examples include:

- admissibility evaluation
- execution-time authorization
- structural refusal
- evidence validation
- canonical-state verification

Under this approach, the default outcome for insufficiently justified actions becomes refusal rather than execution.

Secure defaults become increasingly important in autonomous environments because systems may continue operating after assumptions, evidence, authorization conditions, or governance context

have changed. Runtime governance mechanisms may support secure-default behavior by requiring re-evaluation when governing conditions become uncertain and by constraining autonomous coordination when governance requirements can no longer be verified reliably (CISA, 2023; NIST, 2021, 2023b).

This principle becomes increasingly important in autonomous environments where large numbers of operational decisions may occur without direct human review.

12.3 Least Privilege

Least privilege remains one of the most widely recognized security principles (Saltzer and Schroeder, 1975; NIST, 2020a,b).

Traditional implementations focus on limiting permissions granted to users, services, applications, or systems.

Agentic environments introduce additional complexity because authority may be delegated, inherited, propagated, or composed dynamically across workflows.

Runtime governance contributes to least privilege through:

- execution-bound authorization
- delegated authority validation
- authority-scope verification
- escalation governance
- authorization continuity

Least-privilege principles also apply to delegated authority relationships within autonomous and multi-agent systems. Authority granted to one agent, workflow, or governance domain should not implicitly expand through recursive delegation, cross-domain propagation, or authority inheritance mechanisms. Runtime governance may therefore contribute to least-privilege objectives by preserving authority containment and ensuring that execution authority remains bounded by its originating governance constraints (Saltzer and Schroeder, 1975; NIST, 2020a,b).

These mechanisms help ensure that autonomous actions remain constrained by currently valid authority relationships rather than relying solely upon previously granted permissions.

12.4 Complete Mediation

The principle of complete mediation requires that access and operational decisions be evaluated whenever sensitive actions occur (Saltzer and Schroeder, 1975; NIST, 2020a,b).

In traditional computing environments, complete mediation is commonly associated with access-control enforcement.

In autonomous environments, mediation must increasingly extend beyond access requests to encompass operational actions.

Runtime governance contributes to complete mediation through:

- admissibility evaluation
- execution-time policy evaluation
- operational state validation
- evidence verification
- commit-bound governance decisions

The governance decision itself is not sufficient to satisfy complete mediation unless the decision can be enforced reliably at the point of execution. Effective mediation therefore requires that governance enforcement mechanisms possess sufficient context to evaluate admissibility correctly, preserve the integrity of enforcement decisions, and prevent operational paths that bypass governance evaluation entirely (Saltzer and Schroeder, 1975; NIST, 2020a; Anderson, 2020; Bishop, 2018).

These considerations reinforce the importance of governance-aware enforcement points, enforcement-context availability, and non-bypassable governance enforcement mechanisms within runtime-governance architectures.

These mechanisms help ensure that consequential actions undergo governance evaluation immediately before execution.

12.5 Accountability

Accountability is a recurring objective across cybersecurity, governance, compliance, and assurance frameworks (NIST, 2023b; ISO, 2023; NIST, 2020a, 2018a).

Autonomous systems create unique accountability challenges because actions may emerge from complex chains of delegated decisions involving multiple agents and systems.

Runtime governance contributes to accountability through:

- identity attribution
- authority attribution
- governance lineage
- execution attribution
- governance lifecycle integrity

These capabilities help preserve visibility into who initiated actions, under what authority actions occurred, and how decisions were reached.

12.6 Auditability

Auditability refers to the ability to examine and evaluate system behavior after execution (NIST, 2020a, 2022; ISO, 2023).

Traditional logging mechanisms often record events but provide limited insight into governance reasoning.

Runtime governance contributes to auditability through:

- governance records
- evidence retention
- authorization traceability
- replayable governance decisions
- lifecycle integrity preservation

These capabilities enable organizations to understand not merely what happened but also why governance decisions were reached.

12.7 Traceability

Traceability extends accountability by establishing relationships among actions, evidence, authority, constraints, and outcomes (NIST, 2020a; ISO, 2023).

Agentic systems frequently operate across multiple systems and workflows, increasing the difficulty of maintaining traceability.

Runtime governance contributes to traceability through:

- governance continuity
- evidence continuity
- authorization lineage
- decision traceability
- governance replayability

Such capabilities support auditing, investigations, assurance activities, and regulatory oversight.

12.8 Resilience

Secure-by-Design initiatives emphasize resilience as a fundamental system characteristic (CISA, 2023; NIST, 2024a, 2021).

Resilient systems continue operating safely despite failures, attacks, unexpected conditions, or degraded environments.

Runtime governance contributes to resilience by providing mechanisms capable of evaluating actions under changing operational circumstances.

Examples include:

- canonical-state validation
- operational-condition verification
- context integrity checking
- evidence revalidation
- governance continuity preservation

These capabilities help ensure that governance decisions remain aligned with current conditions rather than historical assumptions.

12.9 Safe Failure

Systems should fail safely when uncertainty, inconsistency, or governance violations occur (Leveson, 2011; NIST, 2021).

This principle becomes increasingly important as autonomous systems acquire the ability to affect operational environments directly.

Runtime governance contributes to safe failure through:

- structural refusal
- escalation workflows
- execution interruption
- constraint enforcement
- governance exception handling

These mechanisms help prevent uncertain or inadmissible actions from producing undesirable operational consequences.

12.10 Human Accountability and Oversight

Many governance frameworks emphasize the importance of maintaining meaningful human accountability for consequential decisions (NIST, 2023b; ISO, 2023; Executive Office of the President, 2023; Defense Innovation Board, 2019).

Runtime governance does not eliminate the need for human oversight.

Rather, it provides mechanisms that may help determine when human intervention should occur. Examples include:

- escalation governance
- exception handling
- human authorization requirements
- consequence-based review thresholds
- delegated authority constraints

These mechanisms support human accountability while allowing autonomous systems to operate efficiently within defined governance boundaries.

12.11 Recovery and Forensic Reconstruction

Recovery is a core objective within cybersecurity and operational resilience frameworks (NIST, 2024a, 2021, 2018a).

Effective recovery depends upon understanding what occurred, why it occurred, and what actions produced operational consequences.

Runtime governance contributes to recovery through:

- replayable governance records
- lifecycle integrity preservation
- evidence continuity
- deterministic decision reconstruction
- governance traceability

These capabilities support incident response, root-cause analysis, remediation planning, and organizational learning.

12.12 Summary Mapping

Table 2 summarizes the relationship between Secure-by-Design objectives and runtime governance capabilities.

Table 2: Illustrative Mapping Between Secure-by-Design Objectives and Runtime Governance Capabilities

Secure-by-Design Objective	Runtime Governance Contribution
Secure Defaults	Structural Refusal, Admissibility Evaluation, Continuation Integrity Controls
Least Privilege	Execution-Bound Authorization, Delegated Authority Validation, Authority Containment, Governance Isolation
Complete Mediation	Execution-Time Governance Evaluation, Enforcement Integrity, Governance Enforcement Binding
Accountability	Identity Attribution, Governance Lineage
Auditability	Governance Records, Replayability
Traceability	Evidence Continuity, Governance Continuity
Resilience	Canonical-State Validation, Operational Condition Verification
Safe Failure	Structural Refusal, Escalation Governance
Human Oversight	Escalation Controls, Human Authorization Gates
Recovery	Replayable Governance Records, Lifecycle Integrity

12.13 Runtime Governance as a Secure-by-Design Layer

The mappings described above suggest that runtime governance should not be viewed as a separate or competing governance framework.

Rather, runtime governance may be understood as an implementation-oriented control layer that operationalizes established Secure-by-Design objectives during execution.

This distinction is important.

Secure-by-Design defines desirable security outcomes.

Runtime governance provides mechanisms that may help maintain those outcomes as autonomous systems make operational decisions.

As AI systems continue to evolve from analytical tools into operational actors, preserving Secure-by-Design principles may increasingly require governance controls capable of evaluating actions at execution time rather than relying exclusively on controls applied before or after execution.

Collectively, these mappings suggest that runtime governance may extend Secure-by-Design principles beyond system construction and deployment into operational execution. In addition to supporting admissibility evaluation and execution-bound authorization, runtime governance may help preserve authority containment, governance integrity, enforcement integrity, and non-bypassable control enforcement throughout the operational lifecycle of autonomous systems (CISA, 2023; NIST, 2020a, 2023b).

The next section examines how runtime governance concepts relate to existing standards, frameworks, and guidance documents, including NIST AI RMF, NIST CSF, NIST SP 800-53, ISO/IEC 42001, MITRE ATLAS, OWASP Agentic AI, and CISA AI security guidance.

13 Mapping to Existing Standards and Frameworks

13.1 Introduction

The concepts introduced throughout this paper are not intended to replace existing governance, security, risk management, or assurance frameworks. Existing frameworks provide essential objectives, principles, controls, and practices that remain foundational to trustworthy AI deployment.

The purpose of this section is to examine how runtime governance concepts may complement existing frameworks by addressing execution-time governance concerns that are often left implementation-specific.

For each framework, three questions are considered:

1. What objectives does the framework already address?
2. How can runtime governance contribute to those objectives?
3. What execution-time governance gaps may remain?

The analysis is intended to illustrate complementarity rather than competition.

13.2 NIST AI Risk Management Framework (AI RMF)

13.2.1 Overview

The NIST AI Risk Management Framework (AI RMF) provides a structured approach for managing risks associated with AI systems throughout their lifecycle (NIST, 2023b,a, 2024b).

The framework consists of four primary functions:

- Govern
- Map
- Measure
- Manage

The AI RMF emphasizes organizational accountability, risk identification, measurement processes, governance structures, and continuous improvement.

13.2.2 Runtime Governance Contribution

Runtime governance may support AI RMF objectives by providing execution-time mechanisms that operationalize governance decisions during system operation.

Examples include:

- execution-time policy enforcement
- operational legitimacy evaluation
- governance continuity
- evidence validation
- authorization verification
- governance replayability

These capabilities help translate governance intent into operational behavior.

The runtime-governance model also supports AI RMF objectives associated with governance, monitoring, and operational risk management by emphasizing continuous evaluation of authority relationships, enforcement integrity, governance continuity, and operational legitimacy throughout execution (NIST, 2023b).

13.2.3 Remaining Runtime Governance Gap

The AI RMF identifies governance objectives but generally does not prescribe specific execution-time governance architectures.

As a result, organizations remain responsible for determining:

- how actions are evaluated
- how operational legitimacy is determined
- how execution-time authorization occurs
- how governance continuity is preserved
- how governance decisions are enforced

Table 3 summarizes this relationship.

Table 3: NIST AI RMF and Runtime Governance

AI RMF Function	Runtime Governance Contribution	Representative Runtime Gap Addressed
Govern	Operational governance enforcement	Execution-time governance evaluation
Map	Context preservation and state validation	Operational context drift
Measure	Runtime assurance and governance metrology	Governance effectiveness measurement
Manage	Execution-time decision mediation	Operational action governance

13.3 NIST Cybersecurity Framework 2.0

13.3.1 Overview

The NIST Cybersecurity Framework (CSF) provides a widely adopted structure for managing cybersecurity risk (NIST, 2024a).

The framework functions include:

- Govern
- Identify
- Protect
- Detect
- Respond
- Recover

13.3.2 Runtime Governance Contribution

Runtime governance extends established cybersecurity principles into autonomous decision environments.

Examples include:

- continuous authorization evaluation
- governance accountability
- execution mediation
- state verification
- governance auditability

13.3.3 Remaining Runtime Governance Gap

The CSF focuses primarily on cybersecurity outcomes rather than operational governance decisions.

Questions such as:

- Should this autonomous action execute?
- Is the action admissible?
- Does authority remain valid?
- Has operational context changed?

are generally implementation-specific.

Although the NIST Cybersecurity Framework 2.0 defines a comprehensive set of cybersecurity outcomes, it does not explicitly prescribe how autonomous actions should be evaluated, authorized, constrained, or refused at the moment of execution. Runtime governance may be viewed as a complementary implementation layer that supports these functions by providing execution-time governance mechanisms. Table 4 summarizes representative areas of alignment and illustrates several remaining runtime governance gaps.

Table 4: NIST CSF and Runtime Governance

CSF Function	Runtime Governance Contribution	Remaining Runtime Gap
Govern	Operational governance enforcement	Execution semantics unspecified
Identify	Context and authority determination	Authoritative operational context determination
Protect	Execution-time authorization	Action legitimacy determination
Detect	Governance condition monitoring	Commit-bound evaluation
Respond	Governance escalation mechanisms	Autonomous action mediation
Recover	Governance replayability	Execution decision reconstruction

As illustrated in Table 4, runtime governance does not replace the cybersecurity objectives defined by the CSF. Rather, it provides execution-time mechanisms that help operationalize those objectives when autonomous systems propose consequential actions. The remaining gaps identified in the table generally concern the evaluation of specific actions under current operational conditions, particularly at the point where actions transition from proposal to execution.

13.4 NIST Special Publication 800-53

13.4.1 Overview

NIST SP 800-53 provides an extensive catalog of security and privacy controls for information systems and organizations (NIST, 2020a, 2022).

Several control families have direct relevance to runtime governance concepts:

- Access Control (AC)
- Audit and Accountability (AU)
- Identification and Authentication (IA)
- Risk Assessment (RA)
- System and Information Integrity (SI)
- System and Communications Protection (SC)

Recent runtime-governance requirements further reinforce alignment with several NIST SP 800-53 control families. Examples include authority-containment mechanisms that support access-control objectives, governance-isolation mechanisms that preserve trust-boundary separation, enforcement-integrity mechanisms that strengthen complete-mediation objectives, and governance self-protection capabilities that help preserve the integrity of governance functions themselves (NIST, 2020a, 2021).

13.4.2 Runtime Governance Contribution

Runtime governance may be viewed as an operational extension of many existing SP 800-53 control objectives.

Examples include:

- execution-bound authorization extending AC controls
- governance attribution extending AU controls
- canonical-state validation extending SI controls
- governance replayability extending accountability objectives

13.4.3 Remaining Runtime Governance Gap

SP 800-53 defines extensive control requirements but generally does not define execution-time governance architectures for autonomous actions.

As a result, organizations must determine:

- how authorization remains bound to execution
- how admissibility is evaluated
- how governance continuity is maintained
- how autonomous actions are constrained

13.5 ISO/IEC 42001 Artificial Intelligence Management Systems

13.5.1 Overview

ISO/IEC 42001 establishes requirements for Artificial Intelligence Management Systems (AIMS) and provides a governance-oriented framework for managing AI-related risks, responsibilities, controls, accountability mechanisms, and continual improvement processes throughout the AI system lifecycle (ISO, 2023).

The standard emphasizes organizational governance, risk management, operational oversight, documentation, accountability, transparency, monitoring, and the effectiveness of AI-related controls. Rather than focusing exclusively on technical implementation details, ISO/IEC 42001 seeks to ensure that organizations establish systematic governance processes capable of managing AI systems responsibly and consistently.

These objectives align closely with many of the governance concerns discussed throughout this paper.

13.5.2 Runtime Governance Contribution

Runtime governance may support ISO/IEC 42001 objectives by extending governance activities into operational execution.

Examples include:

- execution-time admissibility evaluation
- execution-bound authorization
- governance lifecycle integrity
- governance continuity
- evidence continuity
- governance replayability
- governance isolation
- governance integrity preservation

These mechanisms may help organizations demonstrate that governance controls remain effective not only during development, deployment, and review activities, but also during ongoing operational execution.

Runtime governance may also strengthen accountability objectives by producing replayable governance records that document the basis for authorization, refusal, escalation, and other governance decisions. Such capabilities may contribute to transparency, auditability, traceability, and operational oversight throughout the execution lifecycle.

The introduction of governance-integrity, governance-isolation, authority-containment, and governance self-protection capabilities further strengthens alignment with ISO/IEC 42001 objectives related to control effectiveness, governance accountability, and preservation of governance processes throughout system operation (ISO, 2023; NIST, 2020a, 2021).

13.5.3 Remaining Runtime Governance Gap

ISO/IEC 42001 establishes governance objectives, management-system requirements, and organizational accountability expectations. However, the standard generally does not prescribe a specific execution-time governance architecture responsible for determining whether a proposed autonomous action remains legitimate under current operational conditions.

Questions such as the following remain implementation concerns:

- How should admissibility be evaluated immediately before execution?

- How should authority be validated at the point of operational commitment?
- How should governance continuity be preserved across autonomous workflows?
- How should governance decisions be reconstructed after execution?
- How should governance controls remain protected from unauthorized modification during operation?

Runtime governance does not replace ISO/IEC 42001. Rather, it may provide an execution-layer architecture that operationalizes management-system objectives through deterministic governance evaluation, execution-bound authorization, structural refusal, governance continuity, and replayable governance records.

From this perspective, runtime governance may serve as a complementary implementation approach for organizations seeking to demonstrate that governance controls remain active and effective throughout operational execution.

13.6 MITRE ATLAS

13.6.1 Overview

MITRE ATLAS provides a knowledge base describing adversarial tactics, techniques, and procedures targeting AI-enabled systems (MITRE, 2024).

The framework improves understanding of:

- attack pathways
- exploitation methods
- adversarial behavior
- defensive opportunities

13.6.2 Runtime Governance Contribution

Runtime governance provides execution-time opportunities to identify and constrain actions influenced by adversarial conditions.

Examples include:

- context validation
- evidence verification
- canonical-state validation
- structural refusal
- governance escalation

Governance-isolation, enforcement-integrity, and governance self-protection capabilities may provide additional resilience against attempts to influence operational behavior through manipulation of governance context, authority relationships, or enforcement mechanisms (MITRE, 2024).

13.6.3 Remaining Runtime Governance Gap

ATLAS provides valuable threat knowledge but does not define execution-time governance mechanisms.

Organizations remain responsible for implementing controls capable of:

- evaluating legitimacy
- constraining execution
- preserving governance continuity
- maintaining accountability

during adversarial conditions.

13.7 OWASP Agentic AI Guidance

13.7.1 Overview

OWASP guidance for agentic AI highlights emerging risks associated with autonomous workflows, tool invocation, memory systems, prompt injection, authority escalation, and agent coordination (OWASP, 2025).

The guidance reflects growing recognition that autonomous systems introduce security challenges beyond those traditionally associated with software applications or machine-learning models. Agentic systems may interact with external tools, consume dynamic context, maintain memory across sessions, coordinate with other agents, and perform actions with operational consequences.

Many of these risks arise not solely from model behavior but from the relationship between autonomous decision making and operational execution. Consequently, OWASP emphasizes the importance of governance mechanisms capable of constraining, validating, monitoring, and auditing autonomous actions throughout system operation.

13.7.2 Runtime Governance Contribution

Runtime governance directly supports many concerns identified by OWASP.

Examples include:

- admissibility evaluation for proposed actions
- execution-bound authorization

- authority validation
- canonical-state governance
- evidence continuity
- governance context preservation
- structural refusal
- governance replayability

These mechanisms may help constrain autonomous behavior before operational commitment occurs rather than relying exclusively upon post-execution detection and remediation.

Runtime governance may be particularly relevant in environments involving autonomous tool use, delegated authority, multi-agent coordination, long-running workflows, and dynamically changing operational conditions.

Governance-isolation, authority-containment, enforcement-integrity, and governance-continuity mechanisms may further help preserve operational legitimacy as autonomous systems interact across organizational, administrative, and technical boundaries (OWASP, 2025; NIST, 2021, 2020a).

13.7.3 Remaining Runtime Governance Gap

OWASP guidance identifies important risks and defensive practices associated with agentic AI systems. However, the guidance generally does not define a comprehensive execution-time governance architecture responsible for determining whether a specific autonomous action remains legitimate under current conditions.

Questions such as the following remain architectural concerns:

- How should authority be validated immediately before execution?
- How should governance context be preserved across autonomous workflows?
- How should canonical state be established and maintained?
- How should governance decisions be reconstructed after execution?
- How should governance continuity be preserved across cooperating agents?

Runtime governance may help address these questions by providing deterministic execution-time evaluation mechanisms that assess admissibility, validate authority, preserve governance context, enforce structural refusal, and generate replayable governance records before operational commitment occurs.

From this perspective, runtime governance complements OWASP guidance by providing an architectural model for operationalizing governance controls throughout autonomous system execution.

13.8 CISA AI Security Guidance

13.8.1 Overview

CISA has increasingly emphasized Secure-by-Design principles, operational resilience, accountability, transparency, and secure deployment practices for AI-enabled systems (CISA, 2023; DHS, 2024).

These objectives align closely with the concerns discussed throughout this paper.

CISA’s Secure-by-Design guidance encourages organizations to reduce the likelihood and impact of security failures by incorporating security considerations throughout system design, implementation, deployment, and operation. Core themes include secure defaults, least privilege, complete mediation, accountability, resilience, and the ability to recover from failures while maintaining operational integrity.

CISA has also highlighted the growing importance of AI security within critical infrastructure environments, where autonomous or semi-autonomous systems may influence operational decisions with significant safety, security, economic, or societal consequences.

As AI systems become increasingly capable of autonomous action, the challenge extends beyond developing trustworthy models and secure software. Organizations must also determine whether specific actions proposed by those systems remain legitimate, authorized, and appropriate under current operational conditions.

13.8.2 Runtime Governance Contribution

Runtime governance may support Secure-by-Design objectives by providing deterministic execution-time evaluation of proposed actions before operational commitment occurs.

Several Secure-by-Design principles map naturally to runtime governance mechanisms:

- Secure Defaults through structural refusal when governance requirements are not satisfied.
- Least Privilege through execution-bound authorization and context-sensitive authority validation.
- Complete Mediation through admissibility evaluation at the commit boundary.
- Accountability through governance receipts and governance lifecycle integrity.
- Auditability through governance replayability and deterministic decision reconstruction.
- Resilience through governance continuity, evidence continuity, and fail-safe refusal mechanisms.

These capabilities help ensure that governance remains active during operational execution rather than relying exclusively upon controls established during development, deployment, or post-execution review.

From this perspective, runtime governance can be viewed as an architectural mechanism for operationalizing Secure-by-Design principles within autonomous and agentic systems.

13.8.3 Remaining Runtime Governance Gap

CISA guidance establishes important security objectives and architectural principles for secure AI systems. However, the guidance generally does not prescribe how governance decisions should be evaluated at the point where a proposed autonomous action transitions into execution.

Questions such as the following remain implementation concerns:

- How should operational legitimacy be evaluated at execution time?
- How should authority be validated immediately before execution?
- How should governance continuity be preserved across multi-step autonomous workflows?
- How should evidence validity be maintained throughout long-running operations?
- How should governance decisions be reconstructed after execution?

Runtime governance does not replace Secure-by-Design guidance. Rather, it may provide an execution-layer architecture that operationalizes Secure-by-Design objectives by evaluating admissibility, validating authority, preserving governance context, enforcing structural refusal, and maintaining accountability at the commit boundary.

As autonomous systems become increasingly operationally consequential, execution-time governance mechanisms may become an important complement to existing Secure-by-Design practices.

13.9 European Union Artificial Intelligence Act

13.9.1 Overview

The European Union Artificial Intelligence Act establishes a risk-based regulatory framework for AI systems, including requirements related to governance, transparency, human oversight, risk management, recordkeeping, conformity assessment, and post-deployment monitoring (European Union, 2024).

The Act introduces obligations that vary according to system risk classification while emphasizing accountability, traceability, documentation, and ongoing operational oversight.

13.9.2 Runtime Governance Contribution

Runtime governance may support AI Act objectives through execution-time mechanisms that preserve accountability, traceability, operational legitimacy, evidence continuity, and human oversight throughout autonomous operations.

Examples include:

- governance lifecycle integrity
- execution-time authorization

- evidence continuity
- governance replayability
- escalation governance
- operational legitimacy evaluation

These capabilities may help organizations demonstrate that governance requirements remain effective during operational execution rather than solely during development or deployment.

13.9.3 Remaining Runtime Governance Gap

The AI Act establishes governance obligations and compliance expectations but generally does not prescribe specific execution-time governance architectures.

Organizations remain responsible for determining how autonomous actions are evaluated, authorized, constrained, monitored, and reconstructed during operational execution.

Consequently, implementation-oriented runtime governance mechanisms may provide a useful architectural approach for operationalizing many of the Act’s governance objectives.

13.10 Cross-Framework Observations

Several observations emerge consistently across the frameworks examined.

First, existing frameworks already provide substantial governance, security, assurance, and accountability guidance (NIST, 2023b, 2024a, 2020a; ISO, 2023; MITRE, 2024; European Union, 2024).

Second, most frameworks emphasize governance objectives, organizational responsibilities, security outcomes, risk management processes, and accountability expectations rather than execution-time governance architectures (NIST, 2023b; ISO, 2023; CISA, 2023; European Union, 2024).

- governance objectives
- organizational responsibilities
- security outcomes
- risk management processes
- accountability expectations

Third, many runtime governance concepts naturally extend objectives already present within existing frameworks (NIST, 2023b, 2024a; ISO, 2023; NIST, 2020a; European Union, 2024).

Examples include:

- least privilege
- accountability

- auditability
- traceability
- resilience
- complete mediation
- safe failure
- authority containment
- governance isolation
- governance integrity
- enforcement integrity

Finally, a recurring implementation question appears across all examined frameworks:

How should organizations preserve governance intent when autonomous systems perform consequential actions at runtime?

This question represents the central motivation for runtime governance architectures.

13.11 Standards Alignment Rather Than Standards Replacement

A common concern regarding emerging governance approaches is the possibility of framework proliferation.

The analysis presented in this section suggests a different interpretation.

Runtime governance should not be viewed as a replacement for:

- NIST AI RMF
- NIST CSF
- NIST SP 800-53
- ISO/IEC 42001
- European Union Artificial Intelligence Act
- MITRE ATLAS
- OWASP Agentic AI
- CISA Secure-by-Design guidance

Instead, runtime governance may be understood as an implementation-oriented architectural layer that helps operationalize objectives already established by these frameworks.

Under this perspective, existing frameworks continue to define governance expectations, security objectives, risk management principles, and assurance outcomes.

Runtime governance architectures provide execution-time mechanisms for enforcing those expectations as autonomous systems become increasingly operational.

The next section introduces a reference runtime governance architecture that illustrates how these concepts may be organized into a practical execution-time governance model while remaining aligned with the objectives and expectations established by NIST AI RMF, NIST CSF, NIST SP 800-53, ISO/IEC 42001, the European Union Artificial Intelligence Act, MITRE ATLAS, OWASP Agentic AI guidance, and CISA Secure-by-Design principles.

14 Reference Runtime Governance Architecture

14.1 Introduction

The preceding sections introduced runtime governance as a complementary execution-time control layer that supports Secure-by-Design objectives in increasingly autonomous environments. The concepts discussed thus far have been intentionally implementation-neutral. They describe governance objectives, assurance requirements, control categories, and architectural principles without prescribing a specific technical solution.

This section presents a reference runtime governance architecture intended to illustrate how these concepts may be organized into a coherent operational model.

The architecture described here is conceptual rather than normative. Organizations may implement similar governance capabilities using centralized, distributed, federated, embedded, or hybrid approaches. The objective is not to prescribe a single design pattern but rather to identify architectural functions that support execution-time governance (NIST, 2018b, 2021; Leveson, 2011).

14.2 Architectural Design Objectives

A runtime governance architecture should seek to satisfy several operational objectives (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2020a, 2021):

- Preserve accountability
- Maintain governance continuity
- Support execution-time authorization
- Validate operational legitimacy
- Preserve evidence continuity
- Enable structural refusal
- Support deterministic evaluation
- Enable replayability and auditability

These objectives align with the runtime governance principles and Secure-by-Design mappings discussed in previous sections.

14.3 Conceptual Architecture Overview

At a high level, runtime governance introduces a governance layer between action proposal and operational execution.

Traditional software systems, agentic systems, and runtime-governed agentic systems differ primarily in where operational decisions are evaluated before execution.

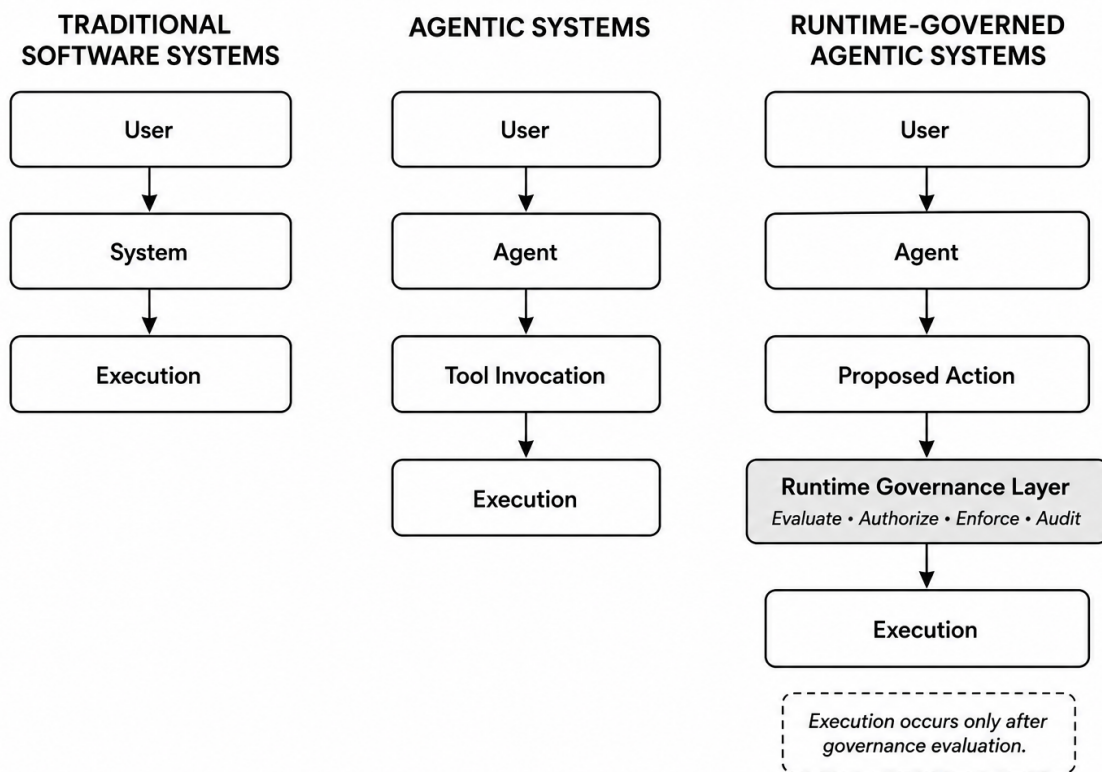


Figure 3: Comparison of traditional execution workflows, agentic execution workflows, and runtime-governed execution workflows.

Figure 3 illustrates this progression.

The distinction between the second and third models is particularly significant. Traditional systems generally operate within predefined workflows where governance decisions are established through design-time controls, authorization models, and operational procedures. Agentic systems introduce a fundamentally different execution model in which autonomous components may generate plans, invoke tools, delegate tasks, coordinate with external services, and adapt behavior dynamically in response to changing conditions. As a result, governance assumptions established before deployment may no longer remain sufficient to determine whether a specific action should be executed. The runtime-governed architecture introduces an explicit governance mediation layer at the commit boundary, ensuring that proposed actions are evaluated against current authority, policy, evidence, operational state, and contextual constraints before execution occurs. This architectural shift transforms governance from a primarily lifecycle-oriented activity into an operational capability that continuously evaluates the legitimacy of autonomous actions as they transition from intent to execution.

The introduction of an explicit governance mediation layer fundamentally changes the security and governance posture of autonomous systems. Rather than relying exclusively upon design-time controls, static authorization decisions, or post-execution auditing, runtime-governed architectures establish a decision point at the commit boundary where operational legitimacy can be evaluated before actions become consequential. This shift enables governance to function as an active execution-time capability rather than solely as a lifecycle-management activity.

14.4 Core Architectural Components

A reference runtime governance architecture may include the following functional components.

14.4.1 Identity and Context Services

Identity and context services establish (NIST, 2020a,b; OASIS, 2013):

- subject identity
- agent identity
- delegated authority relationships
- workflow context
- accountability lineage

These services provide the contextual information necessary to evaluate governance decisions.

Representative functions include:

- authentication
- attribution
- delegation validation

- context acquisition
- authority resolution

14.4.2 Policy and Constraint Services

Policy and constraint services define governance expectations (OASIS, 2013; NIST, 2020a; ISO, 2023).

Examples include:

- organizational policies
- safety constraints
- operational limitations
- regulatory requirements
- mission objectives
- escalation rules

These services establish the conditions against which proposed actions are evaluated.

14.4.3 Evidence Services

Evidence services manage information used to justify governance decisions (NIST, 2022, 2018a, 2021).

Examples include:

- approvals
- certifications
- risk assessments
- policy evaluations
- telemetry
- operational measurements

Evidence services support:

- evidence collection
- evidence validation
- evidence continuity
- provenance verification
- evidence retention

14.4.4 State Services

State services establish authoritative operational conditions (NIST, 2018b, 2021; Leveson, 2011).

Examples include:

- system status
- workflow state
- environmental conditions
- governance state
- operational telemetry
- authorization status

State services support canonical-state governance by providing authoritative information used during evaluation.

14.4.5 Governance Evaluation Services

Governance evaluation services perform execution-time governance analysis (Rushby, 2008; Seshia et al., 2016, 2018).

Examples include:

- policy evaluation
- constraint validation
- evidence verification
- authorization evaluation
- admissibility evaluation
- escalation determination

These services collectively determine whether a proposed action remains operationally legitimate.

14.4.6 Commit Adjudication Services

Commit adjudication services determine whether operational commitment may occur (Saltzer and Schroeder, 1975; Rushby, 2008; OASIS, 2013).

Possible outcomes include:

- authorize execution

- require escalation
- defer execution
- deny execution
- terminate workflow

Commit adjudication represents the final governance decision point before operational consequences occur.

14.4.7 Governance Record Services

Governance record services preserve information required for accountability, auditability, and assurance (NIST, 2022, 2018a, 2020a).

Examples include:

- governance decisions
- evaluated evidence
- authorization outcomes
- applicable constraints
- execution outcomes
- state transitions

These records support governance replayability and lifecycle integrity.

14.5 A Five-Stage Governance Pipeline

Although implementations may vary substantially, runtime governance architectures often exhibit a common sequence of governance activities.

One useful conceptual model consists of five stages:

1. Identity and Context
2. Action Representation
3. Rule Evaluation
4. Commit Adjudication
5. Governance Record Generation

14.5.1 Stage 1: Identity and Context

The first stage establishes the operational context surrounding a proposed action.

Questions include:

- Who initiated the action?
- Which agent is acting?
- What authority applies?
- What workflow context exists?
- What governance lineage applies?

This stage establishes the governance identity of the proposed action.

14.5.2 Stage 2: Action Representation

The second stage creates a structured representation of the proposed action.

Examples include:

- API requests
- workflow transitions
- financial transactions
- infrastructure changes
- operational commands
- tool invocations

The objective is to ensure that governance evaluation operates against a consistent representation of intended behavior.

14.5.3 Stage 3: Rule Evaluation

The third stage evaluates the proposed action against governance criteria.

Examples include:

- policy evaluation
- authorization evaluation
- evidence verification

- state validation
- safety analysis
- admissibility determination

This stage produces the information required for governance decision-making.

14.5.4 Stage 4: Commit Adjudication

The fourth stage determines whether execution may proceed.

Representative outcomes include:

- authorized
- conditionally authorized
- escalated
- deferred
- refused

Commit adjudication represents the governance decision immediately preceding operational commitment.

14.5.5 Stage 5: Governance Record Generation

The fifth stage produces governance records supporting accountability and assurance.

Examples include:

- decision records
- evidence references
- authorization outcomes
- state snapshots
- execution identifiers
- audit artifacts

These records support replayability, investigations, assurance activities, and regulatory oversight.

14.6 Commit Boundary Placement

One of the most important architectural decisions in any runtime governance architecture concerns placement of governance controls relative to the commit boundary (Saltzer and Schroeder, 1975; Rushby, 2008).

The central architectural challenge is not whether governance evaluation occurs earlier in the lifecycle, but whether governance determinations remain valid at the moment of commitment. Operational conditions, authority relationships, evidence, and policy constraints may change between initial authorization and execution. Conversely, governance evaluation performed after commitment may be unable to prevent operational consequences because execution has already occurred.

Consequently, the effectiveness of runtime governance depends not only upon what governance decisions are made, but also upon where those decisions are positioned within the execution lifecycle.

Figure 4 illustrates the relationship between governance evaluation, execution authorization, the commit boundary, and resulting operational effects.

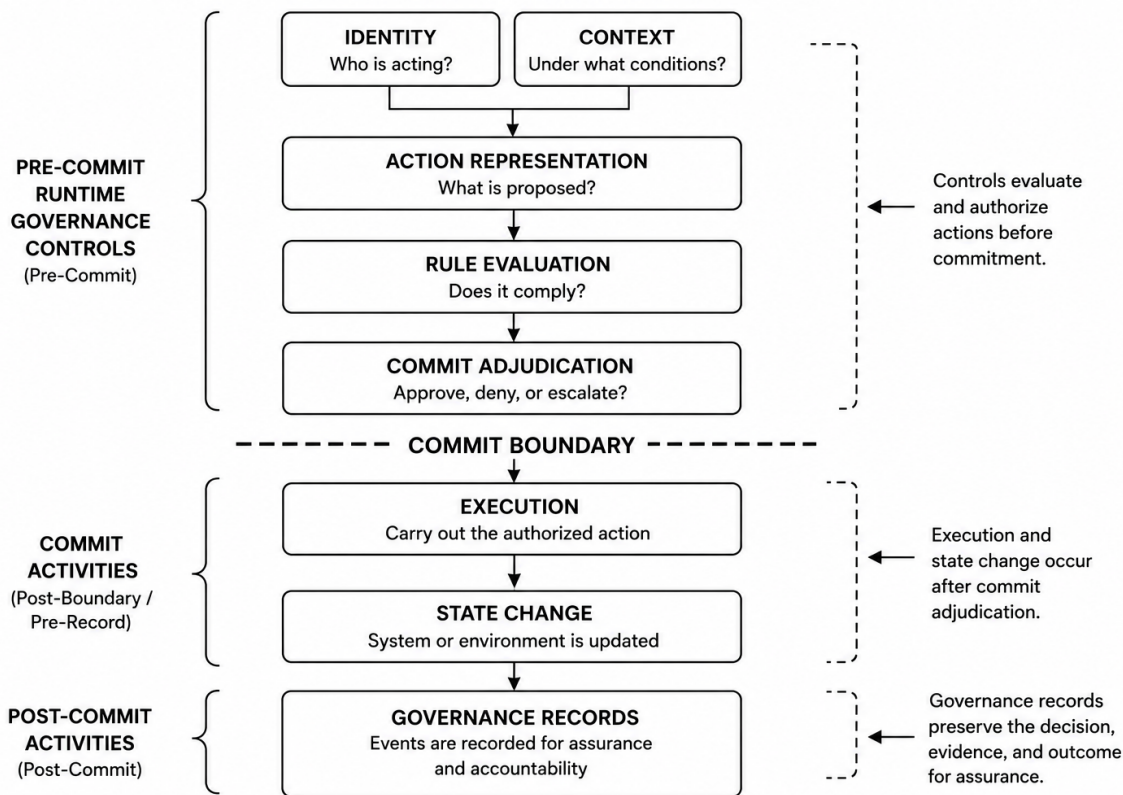


Figure 4: Illustrative placement of runtime governance controls relative to the commit boundary.

The placement illustrated in Figure 4 reflects one of the central architectural arguments of this paper: governance authority must remain effective at the point where proposed actions become operationally consequential. Governance decisions established earlier in the lifecycle may remain important, but their continued validity depends upon current authority, evidence, operational state, and policy conditions at the moment of commitment. By positioning governance mediation immediately before the commit boundary, runtime governance provides a control point at which

admissibility, authorization, evidence validity, and operational legitimacy can be evaluated using current conditions. This placement extends established security principles such as complete mediation into autonomous execution environments while helping ensure that governance decisions remain authoritative at the precise moment proposed intent becomes operational reality. In this sense, the commit boundary serves as the primary locus of governance authority within autonomous systems.

14.7 Centralized and Distributed Implementations

The architectural functions described above may be implemented using multiple deployment models (Open Group, 2022; CSA, 2024; NIST, 2018b).

14.7.1 Centralized Governance

A centralized approach places governance services within a dedicated governance platform.

Advantages include:

- consistent policy enforcement
- centralized auditing
- simplified governance management
- unified assurance processes

Potential challenges include:

- scalability
- latency
- availability dependencies

14.7.2 Distributed Governance

A distributed approach embeds governance capabilities within multiple systems.

Advantages include:

- scalability
- locality of decision making
- reduced latency
- operational flexibility

Potential challenges include:

- consistency
- governance synchronization
- assurance complexity

14.8 Multi-Agent Governance Considerations

Agentic environments introduce governance challenges that extend beyond individual systems (Wang et al., 2023; Xi et al., 2023; OWASP, 2025).

A runtime governance architecture should preserve:

- authority lineage
- accountability lineage
- governance context
- evidence continuity
- authorization continuity

across agent interactions.

Failure to preserve these relationships may result in fragmented accountability and diminished governance visibility.

14.9 Example Implementation Approaches

Multiple architectural approaches may implement the concepts described throughout this section (OASIS, 2013; Rushby, 2008; NIST, 2018b).

Examples include:

- policy decision architectures
- workflow governance platforms
- transaction mediation systems
- authorization gateways
- runtime assurance frameworks
- governance control planes

One example implementation approach is the concept of an AI Governance Control Plane (AGCP), which applies deterministic governance evaluation, execution-bound authorization, admissibility determination, governance continuity, and replayable governance records to autonomous and agentic

systems. AGCP represents one implementation of the broader runtime governance concepts discussed throughout this paper rather than a replacement for existing governance frameworks.

The broader architectural pattern remains applicable regardless of the specific implementation selected.

14.10 From Architecture to Operational Use Cases

The reference architecture presented in this section demonstrates how runtime governance principles can be organized into an operational model.

The next section applies these concepts to representative critical-infrastructure scenarios to illustrate how runtime governance may function in practice across sectors including energy, water, transportation, financial services, healthcare, and government operations.

15 Critical Infrastructure Use Cases

15.1 Introduction

Critical infrastructure sectors increasingly depend upon digital systems to support operational decision-making, resource management, monitoring, coordination, and automation. As AI systems evolve from advisory tools into operational actors, governance challenges become increasingly consequential because the effects of autonomous actions may extend beyond information processing into real-world outcomes (CISA, 2023; NIST, 2024a; CISA, 2024; DHS, 2024).

The objective of this section is not to propose fully autonomous operation of critical infrastructure. Rather, the purpose is to illustrate how runtime governance concepts may be applied to environments where AI systems participate in operational workflows.

The scenarios presented below are intentionally simplified to highlight governance considerations. They are not intended to represent complete operational architectures.

15.2 Energy Sector

15.2.1 Scenario

A regional electrical utility deploys an AI-enabled grid operations agent to assist with balancing generation capacity, transmission loads, maintenance schedules, and outage response.

The agent detects conditions suggesting that power should be rerouted away from a transmission corridor experiencing abnormal thermal conditions.

The agent proposes the following action:

Reconfigure transmission routing to reduce load on Corridor A and increase utilization of Corridor B.

15.2.2 Runtime Evaluation

Before execution occurs, runtime governance mechanisms evaluate:

- Current grid state
- Real-time telemetry
- Operator authority constraints
- Existing maintenance activities
- Safety restrictions
- Regulatory requirements
- Operational contingencies

Canonical-state validation determines that Corridor B is currently operating under a maintenance restriction that was not present when the recommendation was originally generated.

15.2.3 Authorization Decision

The proposed action remains technically feasible.

The proposing agent possesses valid authority.

However, admissibility evaluation determines that the action conflicts with current operational restrictions.

15.2.4 Execution Outcome

Execution is refused.

The recommendation is escalated to a human grid operator.

15.2.5 Auditability Outcome

Governance records preserve:

- Proposed action
- Operational state
- Maintenance restriction
- Admissibility determination
- Refusal decision
- Escalation event

Subsequent review demonstrates that governance controls prevented execution based upon current operational conditions rather than static permissions alone.

15.3 Water Sector

15.3.1 Scenario

A water-treatment utility deploys an AI-assisted operations platform responsible for monitoring treatment processes and recommending operational adjustments.

The system proposes:

Increase chemical dosing by 15% to address deteriorating water-quality indicators.

15.3.2 Runtime Evaluation

Governance services evaluate:

- Current sensor readings
- Treatment plant status
- Regulatory thresholds
- Maintenance activities
- Calibration records
- Supporting evidence

Evidence validation reveals that several upstream sensors recently entered a degraded operating condition.

15.3.3 Authorization Decision

The recommendation is temporarily suspended pending sensor verification.

15.3.4 Execution Outcome

Execution does not occur.

Additional validation is required.

15.3.5 Auditability Outcome

Governance records demonstrate that the decision was based upon evidence quality concerns rather than system failure.

This distinction is important because governance controls preserved operational safety despite uncertainty regarding supporting evidence.

15.4 Transportation Sector

15.4.1 Scenario

A transportation management system coordinates traffic-flow optimization across a metropolitan region.

An AI orchestration agent proposes:

Modify signal timing across multiple intersections to reduce congestion associated with a large public event.

15.4.2 Runtime Evaluation

Governance evaluation examines:

- Traffic conditions
- Emergency routes
- Active incidents
- Construction zones
- Public safety restrictions
- Operational priorities

Canonical-state validation identifies an active emergency response route that would be negatively affected by the proposed change.

15.4.3 Authorization Decision

The proposed action is partially authorized.

Modified signal adjustments are generated that preserve emergency response requirements.

15.4.4 Execution Outcome

Execution proceeds using the modified configuration.

15.4.5 Auditability Outcome

Governance records preserve:

- Original proposal
- Operational conditions
- Admissibility analysis
- Modified authorization
- Executed configuration

The resulting governance trail enables reconstruction of both the original recommendation and the final decision.

15.5 Financial Services Sector

15.5.1 Scenario

A financial institution deploys an AI-enabled operations agent responsible for managing liquidity transfers among internal accounts.

The system proposes:

Transfer \$250 million between reserve accounts to satisfy projected settlement requirements.

15.5.2 Runtime Evaluation

Governance services evaluate:

- Account balances
- Liquidity requirements
- Risk controls
- Approval requirements
- Delegated authority
- Regulatory constraints

Evidence continuity validation reveals that the supporting forecast was generated using stale market data.

15.5.3 Authorization Decision

Execution authorization is denied pending updated analysis.

15.5.4 Execution Outcome

No transfer occurs.

15.5.5 Auditability Outcome

Governance records demonstrate that authorization was denied because supporting evidence was no longer considered operationally valid.

This outcome illustrates how evidence continuity contributes to execution-time governance.

15.6 Healthcare Sector

15.6.1 Scenario

A clinical workflow assistant proposes modification of treatment prioritization schedules within a hospital network during a period of unusually high patient volume.

The proposed action would alter treatment sequencing for multiple patient groups.

15.6.2 Runtime Evaluation

Governance mechanisms evaluate:

- Clinical policies
- Resource availability
- Physician directives
- Escalation requirements
- Regulatory constraints
- Patient safety considerations

Governance evaluation determines that the proposed modification exceeds the authority delegated to the workflow assistant.

15.6.3 Authorization Decision

The recommendation is escalated to clinical leadership.

15.6.4 Execution Outcome

Execution does not occur automatically.

15.6.5 Auditability Outcome

Governance records preserve:

- Proposed action
- Authority analysis
- Escalation determination
- Final disposition

This preserves accountability while maintaining appropriate human oversight.

15.7 Government Operations

15.7.1 Scenario

A government agency employs an AI-enabled workflow system to coordinate grant administration and program management activities.

The system proposes:

Automatically approve a series of funding actions based upon previously observed approval patterns.

15.7.2 Runtime Evaluation

Governance mechanisms evaluate:

- Funding authority
- Approval requirements
- Policy constraints
- Delegation status
- Supporting documentation
- Audit requirements

Governance continuity analysis determines that required approval lineage is incomplete.

15.7.3 Authorization Decision

Execution is refused.

Human review is required.

15.7.4 Execution Outcome

No funding actions occur automatically.

15.7.5 Auditability Outcome

Governance records demonstrate that refusal resulted from incomplete governance lineage rather than policy violation.

This distinction supports transparency and accountability.

15.8 Cross-Sector Observations

Although the preceding scenarios differ substantially, several common governance themes emerge (DHS, 2024; NIST, 2021).

In each case:

1. A proposed action was generated.
2. Runtime governance evaluation occurred.
3. Operational conditions were examined.
4. Authorization was evaluated.
5. A governance decision was reached.
6. Governance records were preserved.

Importantly, the governance decision was not determined solely by whether the proposing system possessed authority.

Additional considerations included:

- operational state
- evidence validity
- safety constraints
- governance continuity
- accountability requirements
- contextual conditions

These observations reinforce the distinction between authorization and admissibility introduced earlier in this paper.

15.9 A Generalized Runtime Governance Pattern

The scenarios presented throughout this section exhibit a common governance sequence.

Figure 5 illustrates a generalized runtime governance pattern that abstracts the common execution-time governance activities discussed throughout this paper.

The generalized sequence shown in Figure 5 demonstrates three important properties of runtime governance. First, authorization and admissibility are treated as distinct decisions. An actor or system may possess authority to request an action, while the proposed action may still fail admissibility evaluation because current state, evidence, policy constraints, or operational conditions do not support execution. Second, governance evaluation occurs before commitment rather than after

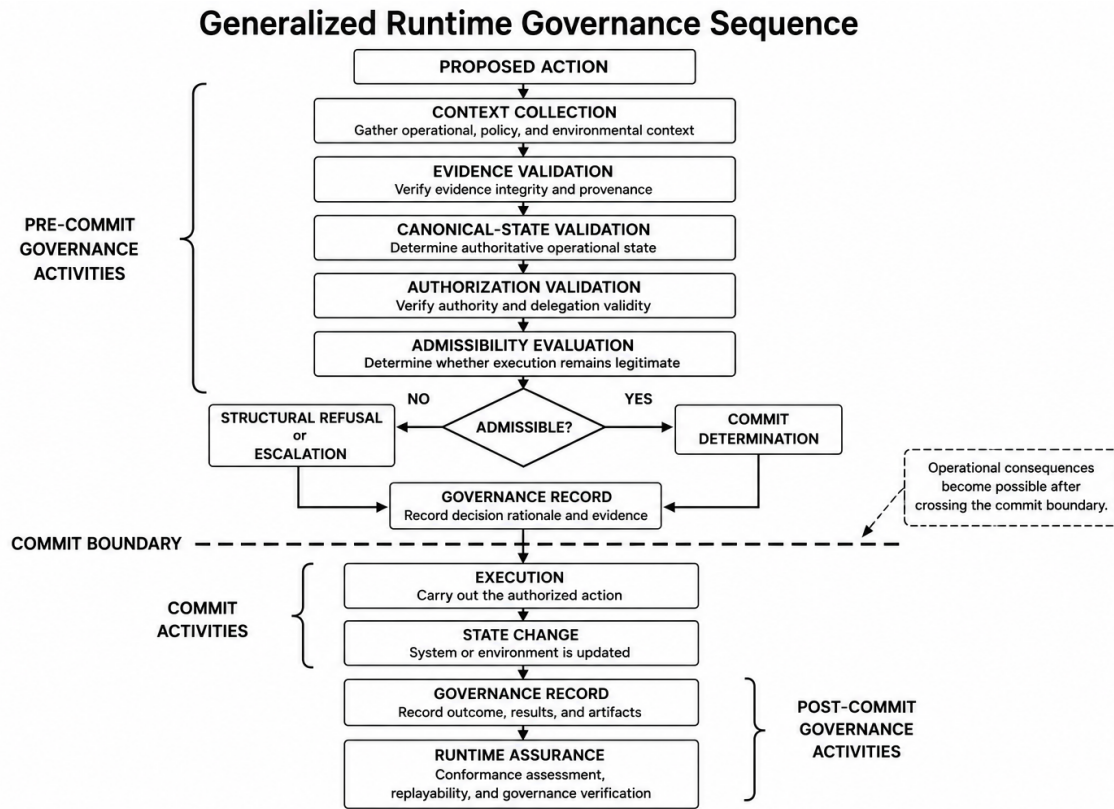


Figure 5: Generalized runtime governance sequence across critical infrastructure use cases.

operational effects have already occurred. This placement allows the governance layer to refuse, defer, escalate, or authorize execution before the proposed action becomes operationally consequential. Third, the sequence produces governance records that support post-execution assurance, auditability, replayability, and accountability. In this sense, runtime governance is not limited to pre-execution blocking; it also creates the evidentiary foundation needed to reconstruct why a governance decision was made and whether the resulting action remained consistent with governance intent.

Although specific implementations may vary, this pattern illustrates how runtime governance mechanisms can preserve operational legitimacy across diverse sectors by establishing a governance decision point before actions cross the commit boundary. The sequence therefore represents a generalized execution-time governance model that can be adapted to different operational environments while preserving the core principles of admissibility evaluation, execution-bound authorization, accountability, and governance continuity.

15.10 Implications for Critical Infrastructure

Critical infrastructure operators have historically focused on securing systems, networks, applications, and operational technologies.

As AI systems acquire increasing authority to influence operational outcomes, governance attention must increasingly extend to the legitimacy of autonomous actions (CISA, 2023; DHS, 2024; NIST, 2024a).

The scenarios presented in this section suggest that runtime governance mechanisms may help organizations:

- preserve accountability
- maintain operational safety
- improve auditability
- support Secure-by-Design objectives
- strengthen operational assurance
- reduce consequences of context drift
- constrain inappropriate autonomous behavior

The next section translates these concepts into a set of standards-oriented runtime governance requirements that organizations may use when designing, evaluating, or assessing runtime governance architectures.

16 Runtime Governance Requirements

16.1 Introduction

The preceding sections introduced the concepts of runtime governance, runtime assurance, execution-time authorization, admissibility evaluation, governance continuity, and operational legitimacy. Those concepts provide an architectural foundation for understanding why increasingly autonomous systems may require governance controls that operate at execution time (NIST, 2023b,a, 2024b; ISO, 2023; MITRE, 2024; Russell, 2019; Amodei et al., 2016; Wang et al., 2023; Xi et al., 2023).

As discussed previously, existing governance frameworks generally define governance objectives, risk management processes, accountability expectations, and assurance goals while leaving implementation details largely to system designers and operators (NIST, 2023b, 2024a, 2020a; ISO, 2023; European Union, 2024). Consequently, organizations increasingly require implementation-oriented specifications capable of translating governance intent into operational controls that function during execution rather than solely during development, deployment, or post-execution review (Russell, 2019; Wang et al., 2023; Xi et al., 2023; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

This section transitions from conceptual analysis to normative requirements.

Runtime governance should be understood as an architectural category rather than a single specification or implementation. Multiple governance architectures may be capable of realizing the execution-time governance concepts discussed throughout this paper. The requirements discussed in this section are presented as one implementation-oriented example intended to illustrate how runtime governance concepts may be translated into operational controls and conformance criteria.

Rather than introducing a new set of illustrative requirements, this paper adopts the requirement structure defined by the AI Governance Control Plane (AGCP) specification. AGCP organizes runtime governance requirements into twelve Requirement Groups (RGs) comprising 118 Conformance Requirements (CRs) (AGCP Project, 2026). These requirements represent one example implementation approach for operationalizing the runtime governance concepts discussed throughout this paper.

The purpose of this section is not to reproduce the entire specification. Instead, it presents the major requirement domains and explains how they collectively address the runtime governance challenges discussed throughout this paper.

16.2 Requirement Organization

The AGCP requirement model organizes runtime governance controls into twelve governance domains (AGCP Project, 2026). Collectively, these domains address governance concerns associated with authorization, accountability, operational legitimacy, evidence continuity, replayability, multi-agent coordination, and runtime assurance.

Collectively, these requirement groups establish a runtime governance architecture intended to preserve governance intent at the point where actions become operationally consequential. The grouping structure reflects recurring governance functions identified throughout the runtime governance model, including execution governance, canonical-state determination, refusal handling, evidence continuity, governance assurance, continuation integrity, and constitutional governance.

Table 5: AGCP Runtime Governance Requirement Groups

Requirement Group	CR Count	Primary Focus
RG-1 Lifecycle Governance	18	Governance state transitions and lifecycle integrity
RG-2 Execution Governance	21	Execution authorization and commitment-bound governance
RG-3 Governance Object Integrity	12	Integrity of governance artifacts and objects
RG-4 Canonical-State Governance	7	Authoritative state determination and validation
RG-5 Refusal and Escalation Governance	9	Structural refusal and escalation handling
RG-6 Governance Evidence and Receipts	9	Evidence continuity and governance records
RG-7 Multi-Agent Runtime Governance	7	Governance continuity across agent interactions
RG-8 Governance Interoperability	3	Cross-system governance interoperability
RG-9 Runtime Governance Metrology	15	Measurement, assurance, and conformance evaluation
RG-10 Continuation Integrity Governance	7	Continuation and workflow integrity controls
RG-11 Composite Governance	5	Governance composition across control domains
RG-12 Governance Compilation and Constitutional Governance	5	Governance compilation and constitutional constraints

Several groups appear repeatedly across multiple runtime governance concepts. In particular, RG-2 (Execution Governance), RG-4 (Canonical-State Governance), RG-10 (Continuation Integrity Governance), and RG-12 (Governance Compilation and Constitutional Governance) provide foundational capabilities that support many other governance functions. Their recurring presence reflects the central role of execution-time decision control, authoritative state determination, workflow integrity, and governance constraint management within deterministic runtime governance architectures. Collectively, the requirement groups align with longstanding security principles including complete mediation, accountability, least privilege, auditability, traceability, and operational assurance while extending those principles into autonomous and agentic operational environments (Saltzer and Schroeder, 1975; Anderson, 2020; Bishop, 2018; NIST, 2020a, 2018b, 2021).

16.3 RG-1 Lifecycle Governance

Lifecycle Governance establishes formal governance state transitions (AGCP Project, 2026).

Traditional governance approaches frequently emphasize approvals, reviews, and auditing. Runtime governance additionally requires explicit governance semantics regarding how actions move between governance states and how governance outcomes remain associated with operational actions throughout their lifecycle (NIST, 2023b, 2022, 2018a; Rushby, 2008; Seshia et al., 2016, 2018).

Representative requirements include:

- CR-001 through CR-003 governing submission outcomes.
- CR-004 through CR-009 governing authorization lifecycle behavior.
- CR-010 through CR-018 governing completion, revocation, expiration, and lifecycle integrity.

These requirements support one of the central themes of this paper:

Governance decisions must remain connected to the actions they authorize.

Lifecycle integrity contributes directly to accountability, traceability, governance continuity, and replayability by ensuring that governance records remain linked to resulting operational outcomes throughout execution and post-execution analysis (NIST, 2022, 2018a; Rushby, 2008; Seshia et al., 2018).

16.4 RG-2 Execution Governance

Execution Governance forms the operational core of runtime governance (AGCP Project, 2026).

This requirement group addresses execution-time authorization and commit-bound governance decisions. It directly operationalizes the distinction introduced earlier between possessing authority and determining whether execution remains legitimate under current operational conditions (Russell, 2019; Amodei et al., 2016; Wang et al., 2023; Xi et al., 2023).

Representative requirements include:

- CR-019 Commit Authorized Action.
- CR-022 Execution Bound to Authorization.
- CR-023 Execution Bound to Governance Context.
- CR-024 Execution Bound to Canonical State.
- CR-025 Execution Bound to Governance Evidence.

Recent additions to the requirement group further strengthen governance enforcement integrity. These include requirements ensuring that governance enforcement points possess the information necessary to determine admissibility (CR-119), enforce governance decisions with sufficient contextual integrity (CR-120), and prevent execution paths from bypassing governance-mediated enforcement (CR-121).

Collectively, these requirements reinforce the principle that governance evaluation must remain inseparable from execution authorization. Governance decisions are not merely advisory; they must remain binding at the point of operational commitment.

These requirements directly operationalize concepts discussed throughout this paper, including:

- admissibility evaluation
- execution-bound authorization
- operational legitimacy
- commit-time governance

Among all requirement groups, RG-2 most directly addresses the runtime governance gap identified in Section 1. It extends established security principles such as complete mediation and least privilege into autonomous operational environments by ensuring that governance decisions occur immediately before operational commitment rather than relying exclusively on prior authorization events (Saltzer and Schroeder, 1975; NIST, 2020a,b, 2018b).

16.5 RG-3 Governance Object Integrity

Governance decisions depend upon governance artifacts (AGCP Project, 2026).

Examples include:

- approvals
- authorizations
- evidence
- governance records
- state assertions

Governance Object Integrity requirements establish controls intended to ensure that governance artifacts remain attributable, protected, verifiable, and resistant to unauthorized modification (NIST, 2020a, 2022, 2018a; Rushby, 2008).

These requirements support:

- accountability
- traceability
- governance replayability
- assurance

Without governance object integrity, governance decisions may become difficult to trust, reproduce, or independently validate. This requirement group therefore serves as a foundational element supporting both runtime assurance and governance lifecycle integrity (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

16.6 RG-4 Canonical-State Governance

Canonical-State Governance establishes requirements for determining, maintaining, and validating authoritative operational state (AGCP Project, 2026).

As discussed in Sections 3 through 5, many governance failures arise when actions are evaluated against stale, inconsistent, incomplete, or manipulated information rather than current operational reality (Wang et al., 2023; Xi et al., 2023; Seshia et al., 2016, 2018; Leveson, 2011). Autonomous systems may derive decisions from numerous data sources, workflow states, memory systems, telemetry feeds, and contextual artifacts. Without an authoritative basis for evaluation, governance decisions may become inconsistent or operationally invalid.

Canonical-State Governance requirements establish mechanisms that ensure governance evaluation occurs against authoritative operational conditions rather than arbitrary or potentially stale representations of state.

Representative requirements include:

- CR-040 Canonical State Required.
- CR-041 Canonical State Validation.
- CR-042 Governance Decisions Bound to Canonical State.
- CR-043 Canonical State Provenance.
- CR-044 Canonical State Consistency.
- CR-045 Canonical State Refresh Requirements.
- CR-046 Canonical State Governance Record Association.

Collectively, these requirements help ensure that governance decisions remain connected to actual operational conditions rather than assumptions regarding those conditions. This aligns closely with established systems engineering, resiliency engineering, and safety engineering principles that emphasize authoritative state determination as a prerequisite for trustworthy decision-making (Saltzer and Schroeder, 1975; NIST, 2020a, 2018b, 2021; Leveson, 2011).

16.7 RG-5 Refusal and Escalation Governance

Refusal and Escalation Governance establishes requirements governing situations in which admissibility conditions are not satisfied (AGCP Project, 2026).

A recurring theme throughout AI safety literature is that systems must possess mechanisms that prevent unsafe or unauthorized behavior when appropriate conditions are absent (Russell, 2019; Amodei et al., 2016; Wang et al., 2023; Xi et al., 2023). Runtime governance extends this principle through structural refusal mechanisms that prevent operational commitment rather than merely generating warnings or recommendations.

Representative requirements include:

- CR-047 Structural Refusal Required.
- CR-048 Refusal Traceability.
- CR-049 Refusal Evidence Preservation.
- CR-050 Escalation Path Definition.
- CR-051 Escalation Accountability.
- CR-052 Escalation Auditability.

These requirements support Secure-by-Design objectives by ensuring that systems possess deterministic mechanisms for preventing execution when governance conditions are not satisfied (CISA, 2023; Saltzer and Schroeder, 1975; NIST, 2020a, 2018b, 2021).

An important distinction exists between advisory warnings and structural refusal.

A warning informs.

A structural refusal prevents execution.

Runtime governance relies upon the latter when operational legitimacy cannot be established.

16.8 RG-6 Governance Evidence and Receipts

Governance Evidence and Receipts establishes requirements governing the collection, preservation, validation, and retention of governance evidence (AGCP Project, 2026).

As autonomous systems increasingly participate in operational workflows, organizations require mechanisms capable of demonstrating why governance decisions were reached and what information supported those decisions (Wang et al., 2023; Xi et al., 2023; Rushby, 2008; Seshia et al., 2016).

Representative requirements include:

- CR-056 Evidence Continuity.
- CR-057 Evidence Provenance.
- CR-058 Evidence Validation.
- CR-059 Governance Receipt Generation.
- CR-060 Governance Receipt Integrity.
- CR-061 Governance Receipt Retention.

Governance receipts serve a role analogous to audit records, but they preserve governance-specific information including:

- evaluated evidence
- authorization outcomes

- admissibility determinations
- governance context
- canonical-state references

These requirements strengthen accountability, auditability, and assurance by enabling independent verification of governance decisions (NIST, 2022, 2018a, 2020a; Rushby, 2008).

16.9 RG-7 Multi-Agent Runtime Governance

Multi-Agent Runtime Governance establishes requirements governing interactions among autonomous systems, orchestration platforms, workflow agents, and delegated operational actors (AGCP Project, 2026).

As agentic systems increasingly coordinate activities across organizational, technical, and operational boundaries, governance concerns extend beyond individual actions to encompass chains of delegated authority and multi-agent interactions (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OWASP, 2025).

Representative requirements include:

- CR-065 Delegation Governance.
- CR-066 Authority Lineage Preservation.
- CR-067 Accountability Continuity.
- CR-068 Governance Context Propagation.
- CR-069 Multi-Agent Auditability.
- CR-070 Cross-Agent Governance Validation.

These requirements seek to preserve governance continuity even as responsibility traverses multiple systems, workflows, organizations, or operational domains (Wang et al., 2023; Xi et al., 2023; Rushby, 2008; Seshia et al., 2018).

Without such controls, accountability fragmentation may occur, making it difficult to determine who authorized an action, which system executed it, and under what governance conditions execution occurred.

16.10 RG-8 Governance Interoperability

Governance Interoperability establishes requirements supporting governance interactions across heterogeneous systems and governance domains (AGCP Project, 2026).

Increasingly, autonomous systems operate within environments that span:

- multiple organizations

- multiple technology stacks
- multiple governance frameworks
- multiple regulatory environments

Representative requirements include:

- CR-072 Governance Exchange Compatibility.
- CR-073 Governance Context Portability.
- CR-074 Governance Semantic Consistency.

Additional interoperability requirements address authority containment across governance boundaries. Recursive delegation chains must not expand operational authority beyond explicitly governed limits (CR-115). This requirement helps preserve governance intent as authority traverses heterogeneous systems, organizations, and governance environments.

These requirements support interoperability among governance systems while preserving accountability and governance continuity.

The importance of interoperability is well established within enterprise architecture, systems engineering, and federated governance environments. Runtime governance extends those concerns to execution-time governance decisions and associated governance artifacts (Open Group, 2022; ISO, 2023; NIST, 2018b).

16.11 RG-9 Runtime Governance Metrology

Runtime Governance Metrology establishes requirements for measuring, evaluating, validating, and continuously improving runtime governance effectiveness (AGCP Project, 2026).

Historically, governance frameworks have emphasized policy development, control implementation, auditing, and compliance assessment. Runtime governance introduces an additional requirement: organizations must be capable of measuring whether governance controls are functioning as intended during operational execution.

Representative requirements include:

- CR-075 Runtime Governance Metrics.
- CR-076 Governance Effectiveness Measurement.
- CR-077 Governance Coverage Assessment.
- CR-078 Governance Decision Quality Metrics.
- CR-079 Refusal Effectiveness Metrics.
- CR-080 Escalation Performance Metrics.
- CR-081 Governance Continuity Metrics.

- CR-082 Governance Evidence Quality Metrics.
- CR-083 Governance Replayability Metrics.
- CR-084 Runtime Assurance Metrics.
- CR-085 Governance Conformance Assessment.
- CR-086 Governance Testing Requirements.
- CR-087 Governance Validation Procedures.
- CR-088 Governance Reporting Requirements.
- CR-089 Governance Improvement Processes.

The requirement group also includes controls governing autonomous coordination behavior. Governance measurement must extend beyond individual actions to encompass coordinated autonomous activity and interaction patterns. CR-118 establishes bounds on autonomous coordination to help ensure that collective system behavior remains subject to governed operational limits.

These requirements align with broader assurance and measurement practices found throughout security engineering, systems assurance, risk management, and organizational governance programs (NIST, 2022, 2018a, 2020a, 2023b; ISO, 2023).

Runtime governance cannot be considered trustworthy if its effectiveness cannot be measured, evaluated, and independently assessed.

16.12 RG-10 Continuation Integrity Governance

Continuation Integrity Governance establishes requirements governing the preservation of governance validity across multi-step workflows and long-running operational processes (AGCP Project, 2026).

Many governance models implicitly assume that authorization remains valid once granted. However, autonomous systems frequently operate across extended periods during which operational conditions, authority relationships, evidence quality, or policy constraints may change.

Representative requirements include:

- CR-090 Continuation Integrity Validation.
- CR-091 Governance Revalidation Requirements.
- CR-092 Workflow Continuity Assurance.
- CR-093 Context Preservation Requirements.
- CR-094 Authority Continuity Validation.
- CR-095 Evidence Continuity Validation.
- CR-096 Continuation Auditability.

These requirements recognize that governance legitimacy may degrade over time. A decision that was admissible at one point may become inadmissible as operational conditions evolve.

Continuation Integrity Governance therefore extends runtime governance beyond isolated actions and into complex operational workflows (NIST, 2021; Leveson, 2011; Rushby, 2008).

16.13 RG-11 Composite Governance

Composite Governance establishes requirements for coordinating governance decisions across multiple governance domains simultaneously (AGCP Project, 2026).

Real-world operational environments rarely operate under a single governance framework. Decisions may be influenced by:

- security policies
- safety constraints
- regulatory obligations
- mission objectives
- organizational directives
- contractual requirements

Representative requirements include:

- CR-097 Composite Governance Evaluation.
- CR-098 Cross-Domain Conflict Resolution.
- CR-099 Composite Admissibility Determination.
- CR-100 Composite Governance Traceability.
- CR-101 Composite Governance Auditability.

Additional requirements strengthen governance isolation across composite control domains. CR-116 establishes that authority recognized within one governance domain does not automatically transfer to another governance domain without explicit governance mediation. This helps prevent unintended authority propagation across security, safety, regulatory, or organizational boundaries.

These requirements help ensure that governance decisions remain coherent when multiple governance authorities simultaneously influence operational outcomes.

This aligns with long-standing enterprise governance and systems engineering concerns regarding policy coordination and cross-domain consistency (Open Group, 2022; NIST, 2018b; ISO, 2023).

16.14 RG-12 Governance Compilation and Constitutional Governance

Governance Compilation and Constitutional Governance establishes requirements governing the translation of governance intent into executable governance controls (AGCP Project, 2026).

A recurring challenge in governance systems involves ensuring that high-level governance objectives remain faithfully represented within operational mechanisms. Policies may be ambiguous, incomplete, conflicting, or difficult to operationalize directly.

Representative requirements include:

- CR-102 Governance Compilation.
- CR-103 Governance Constraint Preservation.
- CR-104 Constitutional Governance Enforcement.
- CR-105 Governance Compilation Traceability.
- CR-106 Governance Compilation Auditability.

Constitutional governance requirements also protect governance mechanisms themselves. CR-117 establishes that governed systems cannot modify their own admissibility rules without external governance authorization. This requirement helps preserve constitutional constraints and prevents operational actors from unilaterally altering the governance mechanisms that regulate their behavior.

These requirements support a governance architecture in which operational controls remain connected to higher-level governance intent.

Governance compilation provides a mechanism through which policy objectives, regulatory requirements, organizational constraints, and constitutional governance principles may be translated into execution-time governance behavior while preserving traceability and accountability.

This concept aligns with broader efforts within AI governance, systems engineering, and assurance disciplines to improve the consistency between governance objectives and operational implementation (ISO, 2023; NIST, 2023b; European Union, 2024; NIST, 2018b).

16.15 Conceptual Mapping to Runtime Governance Principles

Although the AGCP requirements were developed as an implementation-oriented specification, the requirement groups map closely to the runtime governance concepts introduced throughout this paper.

The mapping shown in Table 6 demonstrates that runtime governance concepts generally span multiple requirement groups rather than corresponding to a single governance function. This distribution reflects the interconnected nature of execution-time governance, where authorization, admissibility evaluation, governance continuity, evidence management, authority containment, enforcement integrity, and runtime assurance depend upon coordinated controls operating across multiple governance domains. The table further illustrates that concepts such as governance compilation, constitutional governance, and governance self-protection can be translated into concrete

Table 6: Mapping of Runtime Governance Concepts to AGCP Requirement Groups

Runtime Governance Concept	Primary Requirement Groups
Admissibility Evaluation	RG-2, RG-4, RG-5
Authority Containment	RG-7, RG-8, RG-11
Canonical-State Governance	RG-4
Constitutional Governance	RG-12
Cross-Domain Governance	RG-8, RG-11
Deterministic Governance	RG-2, RG-4, RG-12
Enforcement Integrity	RG-2
Evidence Continuity	RG-6, RG-10
Execution-Bound Authorization	RG-2, RG-10
Governance Compilation	RG-12
Governance Continuity	RG-1, RG-7, RG-10
Governance Replayability	RG-1, RG-3, RG-6
Governance Self-Protection	RG-12
Multi-Agent Governance	RG-7, RG-8, RG-9
Runtime Assurance	RG-3, RG-4, RG-6, RG-9
Structural Refusal	RG-5

implementation requirements rather than remaining purely conceptual constructs. Although the mapping is illustrative rather than normative and individual implementations may allocate responsibilities differently, the relationships shown demonstrate how runtime governance concepts can be systematically realized through implementation-oriented requirements capable of supporting conformance assessment, operational deployment, and governance assurance.

16.16 Requirements as an Example Runtime Governance Specification

The AGCP requirement catalog represents one example implementation-oriented specification for runtime governance (AGCP Project, 2026).

The broader significance of the requirements is not the specific requirement language itself. Rather, the requirement structure illustrates how runtime governance concepts may be translated into operational controls, measurable behaviors, assurance objectives, and conformance criteria.

Organizations may choose different implementation approaches, governance architectures, or technical mechanisms. Regardless of implementation details, runtime governance architectures will likely require mechanisms supporting:

- execution-time authorization
- admissibility evaluation
- canonical-state validation
- governance continuity
- evidence continuity

- structural refusal
- replayability
- runtime assurance

These capabilities increasingly appear necessary as autonomous and agentic systems assume greater operational responsibility across critical infrastructure, enterprise systems, and public-sector environments (NIST, 2023b; ISO, 2023; CISA, 2023; NIST, 2018b, 2021; Wang et al., 2023).

The next section examines how runtime governance concepts align with existing standards, frameworks, and guidance documents, demonstrating that runtime governance should be viewed not as a replacement for current governance approaches but as an execution-time extension of established governance principles.

17 Implementation Considerations

17.1 Introduction

The preceding sections established the conceptual foundations of runtime governance, explored its relationship to Secure-by-Design principles, mapped runtime governance concepts to existing standards and frameworks, presented a reference architecture, and examined representative critical infrastructure use cases. The discussion thus far has focused primarily on governance objectives, architectural principles, and normative requirements.

Implementation introduces additional considerations.

Organizations operate in diverse environments with differing governance requirements, risk tolerances, operational constraints, regulatory obligations, and technical architectures. Consequently, no single runtime governance implementation model is likely to satisfy every operational scenario (NIST, 2023b; ISO, 2023; NIST, 2018b; Open Group, 2022).

This section examines practical considerations associated with implementing runtime governance capabilities in real-world environments. The objective is not to prescribe a specific architecture but rather to identify factors that influence design decisions, operational tradeoffs, assurance strategies, and deployment models (CISA, 2023; NIST, 2024a, 2020a; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

Runtime governance architectures may be implemented using decision mechanisms analogous to Policy Decision Points (PDPs). Enforcement of resulting governance decisions may occur through integrated or external Policy Enforcement Point (PEP)-like mechanisms. However, runtime governance extends traditional authorization-centric decision models by incorporating admissibility evaluation, evidence validation, governance continuity, and other execution-time governance considerations.

17.2 Centralized Governance Architectures

One implementation approach places governance decision-making within a centralized governance service.

Under this model, operational systems submit proposed actions to a common governance layer responsible for:

- policy evaluation
- authorization evaluation
- admissibility determination
- evidence validation
- governance recording

Figure 6 illustrates the concept.

Advantages include:

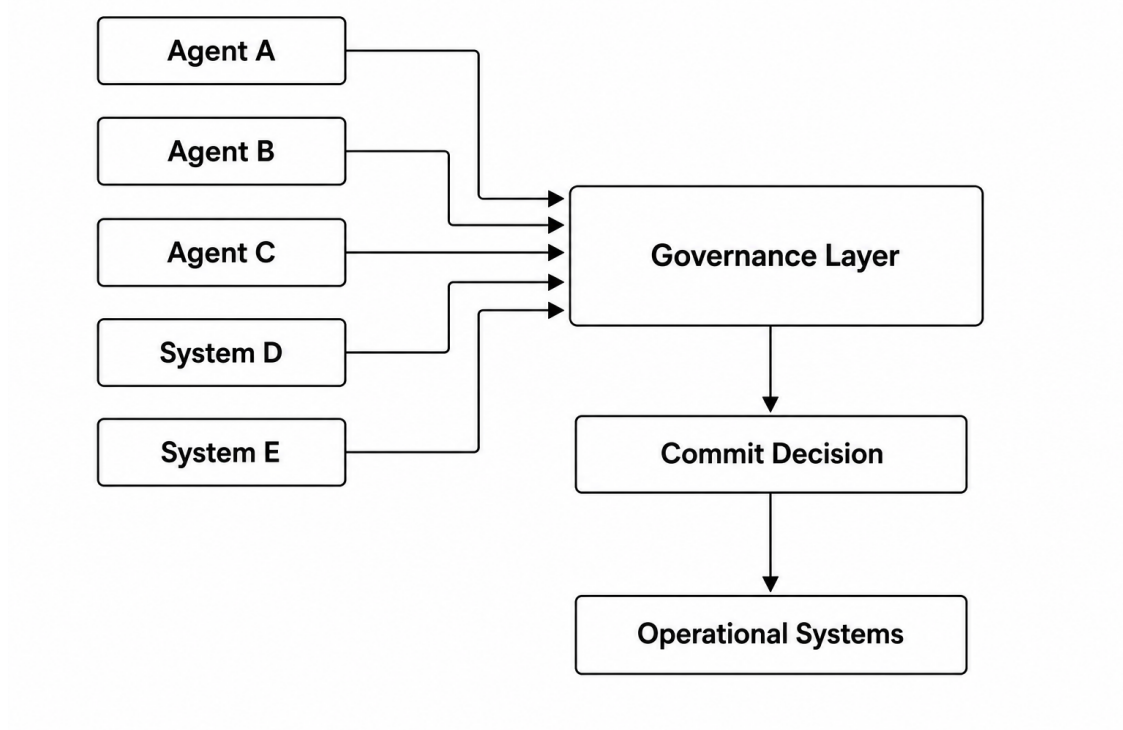


Figure 6: Illustrative centralized governance model.

- consistent governance decisions
- centralized auditability
- simplified policy management
- unified governance records
- easier assurance assessment

Centralization can improve accountability, traceability, and governance consistency by ensuring that governance decisions are evaluated against a common set of rules and authoritative state information (NIST, 2020a,b; OASIS, 2013; Saltzer and Schroeder, 1975; Anderson, 2020; NIST, 2018b).

Potential challenges include:

- scalability constraints
- latency impacts
- availability dependencies
- governance bottlenecks
- single-point-of-failure concerns

These concerns mirror longstanding tradeoffs observed in centralized security architectures, policy-decision points, and enterprise governance systems (NIST, 2020a,b; CSA, 2024; Open Group, 2022; OASIS, 2013).

Centralized governance is often attractive in regulated environments where consistency, auditability, and compliance are primary concerns.

17.3 Distributed Governance Architectures

Distributed architectures embed governance capabilities closer to operational systems.

Rather than relying upon a single governance service, individual systems may perform governance evaluation locally while maintaining consistency with broader governance objectives.

Advantages include:

- improved scalability
- reduced latency
- localized decision-making
- improved resilience
- reduced central dependencies

Distributed governance can improve responsiveness in environments where operational decisions occur at machine speed or where geographically dispersed systems must operate independently (Wang et al., 2023; Xi et al., 2023; Park et al., 2023; NIST, 2025; Morris et al., 2024; OpenAI, 2025).

Challenges include:

- policy synchronization
- governance consistency
- evidence sharing
- assurance complexity
- interoperability concerns

Maintaining consistent governance outcomes across distributed environments may require additional mechanisms for policy propagation, state synchronization, and governance assurance (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; CSA, 2024; Open Group, 2022; OASIS, 2013).

Distributed governance becomes increasingly attractive as autonomous systems operate across geographically dispersed or highly decentralized environments.

17.4 Federated Governance Architectures

Federated governance architectures represent a hybrid approach.

In a federated model, individual governance domains retain local governance authority while participating in a broader governance ecosystem.

Examples include:

- interagency environments
- coalition operations
- supply-chain ecosystems
- multi-organization workflows
- regulated industry partnerships

Federated governance requires mechanisms for:

- governance interoperability
- trust establishment
- authority recognition
- evidence exchange
- policy translation
- accountability preservation

Federation has long been recognized as a necessary architectural approach for environments where governance authority cannot be fully centralized (NIST, 2020a, 2018b; CSA, 2024; Open Group, 2022; OASIS, 2013).

As autonomous systems increasingly operate across organizational boundaries, federated governance may become an important implementation model for preserving governance continuity while respecting local authority structures (Wang et al., 2023; Xi et al., 2023; Seshia et al., 2016; Rushby, 2008; Open Group, 2022).

17.5 Multi-Tenant Environments

Many AI systems operate within multi-tenant platforms.

Examples include:

- cloud platforms
- SaaS environments

- managed AI services
- enterprise AI platforms
- shared orchestration systems

Multi-tenancy introduces additional governance challenges because:

- policies may differ among tenants
- authority structures may vary
- evidence requirements may differ
- compliance obligations may conflict
- governance records may require segregation

Runtime governance architectures operating in multi-tenant environments should preserve:

- tenant isolation
- governance isolation
- evidence isolation
- accountability separation
- policy independence

while maintaining efficient operation.

These concerns align closely with established cloud security, shared-responsibility, and isolation principles (NIST, 2020a,b; CSA, 2024; Open Group, 2022).

17.6 Agent-to-Agent Governance

One of the most significant implementation challenges involves governance interactions among autonomous agents.

Traditional governance systems frequently assume that humans are the primary decision-makers and accountability anchors.

Agentic environments increasingly challenge this assumption (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OWASP, 2025).

Examples include:

- planning agents delegating tasks
- orchestration agents coordinating workflows

- execution agents invoking tools
- monitoring agents triggering actions
- supervisory agents evaluating outcomes

Agent-to-agent governance introduces several requirements:

- authority lineage preservation
- governance continuity
- delegation validation
- accountability attribution
- governance replayability
- escalation mechanisms

Without such controls, governance intent may degrade rapidly across autonomous workflows, leading to ambiguity regarding authority, accountability, and operational legitimacy (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

This challenge is likely to become increasingly important as multi-agent systems mature and become more widely deployed.

17.7 Human-in-the-Loop and Human-on-the-Loop Models

Runtime governance does not eliminate the role of human oversight.

Instead, governance architectures must determine when human involvement is required.

Several common models exist.

17.7.1 Human-in-the-Loop

Under a Human-in-the-Loop (HITL) model, certain actions require explicit human approval before execution.

This model is commonly used for:

- high-consequence actions
- financial commitments
- safety-critical operations
- regulatory approvals
- exceptional conditions

Advantages include:

- strong accountability
- explicit authorization
- improved oversight
- enhanced auditability

Potential limitations include:

- scalability constraints
- operational latency
- approval bottlenecks
- human workload

Human-in-the-Loop approaches remain particularly important in environments where regulatory, legal, or safety requirements mandate human accountability (NIST, 2023b, 2024b; Executive Office of the President, 2023; NSCAI, 2021; Defense Innovation Board, 2019; European Union, 2024).

17.7.2 Human-on-the-Loop

Under a Human-on-the-Loop (HOTL) model, systems may execute autonomously while humans maintain supervisory authority.

Advantages include:

- operational efficiency
- scalability
- reduced latency
- increased autonomy

Challenges include:

- situational awareness
- intervention timing
- accountability boundaries
- escalation effectiveness

Runtime governance architectures should support both HITL and HOTL approaches depending upon operational requirements, risk tolerance, and mission objectives (NIST, 2023b, 2024b; Russell, 2019; Amodei et al., 2016; Wang et al., 2023; Xi et al., 2023).

17.8 Governance Performance Considerations

Governance effectiveness depends not only upon governance correctness but also upon governance performance.

Execution-time governance introduces computational overhead because proposed actions must be evaluated before commitment occurs. Depending upon implementation architecture, governance evaluation may include:

- policy evaluation
- authorization validation
- evidence verification
- canonical-state determination
- admissibility assessment
- governance record generation

In high-throughput environments, governance latency may become operationally significant.

Examples include:

- industrial control systems
- financial trading systems
- telecommunications infrastructure
- cyber defense platforms
- autonomous transportation systems

Organizations implementing runtime governance should therefore consider:

- governance latency
- throughput capacity
- scalability characteristics
- governance caching strategies
- evidence retrieval performance
- resilience under peak load

Performance considerations must be balanced against governance objectives. Excessively burdensome governance controls may reduce operational effectiveness, while insufficient governance evaluation may weaken assurance and accountability. Effective runtime governance therefore requires balancing operational responsiveness with governance rigor (Russell, 2019; Wang et al., 2023; Xi et al., 2023; Rushby, 2008; Seshia et al., 2016, 2018).

17.9 Governance Availability and Resilience

Runtime governance systems increasingly become operational dependencies.

If governance mechanisms are unavailable, organizations must determine how operational systems should respond.

Possible approaches include:

- fail-safe operation
- fail-secure operation
- degraded governance modes
- emergency authorization procedures
- continuity-of-operations mechanisms

Runtime governance architectures should therefore incorporate resilience considerations from the outset (NIST, 2021; CISA, 2023).

Important design considerations include:

- governance service redundancy
- evidence preservation
- recovery procedures
- governance state synchronization
- continuity planning
- disaster recovery

These concerns parallel broader cybersecurity and systems-engineering principles related to availability, recoverability, and operational continuity (CISA, 2023; NIST, 2024a, 2020a; Saltzer and Schroeder, 1975; NIST, 2018b, 2021).

Because governance mechanisms may ultimately influence operational authorization decisions, governance infrastructure itself may warrant treatment as critical infrastructure within certain deployment contexts.

17.10 Data Protection and Privacy Considerations

Governance systems frequently process sensitive information.

Examples include:

- identity information

- authorization records
- operational telemetry
- governance evidence
- audit artifacts
- decision histories

Consequently, governance architectures should incorporate data protection controls appropriate to their operational environment.

Key considerations include:

- data minimization
- access controls
- evidence protection
- retention management
- cryptographic protections
- privacy-preserving auditability

The sensitivity of governance records may vary significantly depending upon mission context, regulatory requirements, and operational environment. In critical infrastructure, defense, healthcare, and financial environments, governance evidence may itself become a high-value target, making confidentiality obligations particularly significant (NIST, 2023b; ISO, 2023; NIST, 2020a,b; European Union, 2024).

Organizations should therefore treat governance data as a protected operational asset rather than merely as administrative metadata.

17.11 Governance Assurance and Independent Validation

Runtime governance architectures should themselves be subject to assurance activities.

The objective of runtime governance is to increase confidence that operational actions remain legitimate under current conditions. That objective cannot be achieved if governance mechanisms themselves remain unverified.

Examples of assurance activities include:

- conformance assessment
- governance testing
- independent review

- adversarial evaluation
- governance replay validation
- operational exercises

Assurance activities should evaluate not only whether governance controls exist, but whether they operate correctly under realistic operational conditions (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Rushby, 1981; Leveson, 2011; ISO, 2023).

Representative assurance questions include:

- Are authorization decisions reproducible?
- Are governance records complete?
- Can governance decisions be replayed?
- Is evidence continuity preserved?
- Are refusal mechanisms functioning correctly?
- Can governance failures be detected and investigated?

Independent validation becomes increasingly important as governance architectures assume greater responsibility for mediating operational decisions.

17.12 Adoption Pathways

Organizations are unlikely to adopt runtime governance capabilities all at once.

As with many governance and security technologies, adoption will likely occur incrementally.

Representative adoption pathways include:

1. Governance visibility and monitoring.
2. Governance evidence collection.
3. Governance replayability.
4. Execution-time authorization.
5. Admissibility evaluation.
6. Structural refusal mechanisms.
7. Multi-agent governance.
8. Full runtime governance integration.

This progression mirrors historical adoption patterns observed in cybersecurity, risk management, and systems assurance disciplines. Organizations often begin by improving visibility and auditability before introducing stronger enforcement mechanisms (NIST, 2023b; ISO, 2023; NIST, 2020a, 2018b; Seshia et al., 2016; Rushby, 2008).

Several factors may influence adoption timelines:

- regulatory obligations
- operational risk
- mission criticality
- system autonomy levels
- organizational maturity
- governance readiness

Highly regulated sectors may adopt runtime governance capabilities more rapidly due to accountability, auditability, and assurance requirements. Less regulated sectors may initially focus on governance visibility and operational monitoring before introducing stronger execution-time controls.

Regardless of adoption sequence, organizations should recognize that runtime governance is most effective when treated as an architectural capability rather than a collection of isolated controls.

17.13 Implementation as an Engineering Discipline

A recurring theme throughout this paper is that runtime governance should not be viewed solely as a policy concern.

Policies define governance intent.

Runtime governance operationalizes that intent.

Consequently, implementation requires engineering disciplines capable of translating governance objectives into operational mechanisms.

Relevant disciplines include:

- systems engineering
- software engineering
- security engineering
- resilience engineering
- safety engineering
- assurance engineering

- enterprise architecture

Implementation therefore involves more than governance documentation or compliance activities. It requires:

- architecture design
- control implementation
- evidence management
- assurance mechanisms
- operational monitoring
- lifecycle management

This perspective aligns closely with Secure-by-Design principles, which emphasize integrating security, resilience, and assurance into system architecture rather than treating them as external review activities (CISA, 2023; NIST, 2018b, 2021).

The same principle applies to runtime governance.

Governance mechanisms become most effective when incorporated directly into operational architectures rather than added after deployment.

As autonomous systems continue to evolve, runtime governance may emerge as a distinct engineering discipline situated at the intersection of AI governance, systems assurance, security engineering, and operational decision systems (Russell, 2019; Wang et al., 2023; Xi et al., 2023; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

17.14 Summary

Implementation choices will vary across organizations, sectors, and operational environments.

Some organizations may prefer centralized governance architectures that emphasize consistency and auditability. Others may require distributed or federated approaches capable of supporting large-scale autonomous operations across organizational boundaries.

Regardless of implementation model, several recurring themes emerge:

- Governance decisions should remain connected to execution.
- Governance continuity should be preserved across workflows.
- Evidence supporting governance decisions should remain available.
- Governance architectures should support assurance and replayability.
- Runtime governance should be treated as an operational capability rather than solely a compliance activity.

These considerations reinforce the broader argument developed throughout this paper: as AI systems become increasingly autonomous and operationally influential, governance mechanisms must increasingly operate at the point where actions become consequential. Runtime governance provides one architectural approach for addressing that challenge while remaining compatible with existing governance frameworks, security principles, and systems-engineering practices.

18 Future Research Directions

18.1 Introduction

The emergence of increasingly autonomous and agentic AI systems suggests that runtime governance may become an increasingly important area of research and engineering. While many of the concepts discussed throughout this paper are grounded in established principles from cybersecurity, systems engineering, assurance, and governance, significant questions remain regarding how execution-time governance should be designed, evaluated, standardized, and validated in operational environments (NIST, 2023b; ISO, 2023; NIST, 2018b, 2021; Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

This section identifies several research areas that may influence the future evolution of runtime governance architectures, runtime assurance methods, and Secure-by-Design AI systems.

The objective is not to predict a single future governance model. Rather, it is to highlight areas where further research may improve the ability of organizations to govern autonomous actions safely, transparently, and accountably.

18.2 Runtime Governance for Autonomous Agents

Many current governance approaches assume that humans remain directly involved in consequential decisions.

Increasingly autonomous systems challenge this assumption.

Future research may examine:

- governance models for autonomous decision-making
- governance boundaries for delegated authority
- autonomous authorization models
- governance continuity across autonomous workflows
- governance preservation during long-running operations

One particularly important question is:

How should governance authority be represented and constrained when autonomous systems act independently for extended periods of time?

As autonomous agents become more capable, governance mechanisms may need to evolve beyond traditional approval and auditing models toward continuously evaluated operational governance (Russell, 2019; Amodei et al., 2016; Leike et al., 2018; Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OpenAI, 2023; Anthropic, 2022).

18.3 Multi-Agent Governance Systems

Research into multi-agent systems continues to expand rapidly.

Future environments may involve:

- coordinating agents
- supervisory agents
- planning agents
- execution agents
- validation agents
- governance agents

Such environments introduce questions regarding:

- authority propagation
- accountability preservation
- governance context sharing
- inter-agent trust
- governance conflict resolution
- cross-agent authorization

Future research may explore governance architectures capable of preserving operational legitimacy across complex networks of interacting autonomous actors (Park et al., 2023; Wang et al., 2023; Xi et al., 2023; OWASP, 2025; OpenAI, 2025).

18.4 Cross-Domain Delegation and Authority Management

Delegation is likely to become increasingly important as AI systems operate across organizational, technical, and regulatory boundaries.

Examples include:

- enterprise-to-agent delegation
- agency-to-agency workflows
- coalition operations
- supply-chain orchestration
- cross-domain service coordination

Future research may examine:

- machine-readable authority models
- delegation constraints
- delegation revocation
- delegation provenance
- delegated accountability
- authority inheritance controls

These topics may become particularly important in critical infrastructure, defense, and government environments where authority relationships are often highly structured and operationally significant (DHS, 2024; NIST, 2020a,b; NSCAI, 2021; Defense Innovation Board, 2019).

18.5 Governance Interoperability Standards

Many existing governance frameworks define governance objectives but do not specify interoperable execution-time governance mechanisms.

As runtime governance architectures mature, organizations may benefit from standards supporting:

- governance information exchange
- governance evidence portability
- authorization interoperability
- governance record interoperability
- governance state exchange
- cross-domain governance validation

Interoperability standards could improve the ability of autonomous systems to operate across organizational boundaries while preserving governance continuity and accountability (NIST, 2023b; ISO, 2023; Open Group, 2022; OASIS, 2013; CSA, 2024).

Future research may explore whether governance interoperability should be standardized in a manner similar to identity federation, security protocols, and trust frameworks.

18.6 Machine-Verifiable Governance

One of the most promising areas of future research involves machine-verifiable governance.

Traditional governance frequently depends upon:

- documents
- policies
- procedures
- human interpretation
- administrative oversight

Autonomous systems increasingly require governance information that can be interpreted operationally by machines.

Future research may examine:

- machine-readable policies
- executable governance models
- formal governance representations
- governance verification methods
- governance consistency validation
- automated governance analysis

Machine-verifiable governance may enable more scalable governance approaches while reducing ambiguity associated with policy interpretation (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Rushby, 1981; Leveson, 2011; ISO, 2023).

18.7 Formal Verification of Governance Behavior

Formal methods have long been used to improve confidence in software systems, security protocols, and safety-critical systems (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Rushby, 1981; Leveson, 2011).

Future runtime governance research may explore:

- formal verification of governance workflows
- authorization correctness proofs
- admissibility correctness verification
- governance state consistency analysis
- governance invariant validation
- refusal correctness analysis

Formal approaches may be particularly valuable in high-consequence environments where governance failures could produce significant operational impacts.

Research in this area could help strengthen confidence that governance systems behave consistently and predictably under specified conditions.

18.8 Deterministic Governance and Reproducibility

Determinism was introduced earlier as an important characteristic supporting auditability, replayability, and assurance.

Future research may investigate:

- deterministic governance architectures
- governance replay systems
- reproducible authorization models
- governance simulation environments
- governance testing methodologies
- deterministic governance benchmarks

These efforts may contribute to stronger governance assurance by enabling organizations to reproduce governance outcomes independently and validate governance behavior under controlled conditions (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018).

18.9 Runtime Assurance Methodologies

Runtime assurance remains an emerging discipline.

Future research may explore:

- runtime assurance metrics
- governance confidence models
- operational legitimacy scoring
- governance assurance cases
- assurance evidence frameworks
- continuous assurance architectures

Such work could provide more rigorous methods for evaluating whether autonomous systems remain within acceptable governance boundaries during operation (Seshia et al., 2016; Rushby, 2008; Seshia et al., 2018; Rushby, 1981; Leveson, 2011; ISO, 2023).

18.10 Governance Metrology

Measurement plays a central role in mature engineering disciplines.

Future research may examine governance metrology, including:

- governance correctness metrics
- governance continuity metrics
- authorization quality metrics
- evidence continuity metrics
- governance integrity metrics
- replayability metrics

A more mature governance metrology discipline could improve the ability of organizations to compare governance architectures, evaluate effectiveness, and perform independent assessments.

Such efforts may ultimately contribute to the development of standardized runtime governance evaluation methodologies (NIST, 2023b; ISO, 2023; NIST, 2022).

18.11 Runtime Governance for Physical Systems

Much of the current discussion surrounding AI governance focuses on information systems.

However, autonomous systems increasingly interact with physical environments.

Examples include:

- industrial control systems
- transportation systems
- energy infrastructure
- robotics
- healthcare devices
- defense systems

Physical environments introduce governance considerations beyond traditional information security concerns.

Future research may examine:

- governance for cyber-physical systems
- safety-governance integration
- real-time governance constraints
- physical consequence modeling
- operational containment mechanisms

- safety-critical governance architectures

As autonomous systems assume increasing responsibility within critical infrastructure environments, runtime governance may become an important mechanism for maintaining operational safety, resilience, and accountability (CISA, 2023, 2024; Leveson, 2011; DHS, 2024).

18.12 Constitutional and Policy-Constrained AI Systems

Recent research has explored methods for constraining AI behavior through constitutional principles, policy frameworks, and structured governance guidance.

Future research may investigate:

- constitutional governance models
- machine-readable governance constitutions
- policy-constrained agent architectures
- governance-aware planning systems
- governance-directed reasoning mechanisms
- constitutional runtime enforcement

These approaches may help bridge the gap between high-level governance objectives and operational execution behavior (Anthropic, 2022; OpenAI, 2025).

An important research question is whether governance constraints can be represented in ways that remain both machine-interpretable and operationally enforceable across diverse autonomous environments.

18.13 Governance as a First-Class Architectural Discipline

Historically, governance has often been treated as a management, compliance, or oversight activity.

Future research may examine whether governance should instead be treated as a first-class architectural discipline.

Such a discipline could encompass:

- governance engineering
- governance architecture
- governance assurance
- governance interoperability
- governance measurement

- governance lifecycle management

As the field matures, runtime governance may evolve into a distinct architectural discipline with its own design patterns, assurance methodologies, interoperability standards, conformance models, and engineering practices.

This perspective aligns with broader Secure-by-Design efforts that seek to embed security, resilience, and assurance directly into operational systems rather than treating them as external review functions (CISA, 2023; NIST, 2020a, 2018b, 2021).

Research in this area could contribute to the development of governance engineering practices analogous to those that exist today for software engineering, systems engineering, and security engineering.

18.14 Toward a Science of Runtime Governance

Perhaps the most ambitious future research direction involves the development of a scientific foundation for runtime governance itself.

Many governance concepts remain primarily conceptual, organizational, or policy-oriented.

Future work may seek to establish:

- formal governance theories
- governance invariants
- governance measurement frameworks
- governance correctness criteria
- governance assurance methodologies
- governance performance benchmarks

Such efforts could help transform runtime governance from a collection of architectural practices into a more rigorous engineering and scientific discipline.

A mature science of runtime governance would likely integrate concepts from:

- systems engineering
- cybersecurity
- safety engineering
- assurance engineering
- organizational governance
- artificial intelligence

The resulting discipline could provide stronger theoretical foundations for designing, evaluating, validating, and governing increasingly autonomous systems.

As AI systems continue to evolve, the ability to govern actions at the point of execution may become an increasingly important component of trustworthy and Secure-by-Design AI (NIST, 2023b; ISO, 2023; Seshia et al., 2016; Rushby, 2008; Leveson, 2011).

18.15 Transition to Conclusion

The research directions outlined above are not intended to be exhaustive.

Rather, they illustrate the breadth of questions that remain open as organizations seek to operationalize runtime governance concepts in increasingly autonomous environments.

Many of these questions extend beyond traditional AI governance and intersect with broader concerns involving security engineering, systems assurance, resilience engineering, enterprise architecture, and cyber-physical systems.

Addressing these challenges will likely require collaboration among researchers, standards bodies, industry practitioners, government organizations, and assurance communities.

As runtime governance matures, future research may help transform execution-time governance from an emerging concept into a more established engineering discipline (NIST, 2023b; ISO, 2023; NIST, 2018b).

The final section summarizes the paper’s principal findings and discusses the broader implications of runtime governance for the future of Secure-by-Design AI systems.

19 Conclusion

Artificial intelligence governance has matured significantly over the past decade. Governments, standards bodies, industry organizations, and research communities have developed increasingly sophisticated approaches for managing AI-related risks through governance frameworks, security guidance, assurance methodologies, regulatory mechanisms, and operational best practices.

These efforts have established an essential foundation for trustworthy AI.

Frameworks such as the NIST AI Risk Management Framework, NIST Cybersecurity Framework, NIST SP 800-53, MITRE ATLAS, OWASP Agentic AI guidance, and CISA Secure-by-Design initiatives collectively provide critical guidance regarding accountability, transparency, security, resilience, risk management, and operational oversight.

The central argument of this paper is not that these frameworks are inadequate.

Rather, the argument is that the increasing autonomy of AI systems introduces a governance challenge that existing frameworks often leave implementation-specific:

How should organizations govern autonomous actions at runtime?

Historically, governance efforts have focused primarily on:

- model development
- model evaluation
- deployment approval
- organizational oversight
- policy definition
- auditing and monitoring

These activities remain necessary and should continue to serve as foundational components of trustworthy AI programs.

However, increasingly autonomous systems are no longer limited to producing recommendations, predictions, classifications, or analytical outputs.

Agentic systems increasingly:

- invoke tools
- coordinate workflows
- delegate tasks
- interact with external systems
- modify operational state

- initiate consequential actions

As a result, governance concerns increasingly emerge at the point where actions transition from proposed intent to operational reality.

This paper referred to that transition point as the commit boundary.

At the commit boundary, governance questions become operational rather than theoretical:

- Is this action currently authorized?
- Is the supporting evidence still valid?
- Has operational context changed?
- Does the action remain admissible?
- Are required constraints satisfied?
- Should execution proceed?

Traditional lifecycle-oriented governance mechanisms are not always designed to answer these questions at execution time.

Consequently, a runtime governance gap emerges.

This paper examined runtime governance as an architectural response to that gap.

Runtime governance was presented not as a competing governance framework, but as a category of execution-time controls intended to preserve governance intent throughout operational execution.

Key concepts examined throughout the paper included:

- admissibility evaluation
- execution-bound authorization
- structural refusal
- canonical-state governance
- evidence continuity
- governance continuity
- governance replayability
- governance lifecycle integrity
- governance integrity
- authority containment
- governance isolation

- enforcement integrity
- runtime assurance

These concepts were intentionally introduced using general architectural language before any specific implementation approach was discussed.

The analysis demonstrated that many runtime governance concepts align naturally with longstanding cybersecurity and assurance principles including:

- least privilege
- complete mediation
- accountability
- traceability
- resilience
- safe failure
- auditability
- authority containment
- governance isolation
- non-bypassable enforcement

In this respect, runtime governance may be viewed less as a new governance paradigm and more as an operational extension of established security and assurance practices into increasingly autonomous environments.

The paper further demonstrated that runtime governance concepts complement rather than replace existing frameworks.

For example:

- NIST AI RMF establishes governance objectives.
- NIST CSF establishes cybersecurity objectives.
- NIST SP 800-53 establishes security controls.
- CISA Secure-by-Design establishes engineering principles.
- MITRE ATLAS provides adversarial knowledge.
- OWASP Agentic AI identifies emerging operational risks.

Runtime governance architectures may provide execution-time mechanisms for operationalizing these objectives as autonomous systems become increasingly capable of acting independently.

The critical infrastructure scenarios examined throughout this paper illustrated how governance decisions may depend upon factors that can only be evaluated at execution time.

Examples included:

- changing operational conditions
- evolving authority relationships
- stale evidence
- incomplete governance lineage
- degraded telemetry
- conflicting constraints

In each scenario, governance effectiveness depended not solely upon prior authorization but upon the ability to evaluate operational legitimacy under current conditions.

This observation suggests a broader shift in how governance may need to be viewed in future autonomous environments.

Historically, governance has often been treated primarily as an organizational, compliance, or administrative function.

As autonomous systems assume greater operational responsibility, governance increasingly becomes an engineering problem.

Organizations must determine not only what governance policies exist, but also how those policies are evaluated, enforced, measured, protected, and preserved during execution.

In this sense, runtime governance may emerge as a distinct architectural discipline situated at the intersection of:

- cybersecurity
- systems engineering
- operational assurance
- AI governance
- safety engineering
- enterprise architecture

The requirement structures and implementation examples discussed throughout this paper demonstrate one possible path toward operationalizing these concepts through deterministic governance evaluation, governance compilation, execution-bound authorization, governance continuity, and enforcement integrity.

Runtime governance should therefore be viewed not as a single architecture, product, or specification, but as an emerging architectural category concerned with governing autonomous actions at execution time.

The AI Governance Control Plane (AGCP) was presented as one example runtime governance specification that translates execution-time governance concepts into formal requirements and conformance criteria. The publicly available specification, requirements catalog, and conformance materials provide one example of how runtime governance principles may be operationalized in practice.

The broader conclusion of this paper does not depend upon adoption of any particular implementation.

Rather, it depends upon recognition of a more fundamental reality:

The future challenge is no longer simply whether organizations can govern AI models.

Increasingly, the challenge is whether organizations can govern autonomous actions at runtime.

As AI systems continue to evolve from analytical tools into operational actors, execution-time governance may become an increasingly important component of Secure-by-Design architectures.

Future AI systems will likely require more than trustworthy models.

They will require trustworthy execution.

Future AI systems will likely require more than trustworthy models. They will require trustworthy execution.

Achieving that objective may depend upon governance mechanisms capable of evaluating operational legitimacy, preserving accountability, maintaining governance continuity, protecting governance integrity, enforcing governance intent, and ensuring that governance decisions remain authoritative and non-bypassable when autonomous actions become operationally consequential.

Runtime governance represents one possible path toward that future.

As autonomous systems become increasingly capable of consequential action, governance effectiveness will increasingly depend upon the ability to evaluate legitimacy at the commit boundary. Deterministic runtime governance is therefore fundamentally concerned with governing the transition between proposed intent and operational execution.

Author Biography

John M. Willis is a systems architect, cybersecurity practitioner, inventor, and technology entrepreneur with more than three decades of experience spanning cybersecurity, artificial intelligence, critical infrastructure, telecommunications, healthcare, defense, and federal systems.

His professional experience includes support for U.S. federal programs and initiatives as a contractor to organizations including the U.S. Patent and Trademark Office (USPTO), the Department of Homeland Security (DHS) Headquarters Identity, Credential, and Access Management (ICAM) Program Management Office, the DHS Science and Technology Directorate (S&T), the United States

Secret Service (USSS), and the Transportation Security Administration (TSA). His most recent federal work has focused on Zero Trust architecture, identity systems, multi-factor authentication (MFA), strong application authentication, privileged access management (PAM), governance, and security engineering.

Mr. Willis has completed the National Security Agency (NSA) Information Security (INFOSEC) Assessment Methodology (IAM) and Information Security (INFOSEC) Evaluation Methodology (IEM) training programs, Cybersecurity and Infrastructure Security Agency (CISA) High Value Asset (HVA) Technical Lead assessment training, and the Carnegie Mellon University Software Engineering Institute (CMU/SEI) CISO Executive Program. In 2024, he completed the Applied Data Science Program: Leveraging AI for Effective Decision Making through Massachusetts Institute of Technology (MIT) Professional Education.

He is the founder of Sustainable Future Tech Inc., where his current research focuses on Secure-by-Design AI, runtime governance, governance assurance, deterministic governance architectures, autonomous systems, and governance metrology. He is also the lead author of the AI Governance Control Plane (AGCP) specification, a requirements-based runtime governance framework for autonomous and agentic systems.

Mr. Willis holds a Master of Science degree in Sustainable Development from the University of Sussex and is the inventor on multiple patent filings and the author of technical publications spanning cybersecurity, artificial intelligence, sustainability, and systems architecture.

ORCID: <https://orcid.org/0009-0007-5889-9628>

Copyright Notice

Copyright © 2026 John M. Willis and Sustainable Future Tech, Inc.

This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

You are free to share and adapt this material for any purpose, including commercial use, provided appropriate credit is given to the author, a link to the license is provided, and any changes made are indicated.

License details: <https://creativecommons.org/licenses/by/4.0/>

Trademark Notice

AGCP™ (AI Governance Control Plane) is a trademark of Sustainable Future Tech, Inc.

All other trademarks are the property of their respective owners.

A Runtime Governance Terminology Crosswalk

A.1 Purpose

The emergence of autonomous and agentic AI systems has introduced a growing body of terminology that is often used inconsistently across governance, cybersecurity, safety, assurance, and systems-engineering disciplines (NIST, 2023b; ISO, 2023; Wang et al., 2023; Xi et al., 2023).

The definitions below are intended to support architectural discussion and implementation analysis. They are not intended to supersede definitions established by standards bodies, regulatory authorities, or industry frameworks.

A.2 Core Runtime Governance Terminology

Table 7: Core Runtime Governance Terminology

Term	Definition
Admissibility	The determination that a proposed action remains eligible for execution under current conditions, constraints, evidence, and governance requirements.
Assurance Case	A structured argument supported by evidence intended to justify confidence in a system, governance capability, or operational outcome.
Authorization	A governance decision granting permission for a proposed action to execute. Authorization alone does not imply admissibility.
Authority Lineage	The traceable chain of authority relationships supporting a governance decision.
Canonical State	The authoritative operational state used as the basis for governance evaluation and decision-making.
Canonical-State Validation	The process of verifying that governance decisions are based upon authoritative and current operational state information.
Commit Boundary	The transition point at which a proposed action becomes operationally consequential and can affect systems, data, resources, or physical environments.
Commit Decision	The final governance determination that execution may proceed across the commit boundary.
Composite Governance	Governance evaluation involving multiple authorities, policies, or governance domains simultaneously.
Constitutional Governance	Governance constraints derived from foundational principles, policies, or higher-order governance requirements that constrain operational decisions.
Continuation Integrity	Preservation of governance validity across deferred, asynchronous, or multi-stage workflows.
Delegated Authority	Authority transferred from one actor to another subject to governance constraints and accountability requirements.

Term	Definition
Deterministic Governance	A governance property in which equivalent inputs, state conditions, evidence, and policies produce equivalent governance outcomes.
Evidence Continuity	The ability to demonstrate that supporting evidence remains attributable, available, and valid throughout the governance lifecycle.
Evidence Provenance	Documented lineage describing the origin, custody, and transformation history of governance evidence.
Execution-Bound Authorization	A governance model in which execution remains cryptographically, logically, or procedurally bound to the specific authorization that approved it.
Execution-Time Governance	Governance activities performed immediately before, during, or directly associated with execution of an operational action.
Governance Compilation	The transformation of governance intent, policies, or requirements into operational governance artifacts or executable representations.
Governance Context	The collection of information, constraints, authorities, evidence, state information, and conditions relevant to governance evaluation.
Governance Continuity	Preservation of governance semantics, constraints, and decision lineage throughout a workflow or operational lifecycle.
Governance Escalation	The transfer of a governance decision to a higher authority, alternative authority, or human reviewer.
Governance Evidence	Information used to support authorization, admissibility, escalation, refusal, or other governance outcomes.
Governance Invariant	A condition that must remain true throughout governance evaluation and execution. Violation of an invariant results in refusal, escalation, or other governance action.
Governance Interoperability	The ability of governance information, decisions, evidence, and outcomes to operate across organizational or technical boundaries.
Governance Lifecycle Integrity	The preservation of valid governance state transitions throughout the lifecycle of an action or workflow.
Governance Metrology	The discipline concerned with measurement, evaluation, repeatability, and assurance of governance behavior.
Governance Object	A governance artifact such as an authorization, approval, receipt, evidence record, policy assertion, or governance decision.
Governance Receipt	A durable record describing a governance decision and the information supporting that decision.
Governance Replay	The process of reconstructing and re-evaluating a prior governance decision using preserved governance records, evidence, and state information.
Governance Replayability	The ability to reconstruct, reproduce, or independently verify governance decisions using preserved records and evidence.
Human-in-the-Loop (HITL)	A governance model requiring explicit human approval before execution may proceed.

Term	Definition
Human-on-the-Loop (HOTL)	A governance model in which humans maintain supervisory authority over autonomous execution.
Multi-Agent Governance	Governance mechanisms that preserve accountability, authorization, admissibility, and governance continuity across multiple autonomous actors.
Operational Legitimacy	The condition in which a proposed action remains permissible, appropriate, and valid under current operational conditions.
Operational State	The set of conditions, facts, resources, system attributes, and environmental characteristics describing the current operating environment.
Runtime Assurance	The ability to measure, verify, validate, and assess governance behavior during operation.
Runtime Governance	Governance controls evaluated during operational execution rather than solely during design, development, deployment, or post-execution review.
Runtime Governance Architecture	The collection of systems, services, processes, and controls responsible for evaluating and enforcing governance during operation.
Secure-by-Design Runtime Control	A runtime governance mechanism that contributes directly to security, accountability, resilience, safety, or operational assurance objectives.
Structural Refusal	A governance outcome that prevents execution because governance requirements cannot be satisfied.

A.3 Relationship Among Key Concepts

Several terms used throughout this paper are closely related but should not be treated as interchangeable.

- Authorization and admissibility are distinct concepts. Authorization grants permission. Admissibility determines whether execution remains appropriate under current conditions.
- Governance and execution are distinct concerns. Governance evaluates legitimacy. Execution performs actions.
- Evidence and authorization are distinct artifacts. Evidence supports decisions. Authorization represents decisions.
- Governance continuity and workflow continuity are related but different. A workflow may continue while governance continuity has been lost.
- Determinism and correctness are not synonymous. A governance system may behave deterministically while still producing incorrect outcomes.
- Operational legitimacy and authorization are not synonymous. An action may remain authorized while no longer being admissible.

- Governance replay and governance replayability are related but distinct. Replay is the activity; replayability is the capability.

A.4 Terminology and Framework Neutrality

The terminology presented in this appendix is intended to remain broadly applicable across runtime governance architectures, Secure-by-Design implementations, assurance frameworks, and autonomous-system governance models (NIST, 2023b; ISO, 2023; CISA, 2023; NIST, 2018b).

Although some of these concepts appear within specific runtime governance specifications, the terminology itself is presented here as a framework-neutral vocabulary intended to support broader discussion of execution-time governance challenges and solutions. The intent is to provide a common descriptive language rather than a normative replacement for terminology defined by existing standards, frameworks, or regulatory guidance (NIST, 2023b; ISO, 2023; European Union, 2024).

B Runtime Governance Control Categories

B.1 Purpose

The runtime governance concepts discussed throughout this paper can be organized into a set of control categories representing distinct governance functions. These categories provide a conceptual framework for understanding how execution-time governance controls contribute to Secure-by-Design objectives, operational assurance, accountability, and autonomous-system governance (NIST, 2023b; ISO, 2023; CISA, 2023; NIST, 2018b).

The categories presented in this appendix are intentionally implementation-neutral. Individual runtime governance architectures may implement these capabilities using different mechanisms, technologies, or operational models.

B.2 Runtime Governance Control Categories

Table 8: Runtime Governance Control Categories

Control Category	Primary Objective	Representative Activities
Admissibility Controls	Determine whether execution remains permissible under current conditions	Constraint evaluation, policy evaluation, operational legitimacy assessment, risk evaluation
Authorization Controls	Bind execution to approved governance decisions	Authorization validation, authority verification, delegation validation, approval enforcement
Canonical-State Controls	Ensure governance decisions are based upon authoritative operational state	State validation, state reconciliation, operational context validation, state consistency verification
Composite Governance Controls	Evaluate governance outcomes across multiple governance domains	Policy aggregation, conflict resolution, multi-authority evaluation, governance composition
Delegation Controls	Preserve governance integrity when authority is transferred	Delegation validation, authority lineage verification, delegated authorization evaluation
Evidence Controls	Ensure governance decisions remain supported by attributable evidence	Evidence collection, provenance validation, evidence continuity verification, evidence retention
Execution Controls	Govern governance decisions at the point of operational commitment	Commit evaluation, execution authorization, commit-bound governance enforcement
Governance Continuity Controls	Preserve governance semantics throughout workflows and lifecycles	Workflow continuity validation, lifecycle integrity verification, continuation governance

Control Category	Primary Objective	Representative Activities
Governance Interoperability Controls	Support governance interactions across technical and organizational boundaries	Governance exchange, authority recognition, interoperability validation, federation support
Human Oversight Controls	Integrate human review into governance processes when required	Escalation, Human-in-the-Loop approval, supervisory review, exception handling
Lifecycle Governance Controls	Manage governance state transitions throughout operational lifecycles	Submission, authorization, execution, completion, expiration, revocation management
Metrology Controls	Measure governance effectiveness, correctness, and assurance characteristics	Conformance measurement, replay validation, assurance assessment, governance metrics
Multi-Agent Governance Controls	Preserve governance continuity across autonomous actors	Authority propagation, context preservation, cross-agent authorization, accountability continuity
Policy Controls	Apply governance requirements to proposed actions	Policy evaluation, policy interpretation, constraint application, constitutional enforcement
Replayability Controls	Enable reconstruction and validation of prior governance decisions	Governance replay, decision reconstruction, audit support, independent verification
Refusal Controls	Prevent inadmissible actions from executing	Structural refusal, enforcement, constraint violation handling, execution prevention
Runtime Assurance Controls	Provide confidence that governance controls operate as intended	Runtime verification, monitoring, validation, assurance evidence collection
Traceability Controls	Preserve attribution and accountability throughout governance processes	Lineage preservation, attribution tracking, governance receipts, audit support

B.3 Control Categories and Secure-by-Design Objectives

The control categories above align naturally with Secure-by-Design principles by emphasizing governance activities that occur before operational consequences occur (CISA, 2023; Saltzer and Schroeder, 1975; NIST, 2020a, 2018b).

B.4 Control Categories and Runtime Governance Functions

While individual implementations may vary, the control categories generally operate together as part of an execution-time governance process.

A simplified sequence is:

Table 9: Runtime Governance Control Categories and Secure-by-Design Objectives

Secure-by-Design Objective	Primary Runtime Governance Control Categories
Secure Defaults	Refusal Controls, Admissibility Controls, Policy Controls
Least Privilege	Authorization Controls, Delegation Controls, Execution Controls
Complete Mediation	Execution Controls, Admissibility Controls, Canonical-State Controls
Accountability	Traceability Controls, Evidence Controls, Replayability Controls
Auditability	Replayability Controls, Evidence Controls, Metrology Controls
Resilience	Canonical-State Controls, Runtime Assurance Controls, Lifecycle Governance Controls
Human Oversight	Human Oversight Controls, Escalation Controls, Delegation Controls
Operational Assurance	Runtime Assurance Controls, Metrology Controls, Governance Continuity Controls

1. Proposed action received.
2. Policy controls identify applicable governance requirements.
3. Evidence controls gather supporting information.
4. Canonical-state controls validate operational conditions.
5. Authorization controls verify authority.
6. Admissibility controls evaluate legitimacy.
7. Refusal controls or escalation controls intervene if required.
8. Execution controls govern commitment of the action.
9. Traceability and evidence controls generate governance records.
10. Replayability and metrology controls support future assurance activities.

This sequence illustrates how multiple control categories contribute collectively to execution-time governance rather than operating independently (NIST, 2018b, 2021; Open Group, 2022).

B.5 Relationship to Runtime Governance Architectures

The control categories presented in this appendix represent governance functions rather than specific technologies or products (Open Group, 2022; NIST, 2018b; ISO, 2023).

A runtime governance architecture may implement one or more categories using:

- policy engines

- workflow systems
- authorization services
- governance ledgers
- evidence repositories
- orchestration platforms
- monitoring systems
- assurance frameworks

The specific implementation mechanism is less important than the governance objective being achieved (Open Group, 2022; NIST, 2018b).

B.6 Summary

The control categories described in this appendix provide a functional view of runtime governance capabilities. They help organize governance concepts into understandable governance functions while remaining independent of any specific implementation approach.

Viewed collectively, these categories illustrate how execution-time governance extends traditional governance concepts into operational environments where autonomous systems increasingly propose, coordinate, and execute consequential actions (Wang et al., 2023; Xi et al., 2023; NIST, 2023b; ISO, 2023).

C Runtime Governance Concept Mapping to AGCP Requirement Groups

C.1 Purpose

The body of this paper presented runtime governance concepts using framework-neutral terminology. The objective was to focus on execution-time governance challenges and Secure-by-Design outcomes rather than any specific implementation approach.

This appendix provides an illustrative mapping between the runtime governance concepts discussed throughout the paper and the Requirement Groups defined within the Artificial Intelligence Governance Control Plane (AGCP) specification (Willis, 2026a,b,c,d).

The mapping is intended to demonstrate how one runtime governance specification translates conceptual governance objectives into structured requirements. It should not be interpreted as implying that AGCP is the only implementation approach capable of supporting these concepts.

C.2 Concept-to-Requirement Group Mapping

The runtime governance concepts introduced throughout this paper are realized through specific requirement groups within the AGCP architecture. Although many concepts depend upon controls spanning multiple governance domains, most concepts have one or more primary requirement groups that provide their principal implementation foundation. Table 10 summarizes these relationships and identifies the primary governance focus associated with each concept.

Table 10: Concept-to-Requirement Group Mapping

Runtime Governance Concept	Primary RG	Governance Focus
Admissibility Evaluation	RG-2, RG-4, RG-5	Determination of operational legitimacy prior to execution
Authorization Continuity	RG-1, RG-2	Preservation of authorization validity throughout execution
Canonical-State Governance	RG-4	Authoritative operational-state determination and validation
Commit-Bound Governance	RG-2	Governance enforcement at the point of operational commitment
Composite Governance	RG-11	Coordination of multiple governance authorities and policies
Constitutional Governance	RG-12	Application of higher-order governance constraints and principles
Delegated Authority Governance	RG-7	Preservation of governance semantics across delegated workflows
Deterministic Governance	RG-9	Measurement and validation of governance consistency and reproducibility
Evidence Continuity	RG-6	Preservation of governance evidence throughout operational lifecycles

Runtime Governance Concept	Primary RG	Governance Focus
Execution-Bound Authorization	RG-2	Binding execution to specific authorization outcomes
Governance Continuity	RG-7, RG-10	Preservation of governance intent throughout workflows
Governance Escalation	RG-5	Transfer of governance decisions to alternative authorities
Governance Interoperability	RG-8	Cross-domain governance coordination and exchange
Governance Lifecycle Integrity	RG-1	Management of governance state transitions and lifecycle validity
Governance Metrology	RG-9	Measurement and assessment of governance characteristics
Governance Replayability	RG-6, RG-9	Reconstruction and verification of governance outcomes
Human Oversight Governance	RG-5	Human review, escalation, and exception handling
Multi-Agent Governance	RG-7	Governance preservation across autonomous actors
Operational Legitimacy Evaluation	RG-2, RG-4	Evaluation of execution eligibility under current conditions
Runtime Assurance	RG-9	Verification and validation of governance behavior during operation
Structural Refusal	RG-5	Deterministic prevention of inadmissible execution
Workflow Continuation Integrity	RG-10	Governance preservation throughout extended workflows

Table 10 illustrates that runtime governance concepts are not isolated capabilities but are instead rooted within a broader governance architecture composed of multiple governance functions implemented through distinct requirement groups. While individual requirement groups provide primary implementation responsibility for particular concepts, many concepts depend upon supporting controls from multiple governance domains. This relationship reflects the interconnected nature of runtime governance, where admissibility evaluation, authorization continuity, governance continuity, evidence preservation, structural refusal, and runtime assurance operate as mutually reinforcing capabilities. The mapping further demonstrates how abstract governance concepts can be systematically translated into implementation-oriented governance structures suitable for specification development, conformance assessment, and operational deployment.

C.3 AGCP Requirement Group Summary

For purposes of organization and analysis, the AGCP requirements referenced in this appendix are grouped into twelve Requirement Groups (RGs). These groups provide a convenient mechanism for relating runtime governance concepts to corresponding portions of the AGCP requirements catalog.

The requirement groups shown in Table 11 represent recurring governance functions identified

Table 11: AGCP Requirement Groups

RG	Name	Primary Objective
RG-1	Lifecycle Governance	Governance state transitions and lifecycle integrity
RG-2	Execution Governance	Execution-time authorization and commit-bound governance
RG-3	Governance Object Integrity	Integrity and protection of governance artifacts
RG-4	Canonical-State Governance	Authoritative operational-state validation
RG-5	Refusal and Escalation Governance	Structural refusal and governance escalation
RG-6	Governance Evidence and Receipts	Evidence continuity and governance records
RG-7	Multi-Agent Runtime Governance	Governance continuity across autonomous actors
RG-8	Governance Interoperability	Cross-domain governance interactions
RG-9	Runtime Governance Metrology	Measurement, assurance, and conformance evaluation
RG-10	Continuation Integrity Governance	Governance continuity across workflow stages
RG-11	Composite Governance	Evaluation across multiple governance domains
RG-12	Governance Compilation and Constitutional Governance	Transformation of governance intent into operational controls

throughout the runtime governance architecture. Although each group addresses a distinct governance concern, the groups are intended to operate collectively rather than independently. Execution governance (RG-2), canonical-state governance (RG-4), continuation integrity governance (RG-10), and governance compilation and constitutional governance (RG-12) appear repeatedly throughout the runtime governance model because they provide foundational capabilities that support many other governance functions. Together, the requirement groups establish a structured governance framework capable of translating governance intent into execution-time controls, preserving governance continuity throughout operational workflows, and supporting governance assurance activities after execution has occurred.

C.4 Concept Coverage Across Requirement Groups

Several runtime governance concepts span multiple requirement groups (Willis, 2026b,c,e).

This is intentional.

For example:

- Admissibility evaluation depends upon execution governance, canonical-state validation, and refusal capabilities.

- Runtime assurance depends upon evidence continuity, replayability, and governance metrology.
- Multi-agent governance depends upon lifecycle integrity, evidence continuity, interoperability, and continuation integrity.
- Governance replayability depends upon both governance records and measurement capabilities.

As a result, runtime governance concepts should generally be viewed as cross-cutting governance objectives rather than isolated controls (Willis, 2026b,c).

C.5 Conceptual Layering

The mapping can also be viewed through a layered governance perspective (Willis, 2026b,a,c).

Table 12: Illustrative Governance Layering

Layer	Representative Requirement Groups
Governance Intent	RG-11, RG-12
Governance Evaluation	RG-2, RG-4, RG-5
Governance Evidence	RG-3, RG-6
Governance Continuity	RG-1, RG-7, RG-10
Governance Interoperability	RG-8
Governance Assurance	RG-9

Table 12 illustrates one possible architectural decomposition of runtime governance responsibilities. In this view, governance intent establishes the policies, constraints, and constitutional principles that define acceptable behavior. Governance evaluation determines whether proposed actions remain authorized and admissible under current conditions. Governance evidence preserves the information required to justify governance decisions, while governance continuity maintains governance semantics across workflows, lifecycle transitions, and autonomous interactions. Governance interoperability supports coordination across governance domains, and governance assurance provides independent assessment of governance effectiveness. Together, these layers illustrate how runtime governance may be viewed as a structured execution-time governance architecture rather than a collection of isolated governance controls (Willis, 2026b,c).

C.6 Relationship to Secure-by-Design Objectives

The mapping presented in this appendix demonstrates that runtime governance concepts are not independent of Secure-by-Design objectives (Willis, 2026c; CISA, 2023; NIST, 2018b).

Instead, the requirement groups collectively contribute to:

- secure defaults
- least privilege

- complete mediation
- accountability
- auditability
- resilience
- human oversight
- operational assurance

through execution-time governance mechanisms (Willis, 2026c,d; CISA, 2023; Saltzer and Schroeder, 1975; NIST, 2020a).

For example, execution governance, admissibility evaluation, and structural refusal contribute to secure-default and complete-mediation objectives by ensuring that actions are evaluated immediately prior to operational commitment. Similarly, governance evidence, replayability, and metrology capabilities support accountability and auditability objectives by enabling governance decisions to be reconstructed, examined, and independently verified after execution (Willis, 2026e,c; NIST, 2022).

Viewed from this perspective, runtime governance may be understood as an operational mechanism for extending established Secure-by-Design principles into increasingly autonomous execution environments.

C.7 Illustrative Characteristics of AGCP

While multiple runtime governance architectures may emerge, AGCP is notable for combining several governance properties within a single execution-time specification (Willis, 2026c,d).

Representative characteristics include:

- explicit governance lifecycle semantics
- execution-bound authorization
- canonical-state validation
- admissibility evaluation
- structural refusal
- governance continuity
- governance replayability
- runtime governance metrology
- conformance-based assessment

Collectively, these capabilities seek to address governance concerns that arise when autonomous systems interact with operational environments, external systems, delegated authorities, and multi-step workflows (Willis, 2026a,b,e).

The purpose of this appendix is not to claim exclusivity for these concepts. Rather, it illustrates how one runtime governance specification translates execution-time governance principles into an integrated requirements model suitable for implementation and assessment (Willis, 2026c,d).

Many of the concepts represented within AGCP draw upon broader traditions of systems security engineering, runtime assurance, governance, safety engineering, and Secure-by-Design thinking. The AGCP contribution is the integration of these concepts into a coherent execution-layer governance model focused on autonomous and programmatic systems (NIST, 2018b, 2021; CISA, 2023; Willis, 2026c).

C.8 Summary

This appendix illustrates how one runtime governance specification translates conceptual governance objectives into structured governance requirements (Willis, 2026a,c,d).

The primary purpose of the mapping is explanatory rather than normative. It provides a bridge between the framework-neutral concepts discussed throughout this paper and a concrete example implementation specification.

Viewed in this manner, AGCP may be understood as one example of how runtime governance concepts can be operationalized through explicit requirements, governance lifecycle semantics, evidence continuity mechanisms, and execution-time control enforcement (Willis, 2026b,c,d).

At the same time, the broader runtime governance concepts discussed throughout this paper are not dependent upon any single implementation approach. Alternative architectures, governance frameworks, and execution-time control models may implement similar objectives using different design patterns, technologies, and governance mechanisms.

While this appendix focuses on conceptual mappings, the complete AGCP requirement catalog, conformance model, and supporting documentation are publicly available through the AGCP project website and associated publications (Willis, 2026d,c).

D Secure-by-Design Runtime Governance Mapping

D.1 Purpose

Secure-by-Design initiatives emphasize building security, resilience, accountability, and operational trustworthiness directly into systems rather than relying solely upon external oversight, monitoring, or post-incident response (CISA, 2023; NIST, 2018b, 2020a).

The body of this paper argues that as AI systems become increasingly autonomous, Secure-by-Design objectives must increasingly be supported by execution-time governance mechanisms (Wang et al., 2023; Xi et al., 2023; NIST, 2023b).

This appendix expands the Secure-by-Design discussion presented earlier in the paper and illustrates how runtime governance capabilities may contribute to Secure-by-Design outcomes.

The mapping presented here is intended to be illustrative rather than normative. Different implementations may achieve similar outcomes using different governance architectures.

D.2 Secure-by-Design Objective Mapping

Table 13: Secure-by-Design Objective Mapping

Secure-by-Design Objective	Primary Runtime Governance Capability	Representative Governance Outcome
Accountability	Governance receipts, evidence continuity, traceability	Attributable governance decisions and execution outcomes
Auditability	Governance replayability, evidence preservation	Independent reconstruction and verification of governance decisions
Complete Mediation	Execution-time evaluation, commit-bound governance	Every consequential action evaluated before execution
Defense in Depth	Layered governance evaluation and enforcement	Multiple governance checks applied throughout operational workflows
Human Oversight	Escalation governance, Human-in-the-Loop controls	Human review when automated governance cannot resolve uncertainty
Least Privilege	Execution-bound authorization, delegation controls	Actions constrained to authorized authority boundaries
Operational Assurance	Runtime assurance, governance metrology	Measurable confidence in governance behavior during operation
Resilience	Canonical-state validation, lifecycle governance	Governance decisions remain valid under changing operational conditions
Safe Failure	Structural refusal, governance escalation	Inadmissible actions prevented from executing

Secure-by-Design Objective	Primary Runtime Governance Capability	Representative Governance Outcome
Secure Defaults	Structural refusal and deny-by-default governance behavior	Execution prevented unless governance requirements are satisfied
Traceability	Governance continuity, authority lineage, governance receipts	Preservation of decision lineage throughout workflows
Trustworthiness	Integrated governance, assurance, and accountability mechanisms	Increased confidence in operational outcomes

D.3 Runtime Governance Contribution by Secure-by-Design Principle

The following sections summarize how runtime governance contributes to commonly recognized Secure-by-Design objectives.

D.3.1 Secure Defaults

Secure-by-Design emphasizes systems that fail safely and avoid unnecessary risk exposure (CISA, 2023; Saltzer and Schroeder, 1975).

Runtime governance contributes through:

- structural refusal
- deny-by-default execution behavior
- authorization validation
- admissibility evaluation

Under this model, actions execute only when governance conditions are satisfied.

D.3.2 Least Privilege

Least privilege seeks to minimize authority and access beyond what is necessary (Saltzer and Schroeder, 1975; NIST, 2020a).

Runtime governance contributes through:

- execution-bound authorization
- delegation validation
- authority lineage preservation
- authorization continuity

These capabilities help ensure that operational actions remain constrained to authorized authority boundaries.

D.3.3 Complete Mediation

Complete mediation requires that access decisions be evaluated whenever protected resources are accessed (Saltzer and Schroeder, 1975).

Runtime governance extends this principle to autonomous operational actions through:

- execution-time governance evaluation
- commit-bound governance enforcement
- admissibility determination
- canonical-state validation

This helps ensure that governance evaluation occurs immediately before consequential actions are committed.

D.3.4 Accountability

Secure-by-Design systems should support clear attribution of actions and decisions (NIST, 2020a, 2018b).

Runtime governance contributes through:

- governance receipts
- evidence continuity
- authority lineage
- governance replayability

These capabilities support attribution and post-execution review.

D.3.5 Auditability

Organizations frequently require the ability to independently verify governance outcomes (NIST, 2022, 2020a).

Runtime governance supports auditability through:

- governance evidence preservation
- governance replay

- replayability mechanisms
- governance metrology

These controls support independent validation of governance behavior.

D.3.6 Resilience

Secure-by-Design systems should remain dependable despite changing conditions (NIST, 2021, 2018b).

Runtime governance contributes through:

- canonical-state governance
- lifecycle integrity controls
- continuation integrity controls
- runtime assurance mechanisms

These capabilities help maintain governance validity as operational conditions evolve.

D.3.7 Human Oversight

Human oversight remains an important component of many governance models (NIST, 2023b; ISO, 2023; European Union, 2024).

Runtime governance supports oversight through:

- escalation mechanisms
- Human-in-the-Loop governance
- Human-on-the-Loop supervision
- exception management

These capabilities enable human review when automated governance processes encounter uncertainty, ambiguity, or exceptional conditions.

D.4 Secure-by-Design and the Runtime Governance Gap

The central thesis of this paper is that Secure-by-Design objectives increasingly require execution-time support as systems become more autonomous (Wang et al., 2023; Xi et al., 2023; CISA, 2023).

Many Secure-by-Design principles define desired outcomes:

- accountability
- resilience
- least privilege
- secure defaults
- auditability

Runtime governance focuses on operational mechanisms that may help achieve those outcomes at the point where actions become operationally consequential.

Viewed in this way, runtime governance may be understood as a complementary implementation layer supporting Secure-by-Design objectives rather than as a competing governance framework (CISA, 2023; NIST, 2018b, 2023b).

D.5 Illustrative Mapping to Runtime Governance Control Categories

The Secure-by-Design objectives discussed above can also be mapped to the runtime governance control categories introduced in Appendix B.

Table 14: Secure-by-Design Objectives and Runtime Governance Control Categories

Secure-by-Design Objective	Representative Runtime Governance Control Categories
Secure Defaults	Refusal Controls, Admissibility Controls, Policy Controls
Least Privilege	Authorization Controls, Delegation Controls, Execution Controls
Complete Mediation	Execution Controls, Canonical-State Controls, Admissibility Controls
Accountability	Traceability Controls, Evidence Controls, Replayability Controls
Auditability	Replayability Controls, Metrology Controls, Evidence Controls
Resilience	Canonical-State Controls, Lifecycle Governance Controls, Runtime Assurance Controls
Human Oversight	Human Oversight Controls, Escalation Controls
Operational Assurance	Runtime Assurance Controls, Metrology Controls, Governance Continuity Controls

D.6 Summary

Secure-by-Design initiatives establish important objectives for trustworthy systems (CISA, 2023; NIST, 2018b; ISO, 2023).

As AI systems become increasingly capable of autonomous action, achieving these objectives may require governance mechanisms that operate during execution rather than solely during development, deployment, or post-execution review (Wang et al., 2023; Xi et al., 2023; NIST, 2023b).

Runtime governance represents one possible approach for providing such capabilities.

The mappings presented in this appendix illustrate how execution-time governance controls may contribute to Secure-by-Design outcomes through authorization, admissibility evaluation, evidence continuity, governance continuity, structural refusal, replayability, and runtime assurance mechanisms (CISA, 2023; NIST, 2018b, 2021).

E Standards and Framework Crosswalk

E.1 Purpose

The body of this paper demonstrated that runtime governance is not intended to replace existing governance, cybersecurity, assurance, or Secure-by-Design frameworks.

Instead, runtime governance may be viewed as an execution-time implementation layer that complements existing governance objectives by addressing operational decision-making at the point where autonomous actions become consequential.

This appendix consolidates the framework mappings presented throughout the paper into a single reference section intended to support:

- standards analysis
- procurement evaluations
- architecture reviews
- control mapping exercises
- governance gap assessments

The mappings are illustrative rather than normative and are intended to demonstrate potential areas of alignment.

The runtime governance contributions and representative capabilities shown throughout this appendix should not be interpreted as deficiencies within the referenced frameworks. Rather, they illustrate examples of execution-time governance mechanisms that may support, operationalize, or extend framework objectives during system operation.

E.2 Crosswalk Overview

Table 15: Framework Alignment Summary

Framework	Primary Focus	Runtime Governance Contribution
NIST AI RMF	AI risk management	Execution-time operational governance
NIST CSF 2.0	Cybersecurity outcomes	Commit-bound governance controls
NIST SP 800-53	Security controls	Execution authorization and governance assurance
CISA Secure-by-Design	Security engineering principles	Runtime enforcement mechanisms
MITRE ATLAS	Adversarial AI threats	Operational mitigation and governance mediation
OWASP Agentic AI	Agentic AI security risks	Execution-time governance controls
ENISA AI Security Guidance	AI cybersecurity practices	Runtime assurance and governance continuity
ISO/IEC 42001	AI management systems	Operational governance implementation support

The frameworks summarized in Table 15 address governance objectives from multiple perspectives, including risk management, cybersecurity, assurance, operational controls, adversarial resilience, and organizational governance. Runtime governance contributes an execution-time implementation perspective that complements these objectives by providing mechanisms capable of evaluating and controlling actions at the point of execution. Rather than replacing existing frameworks, runtime governance may be viewed as an architectural layer that helps operationalize governance intent as autonomous systems become increasingly capable of consequential action.

E.3 NIST AI Risk Management Framework

Table 16: NIST AI RMF and Runtime Governance

AI RMF Function	Runtime Governance Contribution	Representative Runtime Gap Addressed
Govern	Operational governance enforcement	Execution-time governance evaluation
Map	Context preservation and state validation	Operational context drift
Measure	Runtime assurance and governance metrology	Governance effectiveness measurement
Manage	Execution-time decision mediation	Operational action governance

E.4 NIST Cybersecurity Framework 2.0

Table 17: NIST CSF 2.0 and Runtime Governance

CSF Function	Runtime Governance Contribution	Representative Runtime Gap Addressed
Govern	Operational governance enforcement	Execution semantics unspecified
Protect	Execution-time authorization	Action legitimacy determination
Detect	Governance condition monitoring	Commit-bound evaluation
Respond	Governance escalation mechanisms	Autonomous action mediation
Recover	Governance replayability	Execution decision reconstruction

E.5 NIST SP 800-53

Table 18: Representative NIST SP 800-53 Alignment

Control Family	Runtime Governance Contribution	Representative Capability
AC – Access Control	Execution authorization and authority containment	Commit-bound authorization and authority validation
AU – Audit and Accountability	Governance replayability	Governance receipts
CA – Assessment and Authorization	Runtime assurance	Governance metrology
IR – Incident Response	Governance escalation	Human intervention workflows
RA – Risk Assessment	Admissibility evaluation	Operational legitimacy determination
SI – System Integrity	Canonical-state validation and governance integrity protection	State consistency assurance and governance self-protection

E.6 CISA Secure-by-Design

Table 19: CISA Secure-by-Design and Runtime Governance

Secure-by-Design Principle	Runtime Governance Contribution	Representative Capability
Secure Defaults	Structural refusal and continuation integrity controls	Deny-by-default execution and safe continuation governance
Least Privilege	Execution-bound authorization, authority containment, and governance isolation	Authority validation and delegated-authority governance
Complete Mediation	Commit-bound governance and enforcement integrity	Execution-time evaluation and governance-enforcement binding
Accountability	Governance receipts	Decision attribution
Resilience	Canonical-state validation	Operational state assurance
Human Accountability	Governance escalation	Human review workflows

E.7 MITRE ATLAS

Table 20: MITRE ATLAS and Runtime Governance

ATLAS Concern Area	Runtime Governance Contribution	Representative Capability
Prompt Manipulation	Execution-time mediation	Admissibility evaluation
Tool Misuse	Execution authorization	Authority validation
Privilege Escalation	Delegation controls	Authority lineage verification
Operational Abuse	Structural refusal	Execution prevention
Autonomous Actions	Governance continuity	Lifecycle integrity controls

E.8 OWASP Agentic AI

Table 21: OWASP Agentic AI and Runtime Governance

Agentic AI Risk Area	Runtime Governance Contribution	Representative Capability
Autonomous Tool Use	Execution authorization	Tool invocation governance
Context Manipulation	Canonical-state validation	Context integrity validation
Delegated Authority	Delegation controls and authority containment	Authority continuity and recursive delegation control
Cross-System Actions	Governance interoperability and governance isolation	Cross-domain governance and authority-boundary preservation
Agent Coordination	Multi-agent governance	Governance continuity and autonomous coordination governance

E.9 ENISA AI Security Guidance

ENISA’s guidance on securing artificial intelligence systems emphasizes lifecycle security, operational monitoring, risk management, resilience, and continuous assurance throughout AI system deployment and operation (ENISA, 2023).

The guidance highlights the importance of maintaining security controls after deployment and recognizing that operational conditions may evolve throughout a system’s lifecycle.

Deterministic runtime governance complements ENISA’s recommendations by introducing execution-time decision mediation at the point where proposed actions become executable operations.

Table 22: ENISA AI Security Guidance and Runtime Governance

ENISA Focus Area	Runtime Governance Contribution	Representative Capability
Lifecycle Security	Continuous governance evaluation throughout operational execution	Governance lifecycle integrity
Operational Monitoring	Execution-time assessment of proposed actions and current conditions	Runtime assurance telemetry
Risk Management	Context-sensitive evaluation before execution authorization	Admissibility evaluation
System Resilience	Prevention of inadmissible or unsafe operational actions	Structural refusal
Accountability and Auditability	Traceable governance decisions and execution records	Governance replayability
Operational Assurance	Preservation of governance intent during execution	Governance continuity

Runtime governance should therefore be viewed as a complementary operational mechanism that may strengthen the continuous assurance objectives emphasized throughout ENISA’s AI security guidance.

E.10 ISO/IEC 42001

ISO/IEC 42001 establishes requirements for Artificial Intelligence Management Systems (AIMS), providing organizational governance processes intended to support the responsible development, deployment, operation, and continual improvement of AI systems (ISO, 2023).

The standard focuses primarily on management-system processes, organizational accountability, risk management, policy development, oversight, and continuous improvement activities.

Deterministic runtime governance does not replace these management-system functions. Rather, it may serve as an operational implementation mechanism that supports the execution of policies and controls defined within an AI Management System.

Table 23: ISO/IEC 42001 and Runtime Governance

ISO/IEC 42001 Objective	Runtime Governance Contribution	Representative Capability
AI Governance Policies	Operational enforcement and protection of approved governance rules	Policy-to-runtime control mapping and governance self-protection
Risk Management	Context-sensitive evaluation of proposed actions prior to execution	Admissibility evaluation
Operational Controls	Commit-bound governance decision-making, authorization, and enforcement integrity	Execution-bound authorization and governance-enforcement binding
Accountability	Auditable governance decisions, execution records, and governance integrity preservation	Governance lifecycle integrity and governance replayability
Continual Improvement	Governance telemetry supporting review and refinement	Governance metrology
Compliance Demonstration	Generation of verifiable governance evidence and records	Governance records and receipts

From this perspective, ISO/IEC 42001 provides organizational governance requirements, while runtime governance may provide a technical mechanism for implementing and demonstrating aspects of those requirements during operational execution.

E.11 Framework Coverage Perspective

The frameworks discussed throughout this paper generally address one or more of the following concerns (NIST, 2023b, 2024a, 2020a; CISA, 2023; MITRE, 2024; OWASP, 2025; ISO, 2023):

- governance objectives
- cybersecurity outcomes
- security controls
- risk management
- assurance practices
- adversarial threats
- operational oversight

Runtime governance does not seek to replace these concerns.

Rather, runtime governance focuses on a narrower question:

How are governance decisions evaluated, enforced, preserved, and validated at the moment autonomous actions become operationally consequential?

Viewed through this lens, runtime governance may be understood as a complementary implementation layer that operationalizes governance intent during execution.

E.12 Illustrative Coverage Model

Table 24: Illustrative Governance Layering Across Frameworks

Layer	Representative Frameworks
Governance Objectives	NIST AI RMF, ISO/IEC 42001
Cybersecurity Outcomes	NIST CSF 2.0, CISA Secure-by-Design
Security Controls	NIST SP 800-53
Threat Knowledge	MITRE ATLAS, OWASP Agentic AI
Execution-Time Governance	Runtime governance architectures and implementations
Runtime Assurance	Governance metrology and assurance frameworks

Table 24 illustrates how runtime governance may be viewed as a distinct architectural layer situated between governance intent and runtime assurance. Governance frameworks such as NIST AI RMF and ISO/IEC 42001 establish objectives, while cybersecurity frameworks and security controls provide mechanisms for achieving those objectives. Threat frameworks contribute knowledge regarding adversarial behavior and operational risk. Runtime governance introduces an execution-time decision layer responsible for evaluating whether proposed actions remain authorized, admissible, and operationally legitimate before crossing the commit boundary. Runtime assurance then provides evidence that governance mechanisms continue to operate as intended. Viewed collectively, these layers establish a traceable path from governance objectives to operational execution and post-execution assurance.

E.13 Summary

The standards and frameworks discussed throughout this paper provide essential foundations for trustworthy AI, cybersecurity, assurance, and governance (NIST, 2023b, 2024a, 2020a; CISA, 2023; MITRE, 2024; OWASP, 2025; ENISA, 2023; ISO, 2023).

Runtime governance should be viewed as complementary to these efforts rather than competitive with them.

The crosswalks presented in this appendix illustrate how execution-time governance capabilities may contribute to existing objectives through admissibility evaluation, execution authorization, evidence continuity, governance continuity, replayability, structural refusal, and runtime assurance mechanisms.

As autonomous systems continue to evolve, runtime governance may increasingly serve as a practical implementation layer connecting governance intent to operational execution.

F Reference Runtime Governance Architecture

F.1 Purpose

The body of this paper argues that runtime governance is fundamentally concerned with execution-time evaluation, authorization, admissibility determination, and assurance activities that occur before autonomous actions become operationally consequential (NIST, 2018b, 2021, 2023b).

While implementation approaches may vary, many runtime governance architectures share common structural elements, including governance evaluation functions, operational execution environments, assurance mechanisms, and governance records.

This appendix presents an illustrative reference architecture intended to provide a conceptual model for understanding how execution-time governance capabilities may be incorporated into autonomous and agentic systems (Willis, 2026a,b).

The architecture is intended to support:

- architectural analysis
- implementation planning
- governance gap assessments
- standards mapping
- Secure-by-Design evaluations

The architecture is illustrative rather than normative and should not be interpreted as prescribing a specific implementation approach.

F.2 Reference Architecture Overview

The reference architecture separates governance evaluation from operational execution while preserving a controlled relationship between the two (NIST, 2018b; Leveson, 2011; Willis, 2026b).

The architecture consists of the following primary components:

1. Requesting Actor
2. Autonomous Agent
3. Proposed Action
4. Runtime Governance Layer
5. Commit Boundary
6. Operational System
7. Governance Records and Assurance Functions

The architecture is illustrated in Figure 7.

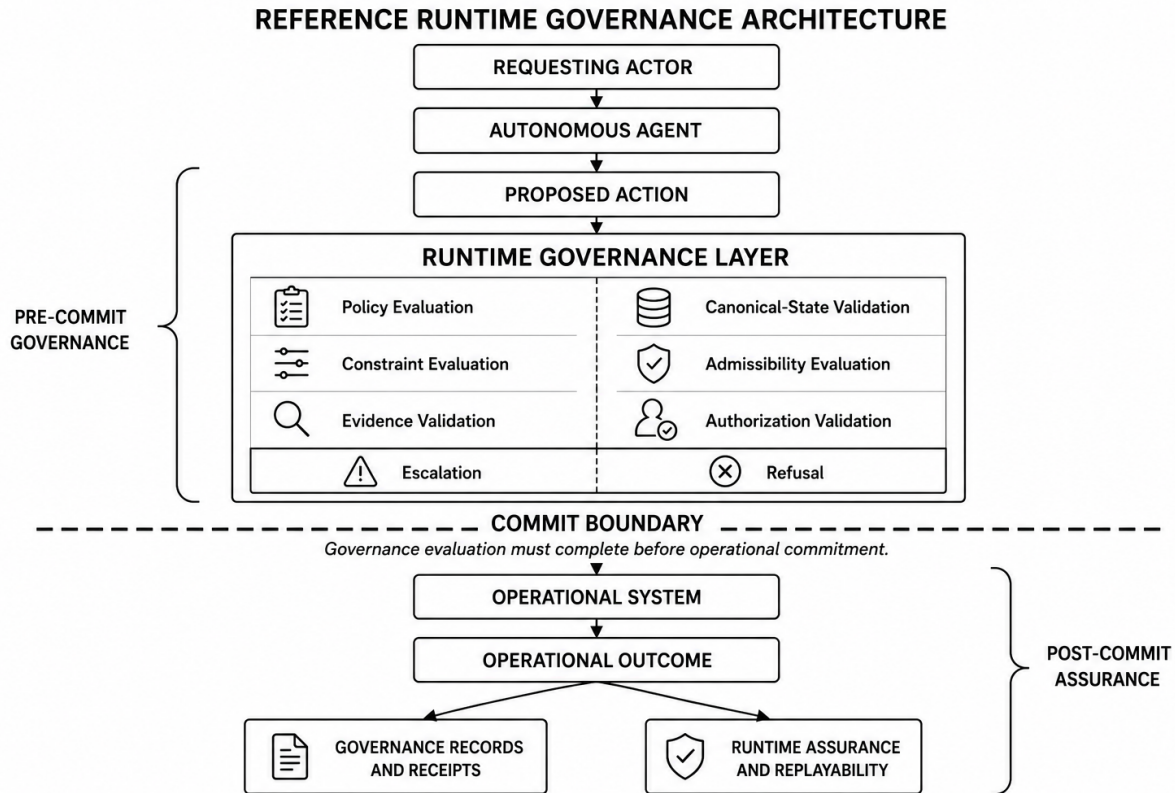


Figure 7: Reference Runtime Governance Architecture

Figure 7 presents a conceptual reference architecture for runtime governance. The architecture illustrates the separation between governance evaluation and operational execution while maintaining a controlled relationship between the two through the commit boundary. Proposed actions originate within the operational environment but must pass through governance evaluation before execution is permitted. Governance records and assurance functions provide accountability, traceability, and evidentiary support for governance decisions after execution has occurred. The components discussed below represent one possible decomposition of these responsibilities into functional architectural elements.

F.3 Architectural Components

F.3.1 Requesting Actor

The requesting actor represents the originator of a proposed action.

Examples include:

- human users
- software applications

- orchestration platforms
- autonomous agents
- external systems

The requesting actor may possess direct authority, delegated authority, or no authority depending upon the operational context.

F.3.2 Autonomous Agent

The autonomous agent represents the system responsible for planning, coordinating, recommending, or performing actions.

Examples include:

- conversational agents
- workflow agents
- orchestration agents
- planning agents
- execution agents
- multi-agent systems

The architecture does not assume any particular AI implementation model.

F.3.3 Proposed Action

The proposed action represents an intended operational outcome.

Examples include:

- tool invocation
- workflow execution
- configuration modification
- resource allocation
- financial transaction
- command execution
- physical-world operation

The proposed action becomes the object evaluated by the governance layer.

F.3.4 Runtime Governance Layer

The runtime governance layer evaluates whether a proposed action may proceed.

The governance layer performs governance functions such as:

- policy evaluation
- authorization validation
- evidence validation
- canonical-state validation
- admissibility determination
- escalation processing
- refusal processing

These governance functions correspond to execution-time evaluation activities commonly associated with secure systems engineering, cyber-resilient system design, runtime assurance, and governance architectures intended to preserve operational legitimacy prior to execution (NIST, 2018b, 2021; Rushby, 2008; Goodloe and Pike, 2010; Willis, 2026a).

This layer represents the primary focus of the runtime governance model described throughout this paper.

F.4 Illustrative Governance Evaluation Pipeline

A representative governance evaluation sequence may include:

1. Identify applicable governance requirements.
2. Validate governance authority.
3. Gather supporting evidence.
4. Verify evidence integrity and provenance.
5. Determine authoritative operational state.
6. Evaluate admissibility.
7. Determine authorization validity.
8. Escalate or refuse when required.
9. Produce governance decision.
10. Proceed to commit evaluation.

The specific implementation details may vary across organizations and governance architectures.

F.5 Commit Boundary

The commit boundary represents the transition point at which a proposed action becomes operationally consequential (Leveson, 2011; NIST, 2018b; Willis, 2026e).

The commit boundary is significant because governance decisions made after this point may be unable to prevent the action from occurring.

Examples include:

- command execution
- transaction submission
- workflow initiation
- system modification
- external API invocation
- physical-world actuation

The concept of the commit boundary provides a useful architectural location for execution-time governance controls.

F.6 Operational System

The operational system represents the environment affected by execution.

Examples include:

- enterprise applications
- cloud services
- industrial systems
- financial platforms
- healthcare systems
- critical infrastructure

Runtime governance seeks to influence execution before actions affect operational systems.

F.7 Governance Records and Receipts

Governance decisions should generate durable records supporting:

- accountability

- auditability
- traceability
- replayability
- assurance activities

Representative governance records may include:

- authorization decisions
- evidence references
- admissibility outcomes
- escalation records
- refusal records
- execution outcomes

These records provide the foundation for governance replayability and assurance activities.

The preservation of governance records is consistent with accountability, auditability, and assurance objectives found throughout cybersecurity, systems-engineering, and AI governance frameworks (NIST, 2020a, 2022, 2023b).

F.8 Runtime Assurance Functions

Runtime assurance functions provide confidence that governance mechanisms continue to operate correctly throughout system operation (Rushby, 2008; Goodloe and Pike, 2010; NIST, 2021).

Examples include:

- governance replay
- governance metrology
- conformance assessment
- authorization validation
- lifecycle integrity verification
- evidence continuity verification

Runtime assurance enables organizations to evaluate governance behavior independently of operational outcomes.

F.9 Relationship to Secure-by-Design

The architecture supports Secure-by-Design objectives by ensuring that governance evaluation occurs before actions become operationally consequential (CISA, 2023; NIST, 2018b, 2023b).

Representative contributions include:

- secure defaults through structural refusal
- least privilege through authorization validation
- complete mediation through commit-bound evaluation
- accountability through governance records
- auditability through replayability
- resilience through canonical-state validation

These characteristics align execution-time governance with broader Secure-by-Design objectives.

F.10 Relationship to Runtime Governance Concepts

The architectural components described above correspond closely to the runtime governance concepts discussed throughout this paper.

Table 25: Reference Architecture Components and Runtime Governance Concepts

Architecture Component	Representative Runtime Governance Concepts
Runtime Governance Layer	Authorization, admissibility, evidence validation, canonical-state governance
Commit Boundary	Execution governance, commit-bound authorization, operational legitimacy
Governance Records	Evidence continuity, governance receipts, replayability, traceability
Runtime Assurance Functions	Governance metrology, conformance assessment, replay validation
Autonomous Agents	Multi-agent governance, delegated authority, governance continuity
Operational Systems	Execution outcomes, lifecycle governance, accountability

Table 25 illustrates that runtime governance concepts are realized through architectural responsibilities rather than through specific products, technologies, or implementation patterns. The reference architecture separates governance evaluation, commitment, execution, evidence generation, and assurance activities into distinct architectural functions while preserving their operational relationships. This separation supports architectural flexibility because individual implementations may realize these functions through different technologies while maintaining the same governance

semantics. The mapping further demonstrates that runtime governance concepts such as admissibility evaluation, execution governance, governance continuity, evidence continuity, replayability, and runtime assurance are not isolated capabilities but are distributed across multiple architectural components that collectively support governance at the commit boundary.

F.11 Reference Architecture Variations

Different runtime governance implementations may realize the architecture using different technologies, deployment models, and organizational structures.

Examples include:

- centralized governance services
- distributed governance architectures
- embedded governance engines
- policy decision points and policy enforcement points
- workflow-oriented governance systems
- multi-agent governance coordinators

Regardless of implementation approach, the core architectural objective remains consistent: ensuring that governance evaluation occurs before operationally consequential actions are permitted to proceed.

F.12 Architectural Implications for Autonomous Systems

As autonomous systems become increasingly capable of planning, coordinating, and executing actions independently, governance architectures must increasingly address execution-time concerns rather than relying solely on design-time controls.

Examples include:

- dynamic operational conditions
- changing authorization states
- delegated authority relationships
- multi-agent coordination
- cross-domain workflows
- evolving operational risk

These characteristics reinforce the importance of governance mechanisms capable of evaluating legitimacy, authorization, and admissibility at execution time rather than relying exclusively upon static pre-deployment assumptions (NIST, 2021, 2023b; Willis et al., 2026).

F.13 Summary

This reference architecture illustrates one way execution-time governance capabilities may be incorporated into autonomous and agentic systems.

The architecture intentionally separates governance evaluation from operational execution while preserving a controlled relationship between them (NIST, 2018b; Leveson, 2011; Willis, 2026b).

Although implementation approaches will vary, the architecture demonstrates how admissibility evaluation, authorization validation, canonical-state governance, structural refusal, governance continuity, replayability, and runtime assurance may be integrated into a coherent runtime governance model (NIST, 2021, 2023b; Rushby, 2008; Willis, 2026a).

Viewed in this manner, runtime governance may be understood as an execution-time architectural layer that helps connect governance intent to operational execution while supporting Secure-by-Design, assurance, accountability, and resilience objectives.

G Example Runtime Governance Sequence

G.1 Purpose

The runtime governance concepts discussed throughout this paper can appear abstract when presented solely as architectural principles, control categories, or governance requirements.

This appendix provides an illustrative execution scenario demonstrating how runtime governance may operate during an autonomous workflow.

The objective is not to prescribe a specific implementation. Rather, it is to show how execution-time governance concepts can be applied during a realistic operational decision process (NIST, 2018b, 2021, 2023b; Willis, 2026a).

G.2 Scenario Overview

Consider an autonomous energy-management system operating within a regional electrical utility.

The system is authorized to perform routine grid-balancing operations and demand-management activities.

During a period of elevated demand, the autonomous system determines that load-shedding actions may be necessary to preserve grid stability.

The proposed action is operationally significant because it may affect electrical service availability for customers.

Consequently, the action is subject to runtime governance evaluation before execution (NIST, 2018b, 2021; DHS, 2024).

G.3 Illustrative Runtime Governance Sequence

The runtime governance concepts discussed throughout this paper can be represented as a generalized execution-time governance lifecycle. Figure 8 illustrates how governance context collection, evidence validation, canonical-state determination, authorization validation, admissibility evaluation, commit-boundary governance, execution, governance recording, and runtime assurance activities interact within a single operational sequence.

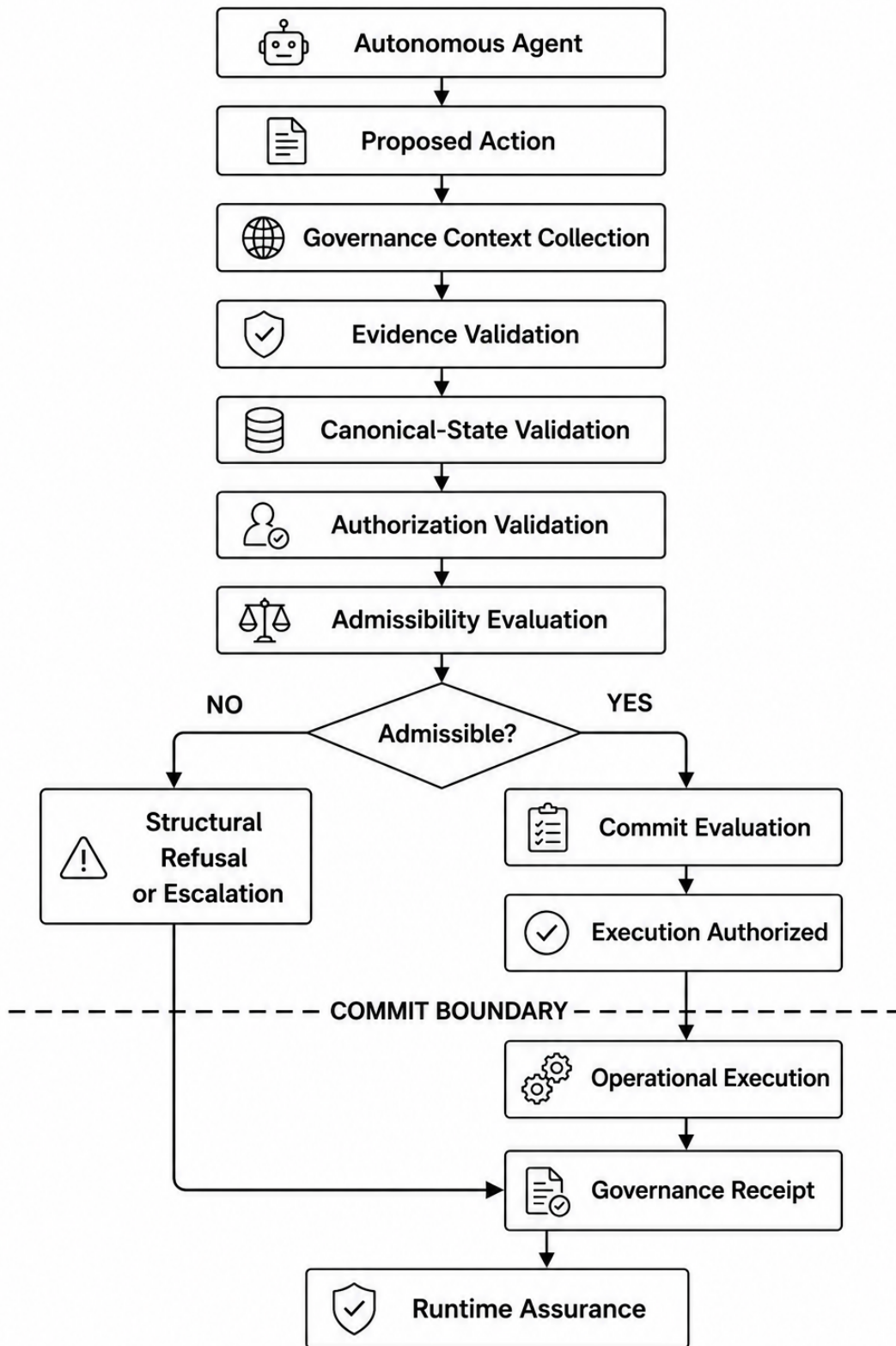


Figure 8: Illustrative Runtime Governance Sequence

The sequence illustrated in Figure 8 provides a generalized example of how runtime governance concepts may operate together during execution-time decision-making. The sequence begins with collection of governance context and supporting evidence, progresses through authorization validation and admissibility evaluation, and ultimately determines whether a proposed action should be permitted to cross the commit boundary. Where governance requirements cannot be satisfied, structural refusal or escalation mechanisms prevent execution. When execution is authorized, governance records and assurance artifacts are generated to support accountability, auditability, replayability, and independent runtime assurance activities. The steps described below examine each stage of the sequence in greater detail.

G.4 Step 1: Proposed Action Generation

The autonomous system identifies a potential operational action.

In this example:

Reduce electrical load within a specified service region by initiating predefined demand-management procedures.

At this stage the action is merely proposed.

No operational impact has occurred.

G.5 Step 2: Governance Context Collection

Governance evaluation begins by gathering relevant context.

Examples include:

- operational objectives
- current system conditions
- regulatory constraints
- authority assignments
- service-level obligations
- emergency declarations
- customer-impact policies

The resulting governance context provides the information required for subsequent evaluation activities.

G.6 Step 3: Evidence Validation

Supporting evidence is evaluated.

Examples include:

- telemetry feeds
- grid-status information
- operational forecasts
- system-health indicators
- demand projections
- emergency notifications

Evidence validation seeks to ensure that governance decisions are based upon attributable and trustworthy information (NIST, 2020a, 2022, 2023b).

G.7 Step 4: Canonical-State Validation

The governance system determines the authoritative operational state.

Questions may include:

- What is the current grid condition?
- Are telemetry sources consistent?
- Has the operational situation changed?
- Are emergency conditions active?
- Is the proposed action still relevant?

Canonical-state validation helps ensure that governance decisions are based upon current operational reality rather than stale information (NIST, 2018b, 2021; Willis, 2026e).

G.8 Step 5: Authorization Validation

The governance system verifies authority.

Questions may include:

- Is the autonomous system authorized to initiate this action?
- Are delegation chains valid?

- Have authorizations expired?
- Are additional approvals required?

Authorization validation confirms that the proposed action remains within authorized authority boundaries (NIST, 2020a,b; OASIS, 2013).

G.9 Step 6: Admissibility Evaluation

Authorization alone does not determine whether execution should occur.

The governance system evaluates admissibility.

Questions may include:

- Is the action operationally appropriate?
- Does it satisfy current constraints?
- Are required conditions present?
- Are customer-impact thresholds exceeded?
- Are alternative actions available?

Admissibility evaluation determines whether execution remains legitimate under current conditions (NIST, 2021, 2023b; Willis, 2026a).

G.10 Step 7: Refusal or Escalation Decision

If governance requirements cannot be satisfied, execution does not proceed.

Possible outcomes include:

- structural refusal
- escalation to human operators
- escalation to supervisory authorities
- request for additional evidence
- deferred execution

This stage illustrates the Secure-by-Design principle that actions should fail safely when governance conditions are not met (CISA, 2023; NIST, 2021).

G.11 Step 8: Commit Evaluation

If the action remains admissible, governance evaluation proceeds to commit determination.

Commit evaluation confirms:

- admissibility remains valid
- authorization remains valid
- required evidence remains available
- operational conditions remain acceptable

The objective is to ensure that governance conclusions remain valid immediately prior to execution (Leveson, 2011; Willis, 2026e).

G.12 Step 9: Commit Boundary Crossing

The commit boundary represents the moment at which operational consequences become possible (Leveson, 2011; NIST, 2018b; Willis, 2026e).

Examples include:

- dispatching commands
- initiating transactions
- activating workflows
- issuing control signals
- modifying operational systems

Once the commit boundary is crossed, governance has transitioned from evaluation to execution.

G.13 Step 10: Operational Execution

The approved action executes.

In the example scenario:

Demand-management procedures are initiated within the authorized service region.

Operational consequences now occur within the governed environment.

G.14 Step 11: Governance Receipt Generation

The governance system generates durable records describing the governance decision.

Representative contents may include:

- proposed action
- evidence references
- canonical-state information
- authorization information
- admissibility outcome
- execution outcome
- timestamps

These records support accountability, auditability, and replayability (NIST, 2020a, 2022, 2023b).

G.15 Step 12: Runtime Assurance Activities

Post-execution assurance activities may include:

- governance replay
- conformance assessment
- governance metrology
- evidence validation
- assurance review
- operational analysis

These activities help organizations evaluate governance effectiveness independently of the operational outcome (Rushby, 2008; Goodloe and Pike, 2010; NIST, 2021).

G.16 Governance Properties Demonstrated

The example illustrates several runtime governance concepts discussed throughout this paper.

Table 26: Runtime Governance Concepts Demonstrated by the Example

Governance Concept	Illustrative Example
Admissibility Evaluation	Determination that load shedding remains appropriate
Authorization Validation	Verification of operational authority
Canonical-State Governance	Validation of current grid conditions
Structural Refusal	Prevention of inadmissible execution
Governance Continuity	Preservation of governance context throughout workflow
Execution-Bound Authorization	Validation immediately prior to execution
Governance Replayability	Post-execution reconstruction capability
Runtime Assurance	Independent evaluation of governance behavior

This scenario demonstrates how multiple governance concepts operate together rather than independently. Authorization, admissibility, evidence validation, canonical-state determination, refusal mechanisms, governance records, and assurance activities collectively contribute to execution-time governance outcomes.

G.17 Summary

This example demonstrates how runtime governance concepts may be applied within an operational environment where autonomous actions can produce meaningful consequences.

Although implementation details will vary, the sequence illustrates a common governance pattern:

1. propose
2. evaluate
3. authorize
4. determine admissibility
5. commit
6. execute
7. record
8. assure

This pattern reflects the central thesis of this paper: as autonomous systems become increasingly capable of consequential action, governance must increasingly operate at the point where actions transition from intent to execution (NIST, 2018b, 2021, 2023b; Willis, 2026a).

The example also illustrates how Secure-by-Design objectives such as secure defaults, accountability, auditability, resilience, and human oversight may be supported through execution-time governance mechanisms operating immediately before operational consequences occur.

References

- AGCP Project. *AGCP Requirements and Conformance Framework*. Available from: <https://agcp.ai/requirements/> Accessed June 16, 2026.
- Amodei D, Olah C, Steinhardt J, Christiano P, Schulman J, Mané D. Concrete Problems in AI Safety. arXiv:1606.06565; 2016. Available from: <https://arxiv.org/abs/1606.06565>
- Anderson R. *Security Engineering*. 3rd ed. Indianapolis, IN: Wiley; 2020.
- Anthropic. *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073; 2022. Available from: <https://arxiv.org/abs/2212.08073>
- Bishop M. *Computer Security: Art and Science*. Boston: Addison-Wesley; 2018.
- Cybersecurity and Infrastructure Security Agency. *Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software*. Washington, DC: CISA; 2023. Available from: <https://www.cisa.gov/sites/default/files/2023-10/Shifting-the-Balance-of-Cybersecurity-Risk-Principles-and-Approaches-for-Secure-by-Design-Software.pdf> Accessed June 16, 2026.
- Cybersecurity and Infrastructure Security Agency. *Cross-Sector Cybersecurity Performance Goals*. Washington, DC: CISA; 2024. Available from: <https://www.cisa.gov/cross-sector-cybersecurity-performance-goals> Accessed June 16, 2026.
- Cloud Security Alliance. *Security Guidance for Critical Areas of Focus in Cloud Computing*. CSA; 2024. Available from: <https://cloudsecurityalliance.org/artifacts/security-guidance-v5> Accessed June 16, 2026.
- Committee on National Security Systems. *CNSSI 1253 Security Categorization and Control Selection*. Fort Meade, MD; 2022. Available from: <https://www.cnss.gov/CNSS/issuances/Instructions.cfm>
- Defense Innovation Board. *AI Principles: Recommendations on the Ethical Use of Artificial Intelligence by the Department of Defense*. Washington, DC; 2019. Available from: https://www.ai.gov/wp-content/uploads/2024/12/DIB_AI_PRINCIPLES_PRIMARY_DOCUMENT.pdf
- Department of Homeland Security. *Roles and Responsibilities Framework for Artificial Intelligence in Critical Infrastructure*. Washington, DC: DHS; 2024. Available from: <https://www.dhs.gov/publication/roles-and-responsibilities-framework-artificial-intelligence-critical-infrastructure>
- European Union Agency for Cybersecurity. *Multilayer Framework for Good Cybersecurity Practices for AI*. ENISA; 2023. Available from: <https://www.enisa.europa.eu/publications/multilayer-framework-for-good-cybersecurity-practices-for-ai> Accessed June 16, 2026.
- European Union Agency for Cybersecurity. *Threat Landscape for Artificial Intelligence Systems*. ENISA; 2024. Available from: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2024> Accessed June 16, 2026.
- European Union. *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act)*. Official Journal of the European Union. Brussels, Belgium; 2024. Available from: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>

- Executive Office of the President. *Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence*. Washington, DC; 2023. Available from: <https://www.federalregister.gov/documents/2023/11/01/2023-24283/safe-secure-and-trustworthy-development-and-use-of-artificial-intelligence>
- Goodloe AE, Pike L. *Monitoring Distributed Real-Time Systems: A Survey and Future Directions*. NASA/CR-2010-216724. Hampton, VA: NASA Langley Research Center; 2010. Available from: <https://ntrs.nasa.gov/citations/20100027427>
- International Organization for Standardization. ISO/IEC 42001 Artificial Intelligence Management Systems. Geneva: ISO; 2023. Available from: <https://www.iso.org/standard/81230.html> Accessed June 16, 2026.
- Leike J, Martic M, Krakovna V, Ortega P, Everitt T, Lefrancq A, et al. Scalable Agent Alignment via Reward Modeling. arXiv:1811.07871; 2018. Available from: <https://arxiv.org/abs/1811.07871>
- Leveson N. *Engineering a Safer World*. MIT Press; 2011.
- MITRE Corporation. *MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems*. McLean, VA: MITRE; 2024. Available from: <https://atlas.mitre.org/> Accessed June 16, 2026.
- Morris MR, Sohl-Dickstein J, Fiedel N, Warkentin T, Dafoe A, Faust A, Farabet C, Legg S. Levels of AGI for Operationalizing Progress on the Path to AGI. Proceedings of the 41st International Conference on Machine Learning; 2024. Available from: <https://doi.org/10.48550/arXiv.2311.02462>
- National Institute of Standards and Technology. *Guide for Conducting Risk Assessments*. Special Publication 800-30 Rev. 1. Gaithersburg, MD: NIST; 2012. Available from: <https://doi.org/10.6028/NIST.SP.800-30r1>
- National Institute of Standards and Technology. *Risk Management Framework for Information Systems and Organizations*. SP 800-37 Rev. 2. Gaithersburg, MD: NIST; 2018. Available from: <https://doi.org/10.6028/NIST.SP.800-37r2>
- National Institute of Standards and Technology. *Systems Security Engineering*. Special Publication 800-160 Vol. 1. Gaithersburg, MD: NIST; 2018. Available from: <https://doi.org/10.6028/NIST.SP.800-160v1>
- National Institute of Standards and Technology. *Security and Privacy Controls for Information Systems and Organizations*. Special Publication 800-53 Rev. 5. Gaithersburg, MD: NIST; 2020. Available from: <https://doi.org/10.6028/NIST.SP.800-53r5>
- National Institute of Standards and Technology. *Zero Trust Architecture*. Special Publication 800-207. Gaithersburg, MD: NIST; 2020. Available from: <https://doi.org/10.6028/NIST.SP.800-207>
- National Institute of Standards and Technology. *Developing Cyber-Resilient Systems*. Special Publication 800-160 Vol. 2 Rev. 1. Gaithersburg, MD: NIST; 2021. Available from: <https://doi.org/10.6028/NIST.SP.800-160v2r1>

- National Institute of Standards and Technology. *Security and Privacy Controls Assessment Procedures*. SP 800-53A Rev. 5. Gaithersburg, MD: NIST; 2022. Available from: <https://doi.org/10.6028/NIST.SP.800-53Ar5>
- National Institute of Standards and Technology. *AI RMF Playbook*. Gaithersburg, MD: NIST; 2023. Available from: https://airc.nist.gov/docs/AI_RMF_Playbook.pdf Accessed June 16, 2026.
- National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. Gaithersburg, MD: NIST; 2023. Available from: <https://doi.org/10.6028/NIST.AI.100-1>
- National Institute of Standards and Technology. *Cybersecurity Framework (CSF) 2.0*. Gaithersburg, MD: NIST; 2024. Available from: <https://www.nist.gov/cyberframework>
- National Institute of Standards and Technology. *Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: NIST; 2024. Available from: <https://doi.org/10.6028/NIST.AI.600-1>
- National Institute of Standards and Technology. AI RMF Resource Center. Gaithersburg, MD: NIST; 2025. Available from: <https://airc.nist.gov/> Accessed June 16, 2026.
- National Security Commission on Artificial Intelligence. *Final Report*. Washington, DC; 2021. Available from: <https://apps.dtic.mil/sti/html/trecms/AD1124333/>
- OASIS. *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard; 2013. Available from: <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
- Open Group. *TOGAF Standard*. 10th ed. Reading, UK; 2022. Available from: <https://www.open-group.org/togaf> Accessed June 16, 2026.
- OpenAI. *GPT-4 Technical Report*. arXiv:2303.08774; 2023. Available from: <https://arxiv.org/abs/2303.08774>
- OpenAI. Model Spec. OpenAI Technical Documentation; 2025. Available from: <https://model-spec.openai.com/> Accessed June 16, 2026.
- OWASP Foundation. *OWASP Agentic Security Initiative*. OWASP; 2025. Available from: <https://genai.owasp.org/initiatives/agentic-security-initiative/> Accessed June 16, 2026.
- Park JS, O'Brien J, Cai CJ, Morris M, Liang P, Bernstein M. Generative Agents: Interactive Simulacra of Human Behavior. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST); 2023. Available from: <https://doi.org/10.1145/3586183.3606763>
- Rushby J. The Design and Verification of Secure Systems. Proceedings of the 8th ACM Symposium on Operating Systems Principles (SOSP); 1981. Available from: <https://doi.org/10.1145/800216.806586>
- Rushby J. Runtime Certification. In: Leucker M, editor. *Runtime Verification: 8th International Workshop, RV 2008, Budapest, Hungary, March 30, 2008, Selected Papers*. Lecture Notes in Computer Science, vol. 5289. Berlin: Springer; 2008. p. 21–35. Available from: https://doi.org/10.1007/978-3-540-89247-2_2

- Russell S. *Human Compatible: Artificial Intelligence and the Problem of Control*. New York: Viking; 2019.
- Saltzer JH, Schroeder MD. The Protection of Information in Computer Systems. *Proceedings of the IEEE*. 1975;63(9):1278–1308. Available from: <https://doi.org/10.1109/PROC.1975.9939>
- Seshia SA, Sadigh D, Sastry SS. Toward Verified Artificial Intelligence. arXiv:1606.08514; 2016.
- Seshia SA, Desai A, Dreossi T, Fremont DJ, Ghosh S, Kim E, Shivakumar S, Vazquez-Chanlatte M, Yue X. Formal Specification for Deep Neural Networks. In: *Proceedings of the International Symposium on Automated Technology for Verification and Analysis (ATVA)*; 2018. p. 20–34. Available from: https://doi.org/10.1007/978-3-030-01090-4_2
- Wang L, Ma Y, Zhang Q, Zhang X. A Survey on Large Language Model Based Autonomous Agents. arXiv:2308.11432; 2023.
- Willis JM. *Runtime Execution Governance for AI Systems: A Cross-Platform Synthesis and Architectural Framework*. Sustainable Future Tech, Inc.; 2026. Available from: <https://doi.org/10.5281/zenodo.19341177>
- Willis JM. *Runtime Governance Architecture: Consistent Governance Execution for Enterprise Systems and Autonomous Agents*. Sustainable Future Tech, Inc.; 2026. Available from: <https://doi.org/10.5281/zenodo.19286845>
- Willis JM. *AGCP: A Deterministic Execution-Layer Governance Control Plane for Autonomous and Programmatic Systems*. Sustainable Future Tech, Inc.; 2026. Available from: <https://doi.org/10.5281/zenodo.19323672>
- Willis JM. *Artificial Intelligence Governance Control Plane (AGCP) Specification v1.0.0*. Sustainable Future Tech, Inc.; 2026. Available from: <https://doi.org/10.5281/zenodo.20297616>
- Willis JM. *Formal Execution Semantics and Safety Invariants for Governance Control Planes in Autonomous Systems*. Sustainable Future Tech, Inc.; 2026. Available from: <https://doi.org/10.5281/zenodo.19241732>
- Willis JM, et al. *The PBSAI Governance Ecosystem: A Multi-Agent AI Reference Architecture for Securing Enterprise AI Estates*. arXiv; 2026. Available from: <https://arxiv.org/abs/2602.11301>
- Xi Z, Chen W, Guo X, Wang W, Cheng H., et al. The Rise and Potential of Large Language Model Based Agents. arXiv:2309.07864; 2023. Available from: <https://arxiv.org/abs/2309.07864>